uc3m | Universidad Carlos III de Madrid

Grado Universitario en Ingeniería Informática

2020-2021

Trabajo Fin de Grado

"Creación de un videojuego de plataformas procedural con Unity3D"

Miguel Ángel Torrijos González

Tutora

Susana Fernández Arregui

Leganés, julio 2021



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento – No Comercial – Sin Obra Derivada**

Resumen

En el presente documento se detalla el proceso de diseño y desarrollo de un videojuego *rogue-like* de plataformas con generación aleatoria de los escenarios. El mundo de los videojuegos es muy extenso y continúa creciendo. Actualmente se define como uno de los sectores de entretenimiento líder en el globo por su gran capacidad para contar y transmitir historias.

El género de plataformas engloba muchas maneras de crear, dejando mucha libertad al desarrollador y es uno de los géneros más influyentes del mundo contando con obras tan relevantes como Super Mario Bros. Por su parte el género *rogue-like* simula la exploración de mazmorras generando salas distintas a la de la partida anterior haciendo uso de la generación procedural.

En este trabajo se realizará un breve resumen del mundo de los videojuegos centrándose en aquellos elementos más relacionados con este proyecto. Se analizarán los géneros principales del juego mostrando ejemplos y similitudes con otros juegos del mercado y se dará una introducción a la historia de este medio.

Habiendo analizado el problema se detallará el proceso de creación de un juego de plataformas al que se ha denominado Beans Out. Desde el diseño de todos los elementos como la trama y el estilo hasta la implementación de los puntos más importantes. Además de las pruebas llevadas a cabo para comprobar el correcto funcionamiento del producto. Se implementarán los cimientos sobre los que se sustentará el resto de ampliaciones o contenido del juego, sin pretender terminar el juego al completo.

Para finalizar, se concluirán los objetivos cumplidos a lo largo del proyecto y los resultados obtenidos. Comentando el posible trabajo para el futuro que se podría realizar. Seis meses de trabajo y casi 400 horas de trabajo han dado como resultado este juego, Beans Out, disponible para Windows al que se le continuará añadiendo contenido hasta llegar a terminar el juego completo y publicarlo de forma gratuita.

Palabras clave: Videojuego; Unity 3D; Plataformas; Rogue-Like; Generación procedural

Abstract

Computing and programming have evolved heavily in the past years. A lot of everyday objects use this knowledge to make our life easier. This also applies in some of the greatest discoveries of the last decades. From been able to see and communicate with each other's through the internet to been capable of getting an image of a black hole. All this functions that we take for granted nowadays depend on computing.

However, the scientific field is not the only one affected and the entertainment media have benefit from this to. The video games industry is possibly the best example for this. This document details the steps taken to carry out the development of a platform video game in Unity 3D.

MOTIVATIONS AND OBJECTIVES

Since I was little, my parents introduced me to some video games. For as long as I can remember, a game was with me. Firstly, with child's games to learn and practice for school and later with some more deep pieces of art, I discovered that this genre is something as powerful as cinema or painting.

That possibility to affect someone's life that won't ever meet you was the feeling that push me into creating my own video games. After coding some short games, I decided that I wanted to call this my job in the future. Later on, a friend of mine contact me and tells me an idea that he has, a video game. This was the final push I needed to start my first real video game. So, I started researching and asking some teachers and decided to make this game my final degree project.



Figure 1: Dead Cells, Motion Twin.

The idea for this project was to design the final game with all the principal components; the main character, the enemies, the plot and the setting and implement those most important. In the end we will have the foundations of the game, so if in the future we want to add more content we could do it easily because we have all the basics components covered. With this project I will learn how to use the Unity 3D development engine and C# programming.

With this been said, the objectives of the work are the following:

- Design the game entirely, including the plot and the style.
- Implement the player movement searching for a fluent sensation of play.
- Creation of procedurally generated maps.
- Implementation of the enemies' behaviours. Each enemy must feel and act different.
- Compose a simple and easy to understand interface for the user.
- Conduct a testing phase with a group of people who provide feedback and report bugs while playing the game.

There are some other objectives that are not as relevant so it's not mandatory that are finished for this project but it will be a good addition to the final result. These are; make the music and sounds for the game, create 3D models for the characters, create animations and translate the game to English.

STATE OF THE ART

The video game as a media has become one of the largest in the world surpassing other traditional media like sports or cinema. It is worth highlighting other methods of consuming these products are also gaining prominence, like the streaming platform Twitch, where you can see other people playing.

The history of video games has been a short but full of advances journey. Since the Second World War where all the progress made where in favour of developing a better way to encrypt messages, all computing advances have been a boost for this sector. In 1972, Nolan Bushnell founded Atari, one of the most relevant business in the video game history. Atari created Pong, commonly known as the first video game ever. This company will go on to create some of the genres of video games that persist nowadays and some of the most iconic console in recent history, such as the Atari 2600.

As the time passed, other companies appeared because of the rapid growth that the industry was experimenting. Some of them persist even in these years and are some of the most

important businesses in the world. Microsoft, with his Xbox, Sony with his PlayStation or Nintendo with Wii and Nintendo Switch are some of the biggest names in the industries right now.



Figure 2: Super Mario 64, Nintendo

This industry is expanding at an accelerated pace and in these days, we are seeing a new change. This decade has been dominated by independent games. Like Minecraft, some of them have become one of the greatest games of all time surpassing other ones with bigger teams and budget. These games were made by groups of very few people or even just by one. This was made it possible thanks to all the information that became available in the recent years.

The roguelike genre is based on the idea of exploring something new every time you play. This is made possible by generating maps randomly and making the player have to complete the game without dying. Every time the player dies, he has to start from scratch. Some games add the possibility to upgrade your character between different attempts of beating the game so the user can see a progression. These elements make the user play a different situation each time a new game start.

The platform genre simply defines how the player is going to move because it gives a lot of importance to the jump. Both genres leave a lot of room for creativity so they are a very strong choice.

ANALYSIS OF THE SITUATION

The name decided for this project was Beans Out. It sounds simple and catchy so it was a very good option. The first thought was to define the most relevant elements of the game.

The game is about a bean's society. This community lives in a jar but they need to plant one of them to carry on with the species. The player incarnates one of the beans that have been trained to explore the house and his mission is to reach the garden and plant himself. On his way, the player will encounter some hazards. A lot of insects will come chasing the bean to eat him, so the player must defend himself. For that purpose, two skills were thought. The first one, allow the player to dash in one direction very fast during a short amount of time. The second one allows the player to shoot a photosynthesis powered bullet. With these skills the player can fight with these insects and find the way to the garden. If the bean dies on his way, another one will take his place and starts again from the beans' jar.

In this way, the basic mechanics of the roguelike genre are united with the plot of the game. The platforms genre defines how the map will be.

For the enemies, insects were thought, since they were consistent with the size of the main character. This also allow to add more enemies anytime because there are so many insects that can be added like cockroach or mosquitoes. For this project, spiders, ants and flies were proposed as the main enemies.

The map must be made randomly, so it can't be predefined. Template rooms must be made in order to use them to form the final map. The more templates are created, the better results it will give, so later on, a lot of care and effort was forced into making and designing these templates

The tools used for this project are; Udemy, Unity 3D, Blender, Atom, Adobe Photoshop, Premier and After Effects, Unity User Manual, Unity Forum, Google Books, Forms and Scholar, Microsoft Excel and Word and Lucidchart.

DESING

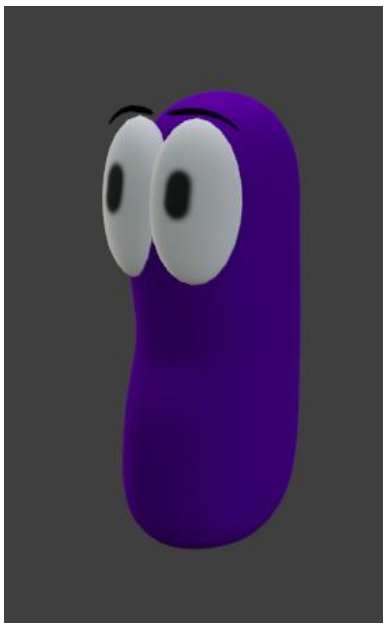


Figure 3: Model for the main character

In this phase, all the important elements of the game were design. The design phase was written up like a Game Design Document (GDD), so it doesn't have explicit requirements. Instead of that, those requirements are implicit in the text.

Beginning with the player, it has four basic abilities. The first one is the horizontal movement. This one allows the player to move to the right or to the left. The second one is the vertical movement. This one is the capacity of the player to jump. Those two abilities form the basic movement of a platform game. The remaining two have already been mentioned above. They were; triggering and dash. Triggering is the ability by which the player can launch a projectile in one direction. That projectile will continue at constant velocity till it goes far enough from the player. If the projectile hits something, it will destroy itself. If that thing is an enemy, it will make a fixed amount of damage to that enemy before destroying itself. The last ability, dash, makes the player run faster for a short amount of time. All characters in the game have three main attributes; health, speed and damage. For the bean, those values are:

- Health: 100 points
- Speed: 10 cm/s
- Damage: 10 points

Some secondary characters were designed as well. These were the Beans' King and chickpea. The beans' king is the oldest bean in the jar, so it is treated as the king of their society. This character is meant to be on the beans' jar and tell the player about the game's story. Chickpea on the other way, is meant to be found while playing randomly through the map. This character will tell the player various information about the game and its story. It will function like a helping hand for players giving then some advice and small picks of the whole game narrative. These two characters combined will give the player the amount of information necessary to feel that they are understanding the game but also that they need to know more. Because of this, the player will likely begin another attempt of beating the game.

On the other hand, we have the enemies' characters. They consist of three insects; the ant, the spider and the fly. Each one of them have a unique set of abilities to make it feel different to the rest.

The ant will go from left to right until it finds the end of a platform. In that case, the ant will turn 180 degrees and continue in that direction. If the player is in front of the ant, it will accelerate to catch the player but it will never change direction unless it reaches the end of the platform. The attributes for the ant are the following:

- Health: 50 points
- Speed: 4 cm/s
- Damage: 10 points

The fly is one of the simplest enemies because of its ability to fly. This makes it possible to the fly to move through the map easily. The direction to the enemy will be calculated to know where the fly must move and then it will check if there are walls on its way. If there is a wall on its way the fly won't move on that direction. The attributes for the fly are the following:

- Health: 15 points
- Speed: 7 cm/s
- Damage: 15 puntos

The spider is the most complex enemy because of its abilities to travel the map. First, the ant will walk on a straight line in the direction of the player. This is the horizontal movement, like the player, it can walk to the right or to the left. The second ability is the vertical jump. This one allows the spider to ascend an upper platform or descend an under one. Finally, the spider can jump between platforms at the same height. This is called the horizontal jump. If, while chasing the player, the spider encounters a gap between two platforms it will perform a jump similar to the player one and land on the other platform. The attributes for the spider are the following:

- Health: 35 points
- Speed: 5 cm/s
- Damage: 10 points

The main goal of this game will be to kill all enemies. In the final version of the game this process will be much harder because more than one map with enemies will be needed to finish to reach the goal.

The player will interact with the game using the keyboard and the mouse. Mouse will be used to interact with menus and keyboard to control the main character.

Die in this game doesn't mean the game was lost. When the player dies, another bean starts a new attempt so if the player keeps trying, he won't lose ever.

For the maps, room templates are needed. Due to the way maps are generated, the more templates are design, the better result it will be obtained. For this version, at least 30 rooms are needed.

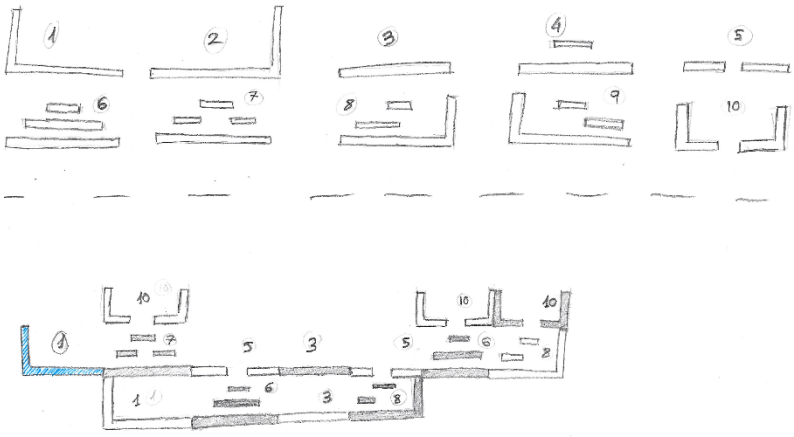


Figure 4: First design for the room's templates

The game will be made for a 1920x1080 resolution. It will be able to scale to other resolutions in the 16:9 ratio. A frame rate of at least 30 will be sought but the preferred value is something around 60. Less than 30 frames per second makes a bad sensation for the player so it won't be acceptable. The game will be coded in C# and will be compatible with Windows. Other operating systems haven't been selected due to the lack of a terminal with that OS to run the tests necessary.

APPLICATION DEVELOPMENT

The implementation of the game was made with Unity. Contrary to what usually happens, in Unity doesn't exists a main script with all the functions of the game. Unity permits the developer to add small scripts with some specific functions to the game objects in the scene. With this method, the scripts can be reused in different elements of the game, saving time of making specific methods for every element. In the following diagram, every class represents a script attached to a game object.

Those classes can be grouped in three different groups depending on the game object that they relate the most. But there is one script that doesn't belong in any of that groups. The Game Manager is the one in charge of controlling the game execution. There are some other scripts, that are attached to the enemies, that can be used in more situations, but for the moment they only appear in those elements.

First, the classes associated with the player will be discussed. The player consists of five scripts. One for every ability he has, another for controlling his health and another use only in one specific place that changes the controls of the player. The first one gives the player the capacity to walk to left or to the right and to jump. To activate the Triggering effect, PlayerShootController has been made. On this script, the input of the user is capture and if the design key is pressed, a bullet is launch in the direction the player is looking. PlayerDashController controls the Dash ability of the player. When the design key is pressed, the player is propelled in the direction he’s facing. This is made with the Unity physics engine.

A health controller script is also attached to the player to keep this attribute supervised. The main function of this class is to alert the Game Manager when the player has died. This scripts then, resets the game to the main menu.

For the enemies, the same structure is repeated. Every enemy in the game consist of three main classes. One to control his health, one that gives them their abilities and movement and another one that makes them a hazard by making damage to the player if he touches them. The first one is very similar to the player’s health controller but in this case, when the enemy dies, it’s destroyed and alerts the Game Manager to update the remaining enemy counter. The second one is specific for each type of enemy and the third uses the physics engine to detect collisions and apply damage if it’s with the player.

For all enemies, the movement controller function in different ways following the design of the enemy but all of them share one feature. All of the enemies need to perceive the map around them to know how to move. This is made by the Unity method RayCast that projects a ray in one direction and returns if it has hit anything or not. Using this rays,

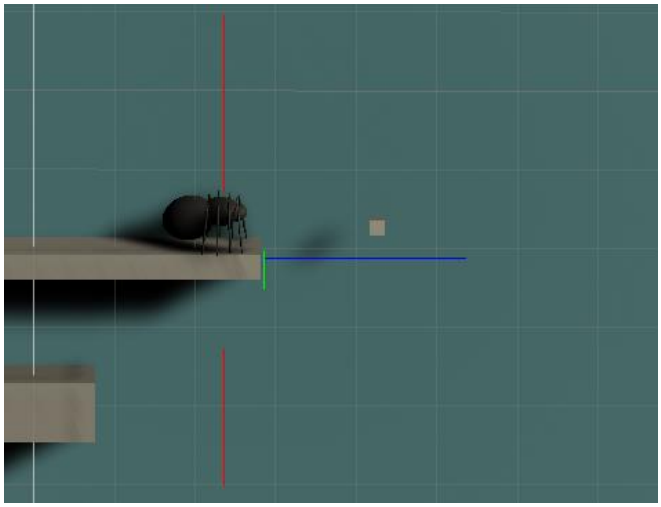


Figure 5: Rays used by the spider

end of platforms, upper or lower platforms or walls can be perceived.

Last but not least are the classes used to generate the map. The RoomTemplates class stores all the possible rooms according to their openings. There are four different groups, top rooms, bottom, left and right rooms. One room can be in several groups at once. To create the rooms, the RoomSpawner class is used. This script is attached to every template room on its open sides. When the scene is loaded, the script selects from the list of possible rooms randomly and instantiate that choice. That room that has been created, could have other open sides so this process continue until no more rooms can be created.

Because of this, the maps generated could be extremely big or small and that's something we don't want for the game so, the AddRoomToList class was created. This class stores all the rooms that have been created and checks the size of that list every time a new one is instantiated. With this method we can reset the creation of the map if the number of rooms is bigger that an upper limit. When the creation of the entire map is completed, the size of the final map is checked and if its less than the minimum, the creation resets as well.

Finally, the InstantiateEnemies class is in charge of spawning al the enemies once the level is finished. This is made by collecting all the enemies spawn points that the rooms have and creating a random enemy on that position with the following probabilities:

- Ant: 14'28 % (1/7)
- Fly: 14'28 % (1/7)
- Spider: 14'28 % (1/7)
- Nothing: 57'14 % (4/7)

When all the map is generated, the Game Manager alerts the enemies to start moving and positions the player in the start room.

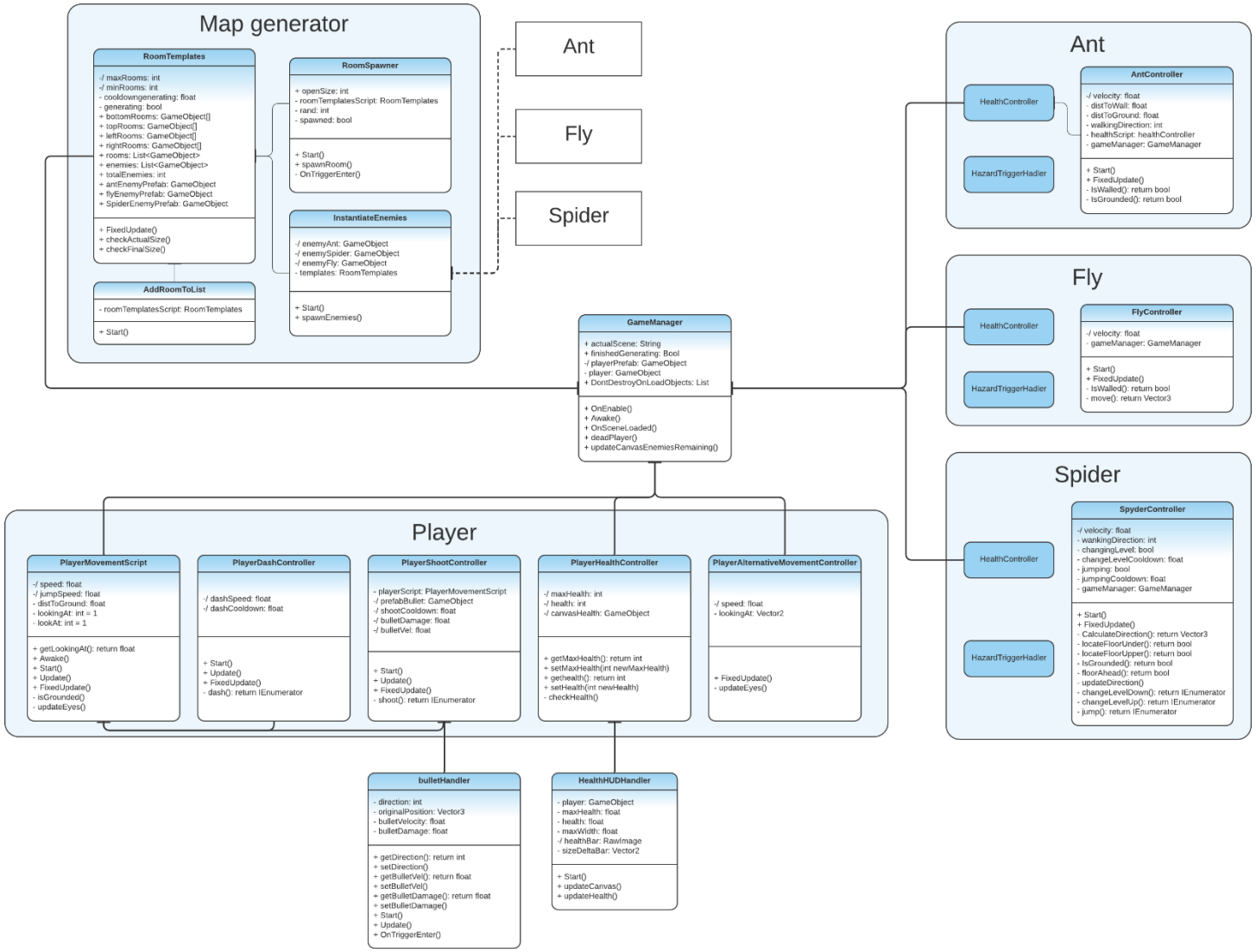


Figure 6: Class diagram

To test the correct functioning of the application, two sets of tests were made. The first one was a set of internal tests performed when the version of the game was finished. This testing set was design to proof every aspect of the game, but this task is really complicated because the large number of possibilities that can appear.

Because of this, a second set of tests were made. This one consists of a group of volunteers that play the game and report the different errors they find. In this form, a total of 30 reports appeared. With this information, a list of errors and its severity was made to arrange those failures and know what were the most important.

PLANNING AND BUDGETING

For the purpose of keeping track of the project advancements and the milestones reached, a Gantt diagram was made. This diagram is divided in five phases; problem planning, application design, solution implementation, tests and writing. The project was predicted to last around six months. In the end, the individual times for each phase shifted by a week or some days but the whole project lasted what was expected.

To calculate the budget, a calculation of the hours worked on the project, the material and indirect costs was carried out. After applying taxes, a final budget of 9.845’3 euros was obtained.

For this project, no current legislation on the benefits obtained enters into force since the game intends to be published in a free-to-play format. The data protection regulation was taken into account when making all the forms throughout the work. Participants may request that their data be deleted from the database at any time and will only be stored for one year.

CONCLUSIONS

During the extension of this project, a demo of a game with all the important elements that compose it has been analysed, designed, implemented and tested: a character controlled by the user with all their abilities, a group of enemies with different and unique behaviours and a system capable of generating totally random scenarios. In addition to studying the state of the art on the field in which this project is included.

Regarding the objectives that were defined at the beginning of the document, we can confirm that all those that were declared as main have been met. The complete set has been designed detailing elements such as plot and style. Player behaviours and user interaction have been implemented. Surveys were conducted with a group of volunteers to find out what the public thought about the game and the results were satisfactory and in accordance with those expected. Three different types of enemies have been implemented with fully autonomous behaviours capable of moving around a platform scenario. A method has been created capable of generating scenarios randomly in a consistent way and adjusting parameters such as the maximum number of rooms that could appear. A simple interface has also been designed that displays relevant information. Finally, these elements have been verified both with internal and external tests with a group of volunteers.

Agradecimientos

Agradecer a todas aquellas personas que me han acompañado durante la realización de este trabajo. Primero a Susana por apoyarme y guiarme en este proyecto. Por otro lado, a mi familia quienes estuvieron ahí en todo momento dispuestos a ayudar. A mis amigos de carrera que se sumaron desde el día uno a esta loca idea de crear un videojuego. A mis amistades del instituto y primaria que se mantuvieron ahí a pesar del tiempo y me apoyaron, en especial a Samuel por compartir conmigo la idea original de este proyecto. Agradecer también a todos los voluntarios que ayudaron en este trabajo probando el juego y a todos aquellos desarrolladores de videojuegos que me enseñaron lo que se podía hacer en este medio. Gracias.

Don't lose your way

– Kill la Kill

Índice de Contenido

1. INTRODUCCIÓN 1

1.1. MOTIVACIÓN 1

1.2. OBJETIVOS 2

1.3. ESTRUCTURA DEL DOCUMENTO 3

2. ESTADO DEL ARTE 4

2.1. EL VIDEOJUEGO COMO MÉTODO DE ENTRETENIMIENTO 4

2.2. LA HISTORIA DEL VIDEOJUEGO 5

2.2.1. Década de 1950 5

2.2.2. Década de 1960 6

2.2.3. Década de 1970 6

2.2.4. Década de 1980 7

2.2.5. Década de 1990 8

2.2.6. Década de 2000 8

2.2.7. Década de 2010 y actualidad 9

2.3. JUEGOS INDEPENDIENTES 9

2.4. PLATAFORMAS Y GÉNEROS 10

2.5. HERRAMIENTAS PARA LA CREACIÓN DE VIDEOJUEGOS 11

2.5.1. Motores de videojuegos 11

2.5.1.1. CryEngine 3 12

2.5.1.2. Source 2 12

2.5.1.3. Unreal Engine 4 13

2.5.1.4. Unity 5 13

2.5.2. Herramientas de modelado 3D 13

2.6. DOCUMENTOS AUXILIARES 14

3. ANÁLISIS DE LA SITUACIÓN 15

3.1. DECISIONES INICIALES 15

3.2. JUGABILIDAD 16

3.3. ELEMENTOS IMPORTANTES 17

3.3.1. Jugador 17

3.3.2. Enemigos 18

3.3.3. Escenario 18

3.3.4. Interfaz 18

3.4. HERRAMIENTAS USADAS 18

3.4.1. Udemy 18

3.4.2. Unity 3D 19

3.4.3. Blender 19
3.4.4. Atom 19
3.4.5. Adobe Photoshop 19
3.4.6. Adobe Premier y After Effects 20
3.4.7. Unity User Manual 20
3.4.8. Unity Forum 20
3.4.9. Stack Overflow 20
3.4.10. Google Forms 20
3.4.11. Google Books y Scholar 20
3.4.12. Microsoft Word y Excel 20
3.4.13. Lucidchart 21
3.4.14. Open Broadcaster Studio (OBS) 21

4. DISEÑO 22

4.1. PERSONAJES 22
4.1.1. Aliados 22
4.1.2. Enemigos 24
4.1.2.1. Hormiga 24
4.1.2.2. Mosca 26
4.1.2.3. Araña 27
4.2. HISTORIA 29
4.2.1. Tono 29
4.3. PROGRESIÓN 29
4.4. GAMEPLAY 29
4.4.1. Meta 29
4.4.2. Interacción del usuario 30
4.4.3. Mecánicas 30
4.4.4. Perder 31
4.5. ESCENARIOS 32
4.6. ESTILO DE ARTE 32
4.7. DESCRIPCIÓN TÉCNICA 33

5. DESARROLLO DE LA APLICACIÓN 34

5.1. DEFINICIONES Y PALABRAS CLAVE 34
5.2. IMPLEMENTACIÓN DE BEANS OUT 35
5.2.1. General 36
5.2.2. Personaje 38
5.2.2.1. PlayerMovementScript 38
5.2.2.2. PlayerHealthController 39
5.2.2.3. PlayerDashController 39
5.2.2.4. PlayerShootController 40

5.2.2.5. PlayerAlternativeMovementController 41
5.2.3. Enemigos 41
5.2.3.1. Hormiga 41
5.2.3.2. Mosca 42
5.2.3.3. Araña 43
5.2.4. Escenario 44
5.3. FUNCIONAMIENTO DE BEANS OUT 49
5.4. PRUEBAS 53
5.4.1. Pruebas internas 54
5.4.2. Pruebas externas 60
5.4.2.1. Errores reportados 61
5.4.2.2. Resultados obtenidos 64
6. PLANIFICACIÓN Y PRESUPUESTO 69
6.1. PLANIFICACIÓN 69
6.2. PRESUPUESTO 74
6.2.1. Costes de personal 74
6.2.2. Costes materiales 74
6.2.3. Costes indirectos y otros costes 75
6.2.4. Coste final 75
6.3. IMPACTO SOCIO-ECONÓMICO 76
6.4. MARCO REGULADOR 76
7. CONCLUSIONES 78
7.1. RESULTADOS OBTENIDOS 78
7.2. OBJETIVOS CUMPLIDOS 79
7.3. LÍNEAS FUTURAS DE TRABAJO 79
BIBLIOGRAFÍA 81

Índice de Figuras

Figura 1: Pokémon Go [4] *4*
Figura 2: Representación del juego Tennis for Two [12] *5*
Figura 3: Super Mario 64 [21] *8*
Figura 4: Dead Cells, Motion Twins, 2017 [41] *10*
Figura 5: Captura del videojuego Crisis 2, Electronic Arts, 2011 [43] *12*
Figura 6: Portada de Metroid Prime, Nintendo, 2003. Uno de los juegos que inventó el género Metroidvania *15*
Figura 7: Modelo del personaje principal (Alubia) *22*
Figura 8: Diagrama de flujo para las entradas del jugador *23*
Figura 9: Diseño original sobre el movimiento de la hormiga *24*
Figura 10: Diagrama de flujo de la toma de decisiones de la hormiga *25*
Figura 11: Diseño original sobre el movimiento de la mosca *26*
Figura 12: Diagrama de flujo sobre la toma de decisiones de la mosca *26*
Figura 13: Diseño original del movimiento de la araña (Desplazamiento horizontal) *27*
Figura 14: Diseño original del movimiento de la araña (Salto Vertical) *27*
Figura 15: Diseño original del movimiento de la araña (Salto Horizontal) *28*
Figura 16: Diagrama de flujo sobre la toma de decisiones de la araña *28*
Figura 17: Diagrama de flujo del funcionamiento del disparo *31*
Figura 18: Boceto original del diseño de 10 salas. *32*
Figura 19: Captura de la interfaz de Unity donde se muestran los componentes PlayerMovementScript y PlayerHealthController. *36*
Figura 20: Diagrama de clases para GameManager *36*
Figura 21: Diagrama de clases para healthController y HazardTriggerHandler *37*
Figura 22: Diagrama de clases para el jugador *38*
Figura 23: Diagrama de clases para HealthHUDHandler *39*
Figura 24: Diagrama de clases para bulletHandler *40*
Figura 25: Diagrama de clases para la hormiga enemiga *41*
Figura 26: Diagrama de clases para la mosca enemiga *42*
Figura 27: Diagrama de clases de la araña enemiga *43*
Figura 28: Captura de los rayos usados por la araña *44*
Figura 29: Sala LRT *45*
Figura 30: Diagrama de clases relacionadas con la generación de escenarios *45*
Figura 31: Sala R, vista isométrica *47*
Figura 32: Diagrama de clases para el proyecto completo *49*
Figura 33: Menu principal de Beans Out *50*

Figura 34: Escena del tarro de alubias justo al comenzar el juego *50*
Figura 35: Escena del tarro de alubias después de descender al piso inferior *51*
Figura 36: Escena del tarro de alubias. A la izquierda se puede ver el modelo del rey alubia y a la derecha la puerta para comenzar una partida *51*
Figura 37: Captura de una ejecución de Beans Out *52*
Figura 39: Captura de una ejecución de Beans Out, la barra de vida no está completa y se observa el disparo *52*
Figura 38: Pantalla de carga *52*
Figura 40: Menú de pausa *53*
Figura 41: Resultados para la categoría de los errores *61*
Figura 42: Resultados obtenidos para el comportamiento del juego después del error *62*
Figura 43: Respuestas obtenidas para la gravedad del error *62*
Figura 44: Resultados obtenidos para la intuición de los controles *64*
Figura 45: Resultados obtenidos para la comodidad de los controles *65*
Figura 46: Resultados obtenidos para la respuesta de los controles *65*
Figura 47: Resultados obtenidos para el comportamiento general de los enemigos *65*
Figura 48: Resultados obtenidos para el comportamiento de las hormigas *66*
Figura 49: Resultados obtenidos para el comportamiento de las moscas *66*
Figura 50: Resultados obtenidos para el comportamiento de las arañas *67*
Figura 51: Resultados obtenidos para la dificultad del juego *67*
Figura 52: Resultados obtenidos con respecto a la movilidad en el escenario *68*
Figura 53: Resultados obtenidos para la diferencia entre partidas *68*
Figura 54: Diagrama Gantt estimado del proyecto *69*
Figura 55: Diagrama Gantt estimado para la fase 1 ..... *69*
Figura 56: Diagrama Gantt estimado para la fase 2 ..... *70*
Figura 57: Diagrama Gantt estimado para la fase 3 ..... *70*
Figura 58: Diagrama Gantt estimado para la fase 4 ..... *71*
Figura 59: Diagrama Gantt estimado para la fase 5 ..... *71*
Figura 60: Diagrama Gantt real del proyecto *71*
Figura 61: Diagrama Gantt real para la fase 1 ..... *72*
Figura 62: Diagrama Gantt real para la fase 2 ..... *72*
Figura 63: Diagrama Gantt real para la fase 3 ..... *73*
Figura 64: Diagrama Gantt real para la fase 4 ..... *73*
Figura 65: Diagrama Gantt real para la fase 5 ..... *73*

Índice de Tablas

Tabla 1: Tabla de ejemplo para los casos de prueba 54
Tabla 2: CP-001 54
Tabla 3: CP-002 55
Tabla 4: CP-003 55
Tabla 5: CP-004 55
Tabla 6: CP-005 55
Tabla 7: CP-006 55
Tabla 8: CP-007 56
Tabla 9: CP-008 56
Tabla 10: CP-009 56
Tabla 11: CP-010 56
Tabla 12: CP-011 56
Tabla 13: CP-012 57
Tabla 14: CP-013 57
Tabla 15: CP-014 57
Tabla 16: CP-015 57
Tabla 17: CP-016 57
Tabla 18: CP-017 58
Tabla 19: CP-018 58
Tabla 20: CP-019 58
Tabla 21: CP-020 58
Tabla 22: CP-021 58
Tabla 23: CP-022 59
Tabla 24: CP-023 59
Tabla 25: CP-024 59
Tabla 26: CP-025 59
Tabla 27: CP-026 59
Tabla 28: CP-027 60
Tabla 29: Recopilación de errores encontrados en las pruebas externas 63
Tabla 30: Costes de personal del proyecto 74
Tabla 31: Costes materiales del proyecto 75
Tabla 32: Costes indirectos y otros costes del proyecto 75
Tabla 33: Coste total del proyecto 76

1. Introducción

La informática y la programación han sido elementos que han ido ganando importancia de forma exponencial en los últimos años. Grandes descubrimientos e inventos se han llevado a cabo gracias a esta tecnología, desde poder comunicarnos en forma de videoconferencia gracias a internet, hasta obtener la primera foto de un agujero negro.

Pero estos conocimientos también han ganado importancia en el sector del entretenimiento y el mejor ejemplo de esto son los videojuegos. Creaciones que permiten al usuario experimentar una infinidad de situaciones tanto plausibles en la vida real como completamente fantásticas u oníricas. En este documento se detallan los pasos realizados para llevar a cabo el desarrollo de un videojuego de plataformas en Unity 3D.

1.1. Motivación

Los videojuegos son unos elementos que cada vez captan más la atención del público general y cada vez es más común jugar a alguno. Con la llegada de los smartphones, muchos son los que tienen o han tenido algún juego instalado para pasar esos minutos sueltos donde no se sabe muy bien qué hacer.

En mi caso, desde joven ya jugaba a varios videojuegos, tanto en ordenador como en consolas. Siempre me ha fascinado la capacidad que tienen para transportarte a otro universo o mundo con sus propias reglas y desarrollar una aventura ahí. Según fui creciendo, me fui interesando por más géneros y fue cuando acabé totalmente enamorado de este mundo. La variedad de estilos, formas, sentimientos… que puede tomar y hacerte sentir un videojuego son infinitas y con el trabajo adecuado se pueden crear obras de una calidad increíble.

Es esta posibilidad de que tu creación pueda impactar en la vida de alguien que probablemente no llegues a conocer nunca, que me atrajo hacia el mundo de crear videojuegos y cuando comencé a programar algunos juegos muy sencillos entendí que era aquello, que quería decir en el futuro, que era mi oficio.

También me contactó un amigo que le gustaría llevar a cabo una idea que tuvo, un videojuego. Este fue el pistoletazo de salida para crear mi primer videojuego completo. Así que uniendo todos estos elementos decidí centrar mi trabajo de fin de grado en este proyecto después de consultar con varios profesores.

1.2. Objetivos

Para este TFG se pretende realizar una versión “demo” del juego final construyendo los elementos más importantes y relevantes que formarán la base de cualquier cambio o mejora que pudiera implantarse en el futuro haciendo uso de la herramienta Unity 3D. Por esto se definen los siguientes objetivos para el proyecto:

- Diseño del videojuego: diseñar el personaje principal, los secundarios, los enemigos, los escenarios y la narrativa que compondrán el juego en su totalidad.
- Implementación del movimiento del personaje principal. Se buscará una sensación de juego fluida e intuitiva para que el usuario se sienta cómodo.
- Creación de escenarios generados de forma procedural.
- Implementación de comportamientos para los enemigos.
- Componer una interfaz de usuario intuitiva que contenga toda la información relevante.
- Realizar una fase de pruebas con un grupo de personas que aporten comentarios y reporten errores mientras juegan al juego.

Mediante estos puntos se pretende obtener una versión con todos aquellos elementos más relevantes del juego y que permita añadir más funcionalidades extras en un futuro. En ningún momento se plantea terminar el juego en su totalidad ya que es una tarea demasiado compleja y extensa como para detallarla y realizarla en este documento, sin embargo, si se detallan muchos elementos del juego final en la sección de diseño para ayudar a entender la visión general del proyecto. Además, dichos puntos servirán para aprender la herramienta Unity 3D casi en su conjunto gracias a la variedad en las tareas.

Para finalizar, también se detallan algunos objetivos que me gustaría llevar a cabo pero que se encuentran más en un segundo plano por ser menos relevantes:

- Implementación de la música y los sonidos del juego.
- Creación de modelos 3D para los personajes del juego.
- Creación de animaciones. Siempre serán más relevantes las animaciones del jugador principal dejando en segundo plano aquellas de los personajes secundarios y/o enemigos.
- Traducción del juego a otros idiomas. El juego será desarrollado en español y el principal idioma al que se traduciría sería el inglés.

En conclusión, se busca generar un primer prototipo de juego que posea los cimientos, sobre los que se soportarían el resto de ampliaciones y mejoras futuras, y realizar un estudio sobre los errores que se encuentren y como aceptaría el público este juego.

1.3. Estructura del documento

Habiendo comentado las motivaciones y objetivos del proyecto procedemos a detallar la estructura que seguirá el presente documento.

Apartado 2: Estado del arte. En esta sección se revisarán los ámbitos que han sido influyentes a la hora de seleccionar este proyecto. Se hará un pequeño resumen de la historia del videojuego para plantear como hemos llegado hasta nuestros días y se definirán conceptos relacionados estrechamente con este trabajo.

Apartado 3: Análisis de la situación. En este apartado se explicarán los procesos que se llevaron a cabo para definir el proyecto y sus características generales. Se analizará el proceso por el cual se crearon los personajes, la trama y el estilo, dando forma al concepto global del juego.

Apartado 4: Diseño. Esta sección detalla el proceso de diseño de los distintos elementos del juego. Aquí se entra en detalle sobre comportamientos, habilidades, movimientos… de los enemigos, personaje, etc.

Apartado 5: Desarrollo de la aplicación. En esta sección se detalla la implementación seguida para cumplir todos los puntos declarados en el apartado de diseño. Se entra en detalle sobre la programación y ensamblaje de la aplicación usando Unity. También se detallan las pruebas seguidas para comprobar el correcto funcionamiento de la aplicación.

Apartado 6: Planificación y presupuesto. En esta sección se detallan aquellos aspectos relevantes a la gestión del proyecto como la planificación de este, los gastos asociados, el impacto económico y social o el marco regulador al que se ajusta este proyecto.

Apartado 7: Conclusiones. Este apartado funciona de cierre para el documento haciendo un resumen de este, recopilando los objetivos que se han cumplido, los resultados que se han obtenido y analizando el posible trabajo en el futuro que se podría realizar.

Apartado 8: Bibliografía. Se detallan los documentos que han sido usados para la realización de este trabajo.

Apartado 9: Anexo. Documentos extras usados en este proyecto.

2. Estado del arte

En la siguiente sección se detalla el estado de la cuestión, haciendo hincapié en todos aquellos ámbitos que influyen o han sido influyentes en la decisión de realizar este trabajo.

2.1. El videojuego como método de entretenimiento

El sector del entretenimiento está en constante cambio y evolución y esta última década se ha entrado en la época donde el videojuego domina. Aparte de la evidente crecida de popularidad de los últimos años con juegos como *Minecraft* [1], *League of Legends* [2], *Fortnite* [3], *Pokémon GO* [4] o *Among Us* [5] más recientemente, los videojuegos ya han superado a otros medios de entretenimientos clásicos como el cine, como muestran datos del 2019 donde el sector cinematográfico generó 42 mil millones de dólares estadounidenses [6] mientras que el sector de los videojuegos se coronó en los 152 mil millones de dólares [7].



Figura 1: Pokémon Go [4]

Otros sectores como la televisión en directo todavía siguen siendo líderes del entretenimiento. Sin embargo, debido a la reciente situación mundial en la que nos encontramos, se han resaltado todavía más estas diferencias entre los diferentes medios de entretenimiento. Destacando como los videojuegos han aumentado de popularidad frente a otros que se han desplomado, como en Estados Unidos donde los beneficios producidos por los videojuegos han sido superiores no solo a los generados por los cines, si no también, a los generados por los deportes tradicionales [8].

Y es interesante como en esta situación de pandemia mundial, un medio de entretenimiento ha sabido no solo mantenerse, sino que además impulsarse. Ya no solo en el formato que se pensó desde un principio, es decir, jugando al juego tú mismo, sino de otras formas que no destacaban tanto anteriormente, esto es, viendo a otras personas jugar. Éxitos

rotundos y masivos como los de la plataforma de *streaming* Twitch [9] [10] muestran que este medio no tiene solo una forma de consumirse y que tiene todavía muchos caminos que explorar y descubrir.

2.2. La historia del videojuego

La historia del videojuego, aunque reciente, es extensa y con varios puntos de inflexión donde se redefinieron ideas o conceptos claves. Por esto, la historia de este sector se suele dividir en décadas ya que, con las llegadas de mejoras en las tecnologías del momento era con lo que los desarrolladores conseguían avances importantes.

2.2.1. Década de 1950

En esta época se contaba con unas tecnologías novedosas pero que no habían sido pensadas para más allá de su uso militar, debido a la reciente guerra que hubo, la Segunda Guerra Mundial. Uno de los mayores avances que se lograron fue en la computación, pero centrándose más en el campo de criptografía para conseguir cifrar y descifrar datos de forma más segura [11]. Es por esto que, años después de terminar la guerra, estos avances fueron usados en otros ámbitos. Por ejemplo, debido al aumento de casas estadounidenses que poseían una televisión en esta década, ya se planteó la idea de crear un sistema para poder interactuar con la pantalla siendo una representación muy primigenia de lo que hoy en día conocemos como videojuego. Finalmente, estas ideas no se acabaron llevando a cabo y se quedaron en conceptos. Más adelante, este plan se llevaría a la práctica aplicando pequeños cambios. *Tennis for Two* [12], considerado uno de los primeros videojuegos de la historia fue desarrollado en 1958 por William Higinbotham. En este sencillo juego, dos jugadores podían interactuar con un punto en la pantalla para simular un “partido de tenis” en dos dimensiones.

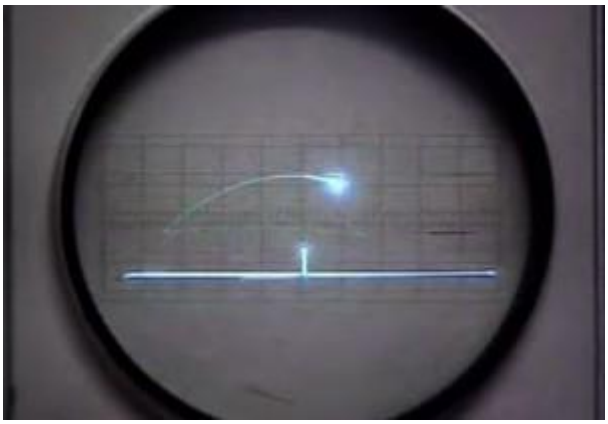


Figura 2: Representación del juego Tennis for Two [12]

2.2.2. Década de 1960

Durante estos 10 años, los videojuegos vieron un parón en sus avances. Pequeños proyectos como, *Spacewar!* desarrollado por un grupo de estudiantes del Instituto de tecnología de Massachusetts [13] fueron creados, pero la mayor parte de los avances en computación se centraron en intentar crear computadoras capaces de jugar al ajedrez cada vez mejor.

2.2.3. Década de 1970

Los videojuegos entran en una época dorada donde se hicieron grandes avances que marcaron el futuro del medio. En 1971 un joven Bill Pits, fascinado con *Spacewar!* creado pocos años antes consiguió, junto a un amigo, crear *Galaxy Game* [13], lo que pasaría a conocerse como el primer videojuego de máquina recreativa de la historia. Otra máquina recreativa que surgió por los mismos años fue *Computer Space*, creada por Nolan Bushnell [13] que, al igual que Bill quedó fascinado por *Spacewar!*. Ambos productos obtuvieron poco éxito en el mercado, pero fue el creador de esta segunda, quien después de problemas con la empresa con la que publicó *Computer Space*, acabó creando Atari [14], la cual se convertiría en una de las mayores influencias para el mundo de los videojuegos.

Nolan Bushnell, atraído por el sector que estaba descubriendo comenzó a aprender e interesarse más y más por este. Asistió a una conferencia de la primera videoconsola de la historia, *Magnavox Odyssey* [14] [15], donde pudo probar esta y, en especial, uno de los juegos que incluía, el *ping-pong*. Este videojuego simple fue la inspiración para el primer lanzamiento de Atari. *Pong*, un juego parecido al ya mencionado *Tennis for Two* y publicado en 1972, fue un éxito rotundo de ventas convirtiéndose en lo que muchos conocen como el inicio de la industria de los videojuegos [14]. Atari obtuvo un estatus muy elevado y grandes beneficios, pero estos buenos resultados también atrajeron a muchas compañías y en poco menos de un año múltiples empresas ya competían en este sector, contrastando con la situación anterior, donde Atari era la única en el mercado. En pocos años Atari publicó varios juegos que por sí solos inventaron algunos de los géneros de videojuegos que conocemos actualmente. Atari también se introdujo en el mercado de las videoconsolas en el año 1975 con su *Telegames Pong*, y años más tarde con *Atari 2600* [14] la consola más famosa de la compañía que además hizo famoso el uso de cartuchos [16] para que el usuario pudiera cambiar de juego.

En esta década, el mundo vería también la llegada de los microprocesadores lo que ayudó mucho al sector a crear mejores dispositivos, más pequeños y menos costosos. Junto a este avance aparecieron los ordenadores personales (PC) [17] con empresas como Apple revolucionando el mercado haciendo posible que la mayor parte de la población tuviera un

ordenador en casa. Esto lo aprovecharon los desarrolladores para crear otro género distinto, las aventuras gráficas [14], que podían ejecutarse en estos PC.

A finales de la década de los 70, el mercado japonés marcó un antes y después en la historia de los videojuegos cuando se publicó *Space Invaders* [13] [18] en 1978. Este juego fue un fenómeno a nivel mundial, tanto que se expandió por todo el globo y se editó en diferentes formatos. Recordado comúnmente como "los marcianitos", fue el juego que inició lo que se conoce como la edad de oro de los videojuegos [14].

2.2.4. Década de 1980

Entre los años 1978 y 1983 los videojuegos vieron un aumento de su popularidad y dejaron de ser un elemento de nicho. Las máquinas recreativas se habían expandido por todo tipo de locales, desde supermercados hasta los bares, estas aportaban beneficios y eran un punto de encuentro de la gente. Además, junto con la aparición de algunos avances en la tecnología como el uso de color en las pantallas, se comenzaron a crear videojuegos mucho más llamativos y con mejor apartado gráfico. En esta época cualquier compañía de entretenimiento del mercado había creado su departamento de videojuegos y ya habían comenzado a sacar títulos al mercado. Un ejemplo de esto sería la gran compañía Walt Disney que sacó juegos como Sorcerer's Apprentice [19] en colaboración con Atari.

Al igual que muchas de estas compañías, Activision apareció en el mercado de los videojuegos, pero de una forma revolucionaria. Al contrario que otras grandes empresas como Atari, Activision no había desarrollado una consola ni tenía pensado hacerlo, su método de trabajo es el que actualmente se conoce como third-party [14] [20]. Esto quiere decir que Activision podía publicar sus juegos en cualquier plataforma que le dejase y no tenía que estar sujeto a las grandes empresas. Este modelo de negocio es el que la empresa continuó llevando incluso hasta el día de hoy.

En el otro lado del globo, Japón seguía consolidándose como uno de los mayores mercados del mundo en cuanto a videojuegos se trataba. Nintendo, la mayor empresa en ese momento, obtuvo gran repercusión en el mundo entero con sus consolas y videojuegos. Una de sus mayores aportaciones fue la creación de licencias a terceras empresas para desarrollar juegos para su consola [14]. Algo parecido a lo que Activision realizaba con Atari. De esta forma, Nintendo ganaba gran cantidad de títulos para su catálogo, pero tenía el control de aquello que se publicaba bajo su marca.

2.2.5. Década de 1990

La década de los años 90 se vería caracterizada por un ligero estancamiento de los juegos y géneros debido al gran trayecto que había recorrido el sector en tan poco tiempo. Aun así, esta etapa vio algunos avances muy relevantes. Los avances en la capacidad de cómputo de los procesadores permitieron mejoras gráficas en los videojuegos como la implementación de gráficos 3D. Juegos de esta época como *Super Mario 64* [21] que hicieron uso de esta tecnología son recordados actualmente por el público como obras maestras o exponentes de cómo hacer un videojuego [14].



Figura 3: Super Mario 64 [21]

En esta época se vivió lo que se conoce como una guerra de consolas, algo que ya se había estado viendo, pero en menor medida ya que siempre había una empresa que dominaba por encima del resto. Pero en esta década, Sony lanzaría su PlayStation [22] marcando el inicio de una de las consolas más relevantes a día de hoy. A su vez, Sega tendría bastante éxito también con su Mega Drive [23] y Nintendo mantendría su estándar con la Nintendo 64 [24].

2.2.6. Década de 2000

A partir de los años 2000 la industria del videojuego continuó creciendo de forma exponencial llegando a convertirse en una de las industrias más relevantes del momento [14]. Por esta época surgieron algunas de las consolas más vendidas de la historia [25] como la PlayStation 2 y PlayStation 3 de Sony, la Wii [26] de Nintendo o la Xbox 360 de Microsoft. Y eran justo estas tres empresas las que pasaron a dominar el panorama dejando en una posición más secundaria a otras como Atari o Sega [14].

En estos años, los avances en este entorno se estabilizaron ya que las empresas no necesitaban innovar en cada uno de sus títulos para captar la atención del público, por lo que en esta época comenzó a aparecer la tendencia de crear sagas de videojuegos en vez de diseñar un juego nuevo desde cero. Esto suponía para la empresa menos trabajo ya que podía reutilizar algunos elementos de sus proyectos anteriores. Debido a esto, a partir de los años 2000, este método se fue haciendo más y más popular entre las empresas importantes del sector.

2.2.7. Década de 2010 y actualidad

La década se los 2010, si bien ha terminado muy recientemente y todavía es pronto para poder hablar del impacto que ha tenido sobre la industria, sí que ha traído algunas nuevas prácticas interesantes. Los juegos independientes han ganado popularidad, algunos juegos se han convertido en fenómenos de masas a nivel mundial, similar a lo que pasó con algunas series o películas como, por ejemplo, Star Wars [27]. Pero quizás, el cambio más revolucionario ha sido el mercado móvil de videojuegos. Juegos sencillos y que permitían al jugador pasar un rato jugando en el móvil como Fruit Ninja [28] o Candy Crush [29] han conseguido millones de descargas transformando a sus creadores en casi millonarios. Y es que el mercado de móvil es un campo que todavía está en constante cambio y está viviendo algunas revoluciones propias.

2.3. Juegos independientes

Después de conocer la historia por la que ha pasado el videojuego, podemos observar que muchos de los avances que se han llevado a cabo han sido gracias a proyectos de gran magnitud de grandes equipos de desarrolladores, pero últimamente ha estado creciendo una nueva tendencia, los juegos independientes. Los *indies* como se les conoce comúnmente son juegos creados por equipos de personas muy pequeños o incluso por una sola persona.

Que cada vez sea mayor la cantidad de estos juegos se debe a las facilidades que han aparecido con el tiempo, que permiten que una sola persona se encargue de llevar a cabo todo el proyecto. Además, se ha producido una eliminación de muchas barreras de entrada ya que, ahora mismo existen multitud de motores de videojuegos y programas enfocados en dicha tarea gratuitos y cada vez es más fácil encontrar información sobre cómo usarlos.

En este campo existen varios ejemplos de juegos que han llegado a puntos de excelencia muy altos e incluso algunos que han podido competir con otros títulos de grandes compañías. Un ejemplo muy claro de esto es Minecraft. Actualmente Minecraft [1] no se considera juego independiente ya que Mojang, la empresa que lo creó, fue comprada por Microsoft en 2014 [30]. Sin embargo, la primera versión de este juego fue desarrollada de forma independiente por un grupo pequeño y ha llegado a convertirse en el juego más vendido de la historia. Otros

ejemplos son; Stardew Valley [31], Celeste [32] o Gris [33], un juego de un estudio independiente español que fue nominado a varios premios por su excelente apartado artístico [34].

2.4. Plataformas y géneros

El género de las plataformas es uno de los más antiguos y usado debido a su simpleza, pero a la vez, gran capacidad para crear algo novedoso. Este género engloba cualquier juego que base su jugabilidad en el movimiento entre plataformas, normalmente haciendo uso del salto. Esto se ha llevado a cabo de infinitas formas, como en Mario Bros [35], donde los niveles eran lineales y tenían un objetivo simple, llegar al final del nivel y continuar avanzando nivel a nivel hasta derrotar al jefe final y completar la campaña, u otros títulos más recientes como Hollow Knight [36] donde las plataformas generan un mundo totalmente abierto y libre que el jugador puede visitar a su antojo teniendo que explorar para encontrar aquello que busca.

El otro género en el que se basa este proyecto en el conocido como Rogue-Like. Cuyo nombre significa literalmente, parecido a Rogue. Rogue [37], fue uno de los primeros juegos en usar mazmorras aleatorias para que cada partida fuese distinta a la anterior. Este género se ha expandido mucho y puede tomar muchas formas. Una de las más importantes actualmente es The binding os Isaac [38], un juego de exploración de mazmorras donde cada partida debes llegar hasta el jefe final pasando por niveles generados proceduralmente obteniendo objetos aleatorios que dan habilidades a nuestro personaje. También se puede destacar Spelunky [39] o Hades [40]. Dos juegos que también comparten la mecánica básica de pasar por zonas que nunca son iguales a la vez anterior pero cada uno con una premisa distinta. En el primero, un explorador busca minerales por cuevas llenas de trampas y peligros, mientras que el segundo, cuenta la historia de cómo Zagreo, hijo de Hades, escapó del infierno para conocer a su madre Perséfone. Como se puede ver, este género también deja mucha libertad creativa.



Figura 4: Dead Cells, Motion Twins, 2017 [41]

Es por esta libertad que ofrecen ambos géneros que estos serán los principales del juego. En concreto, la mayor inspiración de este proyecto fue el juego Dead Cells [41] ya que es un Rogue-Like de plataformas. De este juego obtuve multitud de ideas e inspiración para llevar a cabo este proyecto.

2.5. Herramientas para la creación de videojuegos

En esta sección se detallan las principales herramientas para la creación un videojuego. Para esta tarea en concreto nos centraremos en dos, el motor de videojuegos y la herramienta de modelado 3D.

2.5.1. Motores de videojuegos

El motor de videojuegos es una herramienta general que se encarga de ensamblar todo el proyecto además de hacer más simple el trabajo de crear elementos dentro del juego gracias a varias herramientas que incluye. Cada motor funciona y posee unas características propias, pero hay muchos elementos que la gran mayoría comparten.

- **Programa principal**. El motor es el encargado de montar todo el juego como un programa final ejecutable. Para esto da la posibilidad al usuario de crear scripts con distintos lenguajes de programación que después él compilará para crear el programa final. Muchos de estos programas también añaden un sistema de cifrado de la información cuando se hace el montaje de la aplicación para evitar que entes externos puedan acceder al código fuente del juego.

- **Renderización**. Este programa se encarga también de la renderización por pantalla de los elementos 3D del juego. Esto se realiza por distintas técnicas (rasterización, trazado de rayos…) según las que permita el motor y las que el usuario elija para su proyecto. Muchos motores de videojuegos también incluyen un modelo de juego en 2D que no necesita de renderización de volúmenes, pero sí de imágenes o *sprites*.

- **Control del audio**. El audio y los sonidos del juego son un elemento de vital importancia para conseguir una experiencia inmersiva para el jugador por lo que estos programas contienen un controlador de audio. Este se encarga de reproducir los sonidos del juego, muchas veces haciendo uso del posicionamiento espacial del jugador para recrear efectos complejos como los ecos o el Efecto Doppler.

- **Motor físico**. Por último, estos programas simplifican la creación de un videojuego aportando un sistema de físicas realistas. En este sistema ya vienen implementadas funciones como la atracción gravitatoria, las colisiones entre objetos, rozamientos con superficies, etc. Normalmente, el usuario tiene acceso a los parámetros generales de estos sistemas para

modificar ciertos comportamientos. Por ejemplo, si quisiéramos hacer un juego que se desarrollase en la Luna, podríamos establecer la gravedad a 1,62 m/s^2 en vez de los 9,8 de la Tierra y los personajes se comportarían como si pesasen menos. También se permite desactivar estas características para algunos elementos como un objeto que quieres que se mueva de una manera específica y no quieres que le afecte la gravedad o un objeto con el que no puedas colisionar y solo atravesarlo.

Los motores de videojuegos han avanzado mucho en el tiempo y han surgido muchas versiones y opciones distintas. A continuación, se mencionan las más relevantes destacando sus puntos fuertes y sus desventajas.

2.5.1.1. CryEngine 3

CryEngine es un motor de videojuegos desarrollado por Crytek [42] que posee compatibilidad con Windows, PlayStation y Xbox. Este motor permite crear escenarios con iluminación muy realista de forma simple, como ya ha demostrado en productos como Crisis 2 [43], realizado en este motor. Este programa está pensado principalmente para crear *shooters* en primera persona. Este programa no es gratuito y es necesario pagar una suscripción mensual para poder usarlo.



Figura 5: Captura del videojuego Crisis 2, Electronic Arts, 2011 [43]

2.5.1.2. Source 2

Source 2 es el motor gráfico propio de la compañía Valve y actualmente es de uso gratuito. Simplemente necesitas unirte al grupo de desarrolladores de Valve y enviar una solicitud. Al igual que el anterior, está pensado para crear principalmente *shooters* en primera persona.

2.5.1.3. Unreal Engine 4

Unreal Engine es uno de los motores de videojuegos que más importancia tiene a día de hoy. Esto se debe a que es gratuito y posee una potencia para crear muy grande, tanto que muchos juegos de grandes empresas son creados con este motor. Su principal desventaja es que, hasta hace poco, este motor no era gratuito y era prácticamente exclusivo para grandes empresas por su precio. Esto hace que no posea demasiada documentación, lo que dificulta la tarea de programar en él.

2.5.1.4. Unity 5

Unity 3D es probablemente el motor de videojuegos más usado actualmente gracias a la gran variedad de posibilidades que ofrece y la gran documentación que posee. Además de gratuito, Unity es el motor de desarrollo de videojuegos preferido por las academias para comenzar a programar videojuegos debido a su facilidad de uso. La principal desventaja de Unity es que al ser un programa tan general que sirve para hacer juegos tanto en 2D como en 3D, tanto en consolas como en PC, no destaca especialmente en nada ni tiene una potencia muy alta. Sin embargo, por la gran comunidad que hay alrededor de esta herramienta que está dispuesta a resolver dudas en los foros, es una de las mejores opciones a elegir.

2.5.2. Herramientas de modelado 3D

Una herramienta de modelado 3D es un programa informático enfocado en la creación, modelado, iluminación y renderizado de gráficos tridimensionales. Algunos programas añaden otras características como animación o pintura digital. En el mercado actual existen muchas posibilidades para realizar esta tarea, pero Blender es quizás la más popular y escogida a nivel global.

Blender es un programa dedicado la creación de modelos tridimensionales. Posee multitud de características distintas para facilitar todo el proceso y prácticamente cualquier modificación que quieras llevar a cabo se puede realizar en este programa, por lo que no es necesario usar herramientas externas. A pesar de tener esta gran variedad de posibilidades, desde renderización, texturizado, iluminación, escultura, dibujo… Blender es gratuito y además de código abierto, lo que quiere decir que la comunidad puede acceder a ese código y crear extras para la aplicación. Todo esto combinado ha convertido a esta aplicación en una de las más usadas tanto por principiantes que comienzan como por profesionales que confían en este software, llegando a tener 90 millones de personas [44] que han visitado su página web.

2.6. Documentos auxiliares

Para la realización de este trabajo se han usado otros tres proyectos de fin de grado como ayuda. El primero de ellos, Diseño y desarrollo de una aplicación en Unity3D para la enseñanza de programación [45] por Sergio de Blas de Miguel y el segundo, Desarrollo de un juego de rol estratégico con LibGDX [46] por Ernesto Gabriel González Chacón. Ambos trabajos realizados en la Universidad Carlos III, están relacionados con el sector de los videojuegos, pero no específicamente con el proyecto que se trata en este documento. El tercero, en cambio, realizado por Daniel Rivero Martinez, para la Universidad de Barcelona, denominado Desarrollo de un videojuego 2.5D con Unity [47] se asemeja bastante en los conceptos generales del juego y me ha servido como guía en determinados puntos.

3. Análisis de la situación

Para este proyecto se pretende realizar un videojuego *Rogue-Like* de plataformas en falso 3D. A continuación, vamos a repasar y analizar el proyecto con respecto a las decisiones iniciales.

3.1. Decisiones iniciales

Antes de poder comenzar a trabajar en el proyecto había que definir ciertos aspectos del juego, como la temática, historia, género, etc. El proyecto pasó por varias fases mientras se exploraban las distintas posibilidades.

Con respecto al género siempre se tuvo claro desde el principio que sería un juego de plataformas, ya que es un género sencillo pero que deja mucha libertad creativa. Aun así, la primera idea fue crear un *metroidvania*, un género en el que el jugador explora un mapa abierto descubriendo o consiguiendo objetos o habilidades que le permiten pasar por zonas que antes parecían inaccesibles. Después de reflexionar sobre la temática del juego, se encontró que este género no cuadraba con la historia que se quería transmitir por lo que se cambió a un *Rogue-Like*. Al final, después de refinar distintos puntos, el juego acabó asentándose en un *Rogue-Like* de plataformas, donde el escenario sería generado aleatoriamente y si el jugador muere debería comenzar la partida de nuevo.

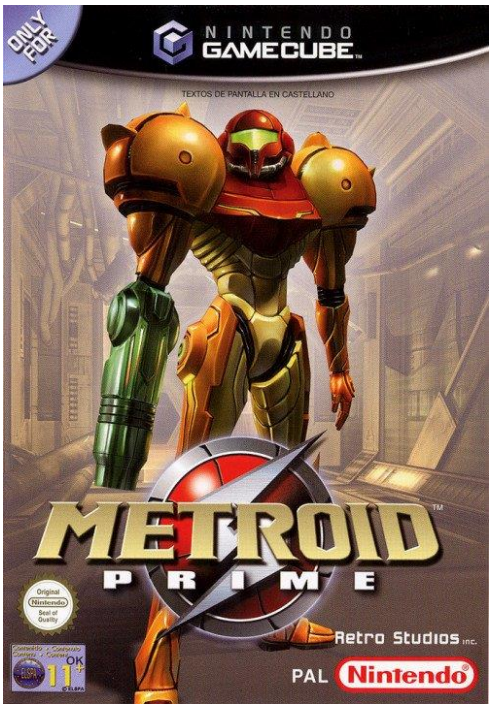


Figura 6: Portada de Metroid Prime, Nintendo, 2003. Uno de los juegos que inventó el género Metroidvania

La historia del juego surgió hace mucho tiempo ya que siempre ha sido un proyecto personal que se ha querido llevar a cabo. Esta narra la vida de una judía, que vive en un tarro de judías hasta que un día es elegida para prosperar su especie y plantarse en el jardín de la casa en la que viven. Ahí comenzaría la aventura que supondría atravesar la casa hasta el jardín. Esta trama unida al género *Rogue-Like* fue lo que confirmó el último punto que le faltaba a la trama para estar totalmente cohesionada con el *gameplay*. Como se había mencionado, el juego anterior pretendía ser un *metroidvania*, es decir el jugador tendría toda la casa para explorar hasta llegar al jardín. Pero surgían problemas como, por ejemplo, ¿qué pasa si muere el

personaje? No se conseguían responder consistentemente estas dudas sin que aparecieran problemas en la trama.

Se había llegado a un punto muerto, así que se tomó la decisión de volver sobre mis pasos y cambiar algo anterior del juego. Ahí fue cuando se pensó en el género del juego. La categoría de las plataformas no daba ningún problema ya que solo definía cómo iba a ser el mapa. Pero la naturaleza *metroidvania* era lo que producía muchos problemas. Es por eso que se pensó en cambiarlo y apareció el género *Rogue-Like* gracias a que en esa época me encontraba probando *Dead Cells*, de esta misma familia de juegos. Con este cambio, la historia final y definitiva quedó de la siguiente manera.

En el tarro de alubias viven miles de ellas, pero necesitan plantarse para dejar descendencia. Es por eso que dentro del tarro cientos de ellas se entrenan para salir a explorar e intentar plantarse en el jardín de la casa. Cuando una judía muere en el camino hasta el jardín, la siguiente le sustituye en su misión.

De esta forma, se une la mecánica básica del género, por la que cada vez que el jugador muere se vuelve a empezar una nueva partida, con la trama del juego sin dejar ningún fallo.

Con respecto a la temática del juego, se tenía claro desde el principio que iba a ser una fantástica, pero ambientada en nuestro mundo actual. Esto era debido al toque cómico que poseía la trama, donde, obviamente las judías no son capaces de pensar, ni moverse, pero aquí se las representa como una sociedad con un fin y unos objetivos claros.

Por último, se necesitaba un nombre con el que referirse al proyecto. Aunque no tenía por qué ser definitivo, se realizaron varias sesiones de *brain-storming* con varios conocidos que ayudaron a escoger el mejor de ellos. Entre los primeros nombres se encontraban; Out of Jar, Ben the Bean o Beans, pero al final el nombre escogido fue *Beans Out*. Este nombre atraía por su simpleza y era sencillo de recordar, lo que lo hacía un candidato perfecto.

3.2. Jugabilidad

Como hemos mencionado, el usuario controlaría una judía, debido a esta idea, elegir las posibilidades de movimiento y acción del usuario fue una tarea complicada ya que se buscaba que, a pesar de lo fantástico e irreal de la situación, estas habilidades tuvieran algo de sentido.

Lo primero que se tuvo claro es que el personaje se iba a poder desplazar de izquierda a derecha y saltar. Esto no podía faltar, es el movimiento esencial para un juego de plataformas. El problema apareció cuando hubo que pensar en las habilidades de combate ya que el usuario encontraría peligros en su viaje por la casa hasta el jardín y tendría que enfrentarlos. Para esto

se pensó en la fotosíntesis y la energía del sol debido a que, al fin y al cabo, una alubia es una planta. Esta idea sufrió algunas pequeñas modificaciones, pero fue la que acabó funcionando mejor. De esta forma, las habilidades de combate del usuario se basarían en el "poder de la fotosíntesis" tratando esta energía como si fuera la vitalidad de las judías.

Para combatir, se pensaron 3 habilidades principales que fueron modificándose con el tiempo. La primera era el disparo. Esta no ha sufrido casi ningún cambio desde el principio del proyecto. Una bola de energía que se desplaza en la dirección que es lanzada y hace daño al impactar con un enemigo. La segunda es la carrera o como se conoce en inglés, *dash*. Esta habilidad permite al jugador desplazarse en una dirección muy rápido para poder escapar de algún enemigo o ataque. La tercera es un escudo, que el jugador podría activar y que resistiría el siguiente golpe que este recibiera. Estas dos últimas habilidades acabaron combinándose en un *dash* que te hiciera invulnerable durante un corto periodo de tiempo.

Con respecto a la interacción del jugador con los menús del juego se usaría el ratón. Para hablar con algunos personajes del juego el jugador debería estar cerca de ellos y pulsar alguna tecla específica. Estas son interacciones menos relevantes para el juego, ya sea porque no están íntimamente relacionadas con él o porque no es un objetivo principal de este proyecto, como es el caso de hablar con otros personajes.

Otro elemento importante son los enemigos del juego. Debido al tamaño del jugador no podían ser caballeros ni dragones como podría imaginarse en otros juegos de fantasía. Una primera opción fue añadir otras legumbres como enemigos, pero no funcionaba muy bien con el resto del juego, así que después de reflexionar bastante se acabó optando por insectos de la casa que traten de comerse a la judía. Los primeros enemigos que se pensaron fueron la hormiga, la mosca y la araña. Este tipo de enemigos hacia muy fácil ampliar el repertorio, pudiendo añadir, mosquitos, cucarachas, etc. Es por esto que fueron la decisión final sobre los enemigos del juego.

3.3. Elementos importantes

Detalladas las ideas sobre las que se sustenta el juego, se procede a detallar aquellos elementos que serán importantes para poder crear este proyecto. Estos han sido divididos en cuatro apartados para ordenar mejor la información. Se comenzará con el jugador, pasando por los enemigos, el escenario y al final la interfaz.

3.3.1. Jugador

Con respecto al jugador se buscará diseñar e implementar todas sus funciones, permitiendo al usuario controlarlo de forma sencilla y cómoda:

- Movimiento en horizontal (izquierda y derecha)
- Movimiento vertical (salto)
- Disparo (habilidad de combate)
- Carrera (habilidad de huida o reposicionamiento)
- Un sistema para representar y mantener actualizada la salud restante del jugador

3.3.2. Enemigos

Será necesario definir los atributos y características de los enemigos. Se buscará que cada enemigo se sienta único por lo que será muy importante que se muevan y se comporten de formas distintas. Los enemigos definidos para este proyecto son la hormiga, la mosca y la araña.

3.3.3. Escenario

Como ya hemos dicho anteriormente, el escenario será generado proceduralmente por lo que no se podrá diseñar nunca "un mapa". Sin embargo, para poder generar un escenario aletorio son necesarias "salas plantilla" que son las que se encadenarán para formar el mapa. Es por eso que, para el escenario, los elementos más importantes son dichas plantillas. Cuantas más plantillas se creen, más variedad se obtendrá más adelante en el escenario.

3.3.4. Interfaz

Para finalizar, la interfaz de la aplicación debe mostrar la información más relevante sin abrumar al jugador. Para este apartado se han considerado relevantes dos datos; la vida del jugador y los enemigos restantes. Otros valores no se han considerado suficientemente importantes.

La vida del jugador es esencial para que el usuario sepa su estado y pueda jugar sabiendo su situación actual, mientras que el número de enemigos es una ayuda a la hora completar un nivel, debido a que debes derrotar a todos los enemigos para poder finalizar, este contador ayuda a que dicha tarea sea más fácil además de dar retroalimentación al usuario para que sepa que lo que está haciendo está teniendo un impacto en el juego.

3.4. Herramientas usadas

3.4.1. Udemy

La plataforma de cursos online Udemy ha sido usada previamente a la realización del proyecto para aprender, mediante dos cursos, las bases de Unity y Blender. El primer curso es *"+ 100 horas - Curso Completo Videojuegos con Unity 2018 y C#"* [48] por Juan Gabriel Gomila Salas. En este curso se explica el funcionamiento y uso de Unity desde una base muy sencilla

hasta elementos muy complejos incluyendo ejercicios donde se crean videojuegos completos junto con el tutor. El segundo, *"Blender 2.9X: Modelado y texturizado enfocado a videojuegos"* [49] por Odín Fernández y su equipo, funciona de igual manera. Empezando por los conceptos más simples hasta aquellos más difíciles sobre la herramienta Blender

3.4.2.Unity 3D

El motor de creación de videojuegos Unity fue el elegido debido a su gran comunidad y documentación que hacía muy sencillo encontrar soluciones a los problemas que se encontraban. Además de contar con el curso anteriormente mencionado que ayudó mucho a la hora de introducirme en este software.

Unity es compatible tanto con Windows, Linux y MacOS y proporciona las herramientas para poder crear un juego en muchas plataformas distintas como Windows, Linux, MacOS o incluso consolas como Xbox, PlayStation, etc. La versión usada durante el proyecto es *Unity 2019.3.10f1*.

3.4.3.Blender

Por razones similares a las anteriores, Blender fue escogido gracias a la gran documentación y comunidad que existe alrededor de este programa además de las grandes prestaciones que proporciona.

Blender está disponible tanto para Windows, Linux y MacOS y gracias a la gran cantidad de formatos en los que puedes exportar tus modelos, la incorporación de estos en el proyecto de Unity es muy sencilla. La versión usada para el proyecto es *v2.81.7*.

3.4.4.Atom

Atom es un software de edición de código fuente. Esta desarrollado según un modelo de código abierto por lo que la comunidad puede crear sus plugin para mejorar o añadir funcionalidades lo que le da mucha potencia a este programa. Se ha elegido este programa por encima de muchos otros debido a que Atom es el editor de código que se lleva usando personalmente los últimos 3 años lo que aporta mucha confianza y conocimiento del mismo.

Atom está disponible tanto para Windows, MacOS y Linux y tiene soporte para muchos lenguajes de programación entre ellos, C#, que es el usado para programar los scripts de Unity.

3.4.5.Adobe Photoshop

Photoshop es un programa de edición de imágenes. Este se ha usado para algunas tareas puntuales que requiriesen retoques o la creación de alguna imagen. Además de ser el programa donde se han llevado a cabo la mayoría de croquis y primeros bocetos del juego ya que se puede unir a una tableta grafica para pintar en ella.

Photoshop está disponible en Windows y MacOS. Este producto y los siguientes pertenecientes a Adobe, no son gratuitos y se adquirieron en un paquete todos juntos.

3.4.6.Adobe Premier y After Effects

Ambos programas están enfocados en la edición de video, cada uno en una parte más concreta, pero para este proyecto no he entrado en mucho detalle sobre estos. Han sido usados principalmente para crear o editar cualquier video que se haya usado durante este trabajo.

Adobe Premier y Adobe After Effects están disponible en Windows y MacOS. Estos productos y el anterior, pertenecientes a Adobe, no son gratuitos y se adquirieron en un paquete todos juntos.

3.4.7.Unity User Manual

El manual de usuario de Unity es un documento web que la desarrolladora deja a disposición de los usuarios para que consulten cualquier elemento de este programa. Se ha hecho uso de esta página para buscar el funcionamiento de algunas variables, parámetros o funciones propias de Unity.

3.4.8.Unity Forum

Unity Forum es un foro web dedicado a la aplicación homónima. Este se ha usado para consultar dudas del programa.

3.4.9.Stack Overflow

Stack Overflow es un foro online dedicado a la programación. Este foro es mucho más general que el anterior por lo que se ha usado principalmente para solucionar problemas menos específicos sobre el proyecto.

3.4.10. Google Forms

Este software online de Google ha sido usado para crear y compartir los formularios usados en este proyecto, así como para almacenar las respuestas a estos.

3.4.11. Google Books y Scholar

Las aplicaciones online de Google, Google Books y Google Scholar, se han usado para buscar documentación, tanto en formato de libros como de artículos científicos.

3.4.12. Microsoft Word y Excel

Las aplicaciones de ofimática de Microsoft se han usado para crear, mantener y actualizar los documentos de texto y hojas de cálculo relacionadas con este proyecto. Estas aplicaciones no son gratuitas y han sido adquiridas.

3.4.13. Lucidchart

Para el diseño de diagramas se ha hecho uso de la herramienta Lucidchart disponible de forma online y con una versión gratuita que incluye gran cantidad de funciones.

3.4.14. Open Broadcaster Studio (OBS)

Este programa enfocado en grabar la pantalla de un terminal ha sido usado para crear la demostración de ejecución del programa. Este programa también se usa para retransmitir en directo la pantalla de un terminal, pero esta funcionalidad no ha sido usada para este proyecto.

4. Diseño

En el siguiente apartado se detallan los procesos de diseño y los pasos seguidos durante el desarrollo de *Beans Out*. Para la redacción de este apartado se ha escogido seguir un formato de documento Game Desing Document o por sus siglas GDD. Este formato de documento está especialmente diseñado para especificar todos los apartados relevantes sobre el videojuego ya que el método de requisitos no es el más eficiente en esta clase de proyectos. Por esto, no se realizará un conjunto de requisitos si no que lo aquí descrito funcionará como estos mismos. Se ha seguido como plantilla, la publicada por Leandro Gonzalez en Gamasutra [50], un blog especializado en videojuegos, adaptándola a este proyecto.

En este apartado se detallan elementos del juego, por ejemplo, relacionados con la trama, que no están incluidos en los objetivos del proyecto más allá de diseñarlos por lo que no serán desarrollados más adelante. Esta sección se centra en dar una visión general a todo lo que engloba el juego al completo.

4.1. Personajes

Los personajes de la trama se podrían separar en 2 grandes grupos. Los “buenos” y los “malos”. Con esta categorización podemos dividir a los aliados del jugador de los enemigos. Para definir a los personajes se hará uso de tres parámetros principales; la vida que representa los puntos de daño que puede aguantar antes de morir, la velocidad y el daño que realiza al impactar con un enemigo o el jugador. Nótese que estos valores pueden no ser exactamente los que al final tengan los personajes implementados, estos valores sirven de guía sobre las proporciones, es decir si un personaje tiene velocidad 1 y otro 10, en el juego puede que acabe siendo 10 y 100 pero la proporción se mantiene.

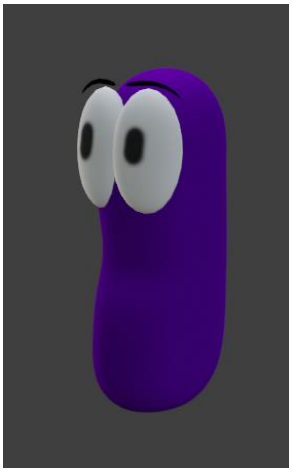


Figura 7: Modelo del personaje principal (Alubia)

4.1.1. Aliados

El primer personaje y el más importante es la alubia que controla el jugador. Esta es de color morado y tiene ojos para personificar al objeto inanimado y darle una apariencia más amigable. La judía principal, dentro de su sociedad ficticia, es un guerrero entrenado para conseguir llegar al jardín y plantarse. Los atributos para este personaje son los siguientes:

- Vida: 100 puntos
- Velocidad: 10 cm/s

- Daño: 10 puntos

El personaje principal tiene 4 habilidades principales; el movimiento horizontal, el movimiento vertical, el disparo y la carrera. El movimiento horizontal es la capacidad que tiene el personaje de moverse a la izquierda o derecha. El movimiento vertical es la capacidad de saltar. La carrera es una habilidad que permite al jugador desplazarse en una dirección, izquierda o derecha, muy rápido durante un periodo de tiempo muy corto. Este periodo de tiempo tendrá que ser valorado mediante pruebas para ver cuál es el valor óptimo y la velocidad de desplazamiento vendría definido en función de la velocidad del jugador, por ejemplo, durante medio segundo se desplazaría a una velocidad igual a diez veces su velocidad de movimiento. Por último, el disparo es una habilidad por la que el usuario puede lanzar un proyectil que se desplazará a una velocidad fija en una dirección hasta impactar con un objeto. Cuando impacta existen dos posibilidades, que sea un elemento del escenario, por lo que la bala desaparecería directamente, o contra un enemigo, caso en el que el enemigo recibiría un daño igual al del jugador. Todas estas habilidades se detallan más profundamente en el apartado 4.4 de Gameplay de esta misma sección.

En la Figura 8: Diagrama de flujo para las entradas del jugador se muestra una selección de teclas que podrían ser los controles del juego. Este diagrama muestra que acción realizaría para cada input del usuario. Se han elegido dichas teclas siguiendo el estándar que predomina en la industria actualmente, donde las teclas W-A-S-D suelen usarse para el movimiento del jugador [51] y el resto del teclado para distintas acciones, usando normalmente las teclas cercanas a estas para que el usuario pueda pulsarlas sin necesidad de desplazar su mano.

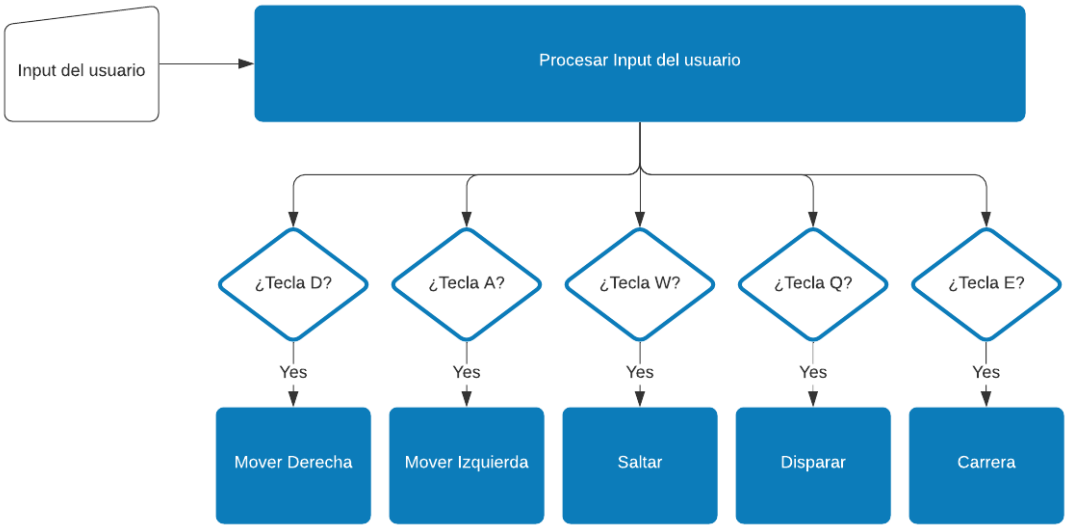


Figura 8: Diagrama de flujo para las entradas del jugador

Aparte del personaje principal existen diferentes personajes secundarios. Uno de ellos es el Rey Alubia. Esta judía es la mayor de todo su grupo por lo que es considerada como el rey de su mundo. Este personaje es el que le comenta al jugador su tarea, explicándole la trama al usuario a través de los propios personajes. El segundo personaje secundario sería Garbanzo. Como su nombre indica, este personaje sería otra legumbre distinta a las judías. Este personaje es un aventurero que lleva ya mucho tiempo explorando la casa. El jugador podrá encontrárselo de forma aleatoria durante sus partidas y este le dará consejos, funcionando cómo un ayudante durante las partidas y una forma de ayudar, como desarrollador, a los jugadores a través del juego. Ninguno de los personajes necesitará atributos de vida o daño ya que no entrarán en combate nunca. Ambos se moverán con una velocidad normal de 10 cm/s, al igual que el jugador.

4.1.2. Enemigos

Dentro de los enemigos encontramos tres personajes principales. Estos aparecerán en mapas generados aleatoriamente de forma procedural por lo que no podemos definir sus posiciones absolutas o sus acciones en función de ningún parámetro establecido con anterioridad, si no que deben de poder moverse en función de su propio entorno. Para cada enemigo se ha definido un patrón de movimiento para que cada uno se sienta distinto y el juego no se sienta monótono. Es importante mencionar que los enemigos siempre harán daño por contacto al jugador, es decir, que necesitan tocarle para hacerle daño. Continuamos definiendo los distintos enemigos.

4.1.2.1. Hormiga

La hormiga es el enemigo con el movimiento más simple. Esta se limitará a desplazarse de izquierda a derecha y viceversa cambiando de dirección cuando encuentre una pared o se

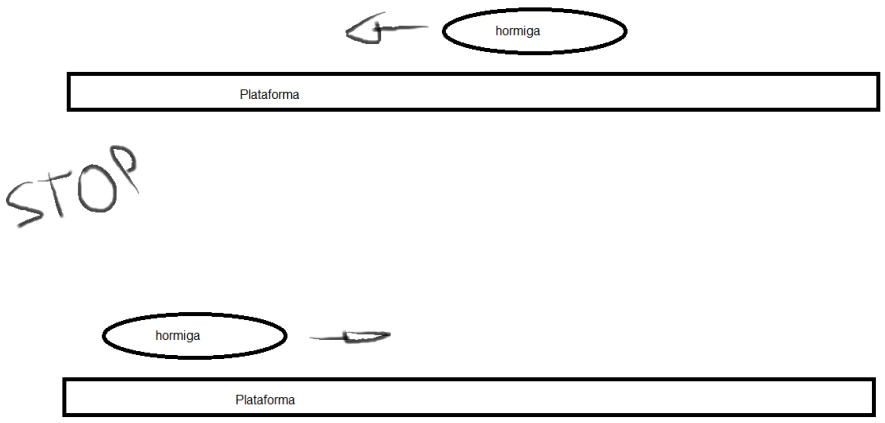


Figura 9: Diseño original sobre el movimiento de la hormiga

termine la plataforma. En la Figura 9: Diseño original sobre el movimiento de la hormiga se muestra la idea del diseño original para este enemigo.

Además de este movimiento, la hormiga se moverá más rápido si es que la alubia está en frente suya, es decir, si el jugador se coloca en la misma plataforma que la hormiga esta acelerará cuando le tenga delante pero nunca dejará de moverse de izquierda a derecha. La hormiga es el enemigo más robusto, es decir, que resiste mucho mejor los disparos del jugador, para compensar que su movimiento es muy predecible. Sus atributos son los siguientes:

- Vida: 50 puntos
- Velocidad: 4 cm/s
- Daño: 10 puntos

En la Figura 10: Diagrama de flujo de la toma de decisiones de la hormiga se muestra la toma de decisión de la hormiga para decidir como moverse. Esta función se ejecutaría una vez tras otra hasta que la hormiga se muriera o el juego acabara. Por esto, cada vez que se llegase a un estado de Fin o de realizar una acción, la línea de flujo volvería al primer estado. Se ha omitido para dejar los diagramas lo más sencillos y limpios posible.

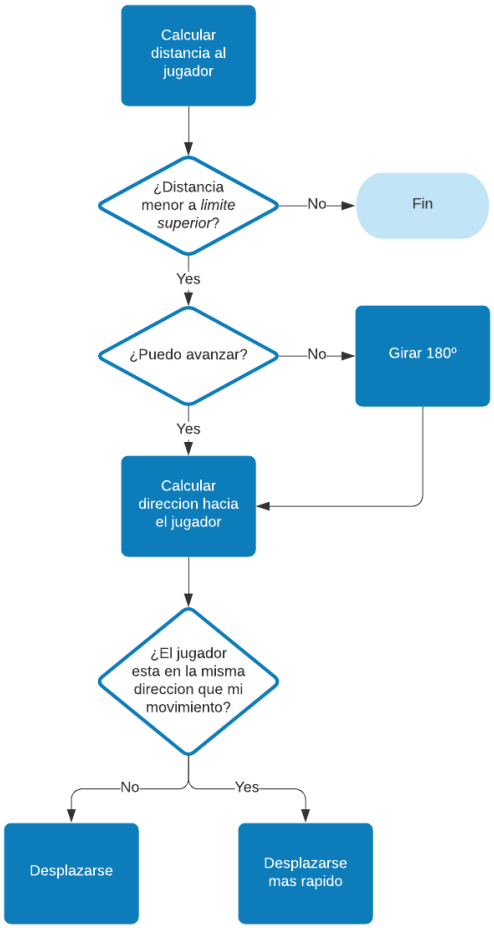


Figura 10: Diagrama de flujo de la toma de decisiones de la hormiga

4.1.2.2. Mosca

La mosca también es un enemigo simple ya que posee la capacidad de volar y puede desplazarse más fácilmente.

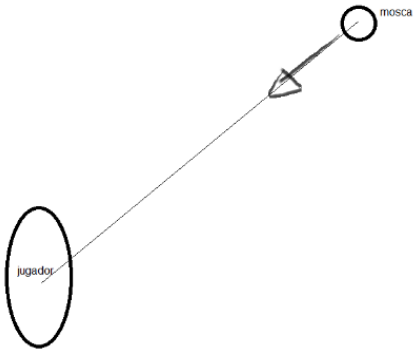


Figura 11: Diseño original sobre el movimiento de la mosca

El movimiento de la mosca será una línea recta directa hasta el jugador como se muestra en la Figura 11: Diseño original sobre el movimiento de la mosca. Este enemigo no puede atravesar paredes por lo que el jugador podría ser lo suficientemente rápido como para colocarse con un muro entre él y la mosca y así protegerse. Es el enemigo más rápido, pero también el más débil. Sus atributos son los siguientes:

- Vida: 15 puntos
- Velocidad: 7 cm/s
- Daño: 15 puntos

En la Figura 12: Diagrama de flujo sobre la toma de decisiones de la mosca se muestra la toma de decisiones de la mosca.

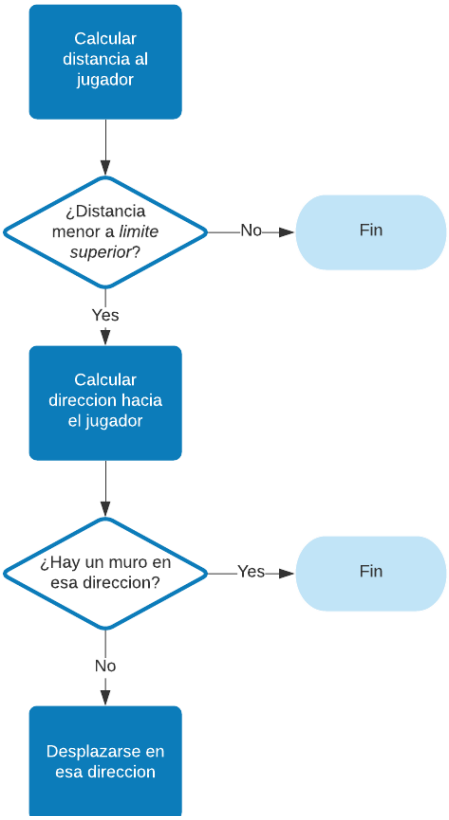


Figura 12: Diagrama de flujo sobre la toma de decisiones de la mosca

4.1.2.3. Araña

Por último, la araña es el enemigo con el comportamiento más complejo. Esta posee tres movimientos básicos: el desplazamiento horizontal, el salto vertical y el salto horizontal. El desplazamiento horizontal es simple, si el jugador está en línea con la araña, esta se desplazará hasta el jugador.

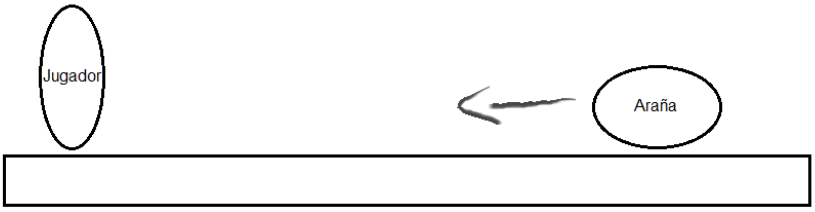


Figura 13: Diseño original del movimiento de la araña (Desplazamiento horizontal)

El salto vertical permite a la araña bajar o subir a una plataforma inmediatamente inferior o superior a ella si es que lo necesita para llegar al usuario. La araña no podrá descender o subir a una plataforma cuando la distancia entre ellas sea muy grande para evitar que esta pueda saltar grandes partes del escenario de forma muy rápida. En la Figura 14: Diseño original del movimiento de la araña (Salto Vertical) se muestra el movimiento por el cual la araña desciende una plataforma. El movimiento de subir una plataforma funcionaria de forma similar pero solo se daría cuando el jugador estuviera por encima de la araña.

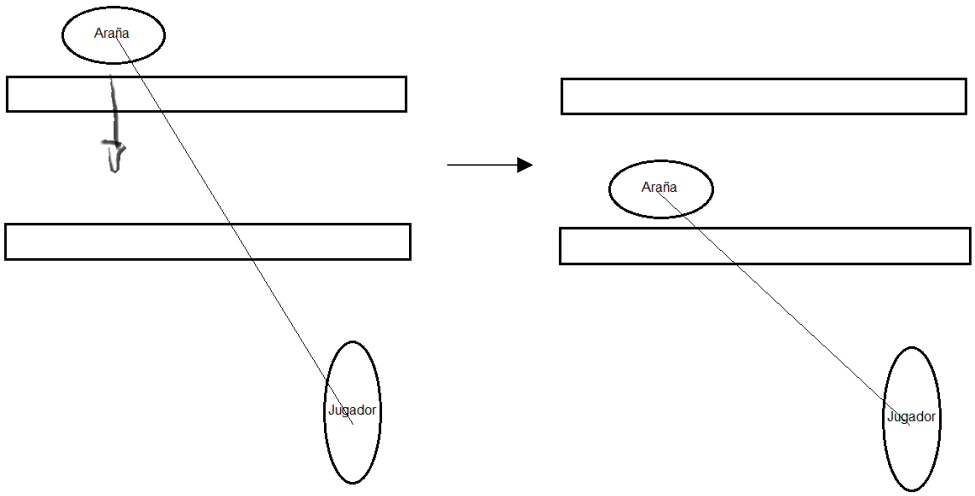


Figura 14: Diseño original del movimiento de la araña (Salto Vertical)

Por último, el salto horizontal permite a la araña saltar huecos entre plataformas. De forma similar al salto vertical, cuando la araña deba continuar en una dirección para encontrar al jugador y se encuentre con un hueco, podrá saltarlo.

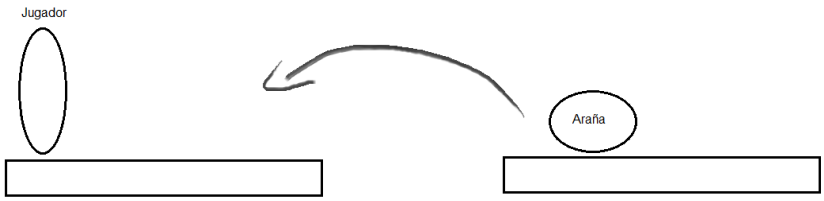


Figura 15: Diseño original del movimiento de la araña (Salto Horizontal)

Al tener tal abanico de posibilidades la araña es el enemigo más complicado de predecir de los tres por lo que será lento, pero tendrá una resistencia y daño medio.

- Vida: 35 puntos
- Velocidad: 5 cm/s
- Daño: 10 puntos

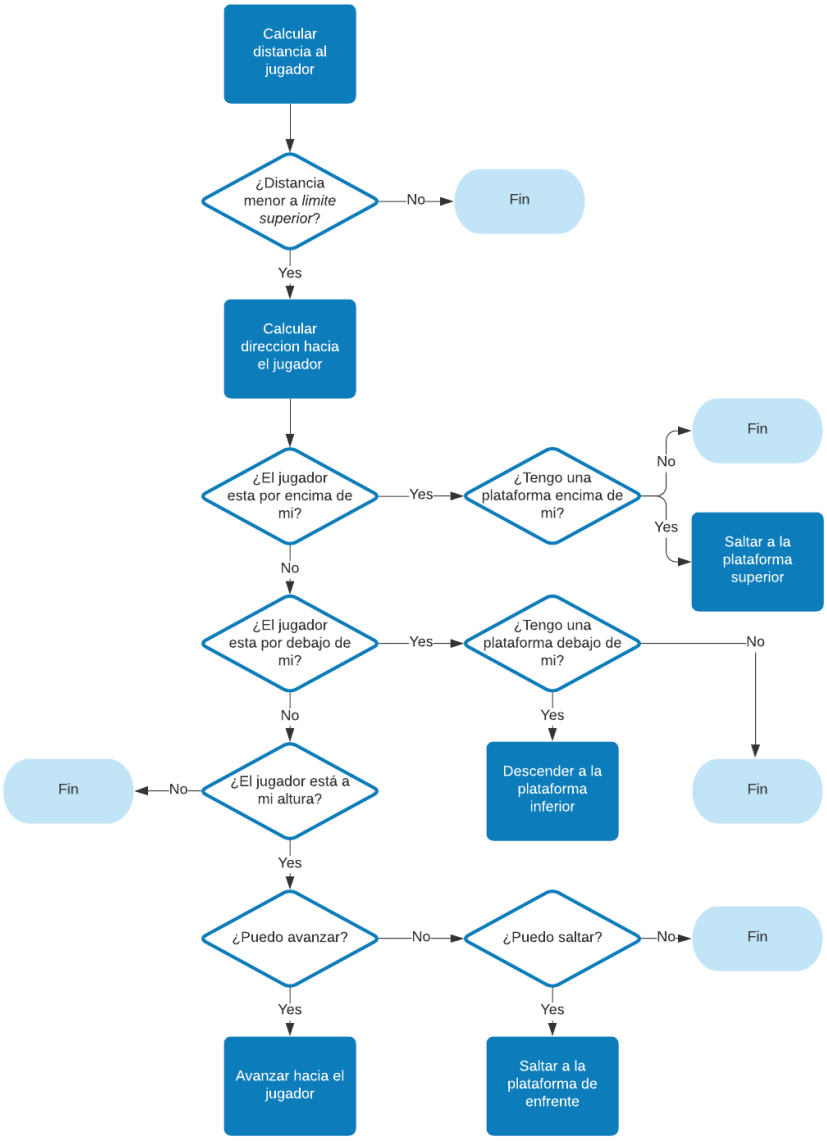


Figura 16: Diagrama de flujo sobre la toma de decisiones de la araña

4.2. Historia

El juego comienza cuando al jugador le han elegido como el siguiente explorador que intentará llegar al jardín para completar su misión. La trama se desarrollará según avance la partida ya que cada nivel que consiga avanzar será una nueva zona de la casa, comenzando en la cocina, pasando por el salón y el baño hasta llegar al jardín. Si esa judía muere durante su exploración, otra la sustituirá y volverá a comenzar el ciclo.

4.2.1. Tono

El juego tiene un tono cómico y juguetón a primera vista gracias a lo absurdo de su trama, pero indagando más se pretende transmitir la idea de continuar adelante por muchos baches que encuentres en tu camino, ya que al final lo conseguirás, dando un mensaje de superación haciendo uso de la mecánica intrínseca por la cual, una vez mueres, debes volver a comenzar. Mensaje que yo mismo me aplico y llevo aplicando sobre mi forma de trabajar desde hace años.

4.3. Progresión

La trama del juego será contada poco a poco cada vez que el jugador hable con algún personaje secundario. Esto obligará al jugador a volver a jugar una vez muera, aparte de para intentar completar el juego, para continuar uniendo piezas de la trama. Este método es el mismo que usa el reciente juego Hades [40], por el cual cada vez que empiezas una nueva *run*, es decir, otro intento de completar el juego, puedes hablar con varios personajes y descubrir detalles sobre la trama.

Como en este proyecto solo se plantea desarrollar los cimientos para más adelante continuar avanzando en el juego, este apartado y todos aquellos enfocados en la historia, no serán desarrollados en más detalle.

4.4. Gameplay

En esta sección se detallan todas aquellas características que definen como es una partida y que afectan al usuario.

4.4.1. Meta

La meta final del juego es la de plantarse en el jardín de la casa. Para ello el jugador deberá pasar por varias estancias de la casa derrotando a todos los insectos que se encuentre hasta poder cumplir su objetivo en el jardín.

Para esta versión del juego solo se necesitará matar a todos los enemigos de una estancia ya que no se dedicará tiempo a crear el final del juego en el jardín por no ser un objetivo principal del proyecto.

4.4.2. Interacción del usuario

Para este juego el usuario hará uso del ratón y el teclado. El ratón en los menús para navegar por ellos y seleccionar la opción que se quiera y el teclado para controlar al personaje principal.

4.4.3. Mecánicas

Las mecánicas definen qué acciones puede realizar el jugador y cuáles son sus efectos. Ya hemos mencionado con anterioridad las acciones del personaje, pero ahora pasaremos a explicarlas en detalle.

El movimiento horizontal, tanto en la dirección izquierda como a la derecha, es el más sencillo de todos. Cuando el jugador se desplaza lo hará a su velocidad y en la dirección que determine el usuario. La judía siempre estará mirando en la dirección en la que se esté desplazando. Esta acción podrá realizarse en cualquier momento, es decir, que incluso si se está en medio de un salto, el jugador podrá redirigir al personaje en el aire.

El movimiento vertical funcionará como un salto. El movimiento del jugador será el de un tiro parabólico, para esto haremos uso de las físicas de Unity que nos darán un resultado realista de la gravedad. Al contrario que el movimiento en horizontal, el salto solo se podrá usar si el jugador está tocando el suelo. Esto se ha diseñado así para evitar que el jugador pueda continuar saltando mientras está en el aire.

Las restantes dos habilidades tienen un temporizador, o como se suele denominar en inglés, un *cooldown*, o tiempo de enfriamiento. Este nombre proviene de las armas de fuego, debido a que estas necesitan enfriarse durante un pequeño periodo de tiempo antes de poder volver a activarse. Este es un concepto bastante utilizado en casi todos los juegos para evitar que el jugador no haga otra cosa que mantener pulsado el botón de atacar. Añadiendo un pequeño *cooldown* a las habilidades, obligamos al jugador a pensar estrategias, a buscar escapatorias o segundos planes, lo que según Ben Cowley y su equipo [52], ayuda a crear una sensación de juego más fluida y dinámica.

La carrera es una habilidad por la que el jugador se desplazará en la dirección en la que esté mirando durante medio segundo a una velocidad muy rápida. La velocidad será 5 veces la velocidad de movimiento normal del jugador. Esta acción se puede realizar en cualquier momento, pero tendrá un tiempo de enfriamiento de 1 segundo.

Por último, el disparo permite al jugador lanzar un proyectil en la dirección que está mirando. Este proyectil se desplazará a una velocidad constante de 10 cm/s hasta que colisione con algo o se aleje demasiado del jugador. En la Figura 17: Diagrama de flujo del funcionamiento del disparo se muestra el comportamiento del proyectil. Esta habilidad, al igual que la carrera, se podrá realizar en cualquier situación, pero tendrá un *cooldown* menor, de 0,75 segundos.

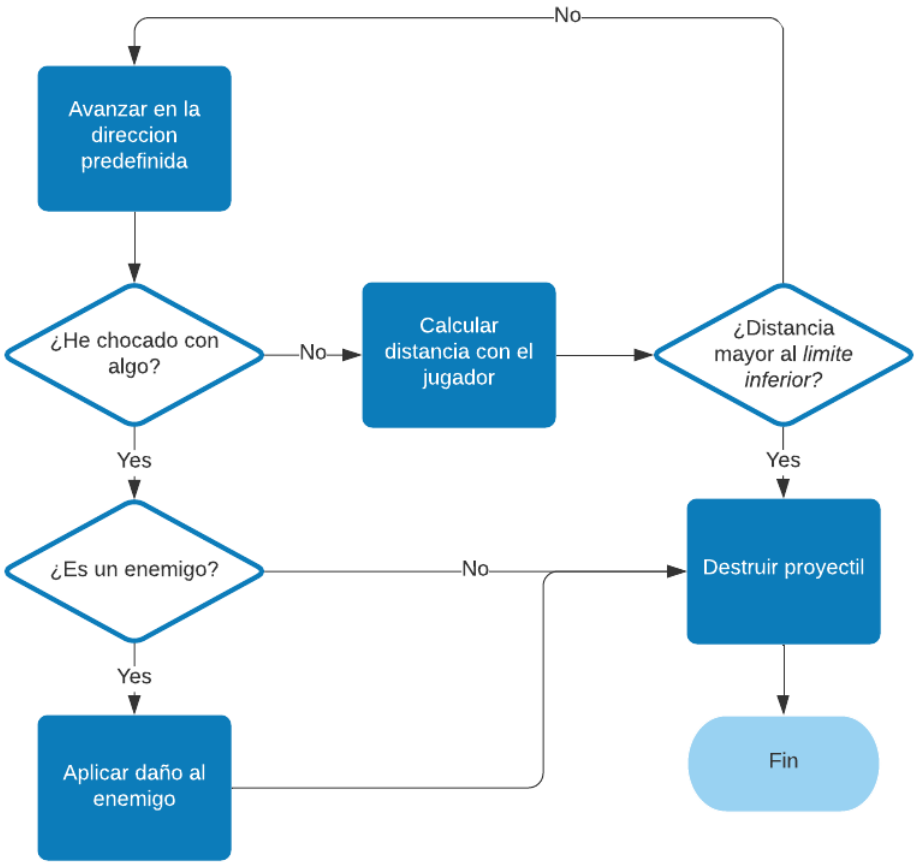


Figura 17: Diagrama de flujo del funcionamiento del disparo

4.4.4. Perder

Como ya se ha comentado en varias ocasiones, cuando el jugador muere comienza un nuevo intento de completar el juego por lo que no existe forma de perder el juego técnicamente ya que mientras el jugador siga intentándolo, el juego nunca mostrara una pantalla de *Game Over* o similares. En este caso se podría considerar como perder, el hecho de morir, ya que eso implica que la partida que se está jugando en ese momento se ha terminado y se tiene que empezar otra.

El juego no pretende ser competitivo con otros jugadores por lo que no existe ningún sistema de puntuación al eliminar a los enemigos.

4.5. Escenarios

El escenario será generado aleatoriamente de forma procedural y para esto será necesario definir una serie de salas que se combinarán de formas aleatorias para generar el diseño final del nivel. En la Figura 18: Boceto original del diseño de 10 salas se muestran las primeras salas que se diseñaron. En la parte superior se muestran las salas individualmente y en la inferior un ejemplo de mapa que se podría generar con estas. La sala en azul es la primera creada y donde se inicia la generación. El proceso de diseño de nuevas salas fue sucediendo a la vez que se implementaban ya que era necesario probar que todo funcionase correctamente al

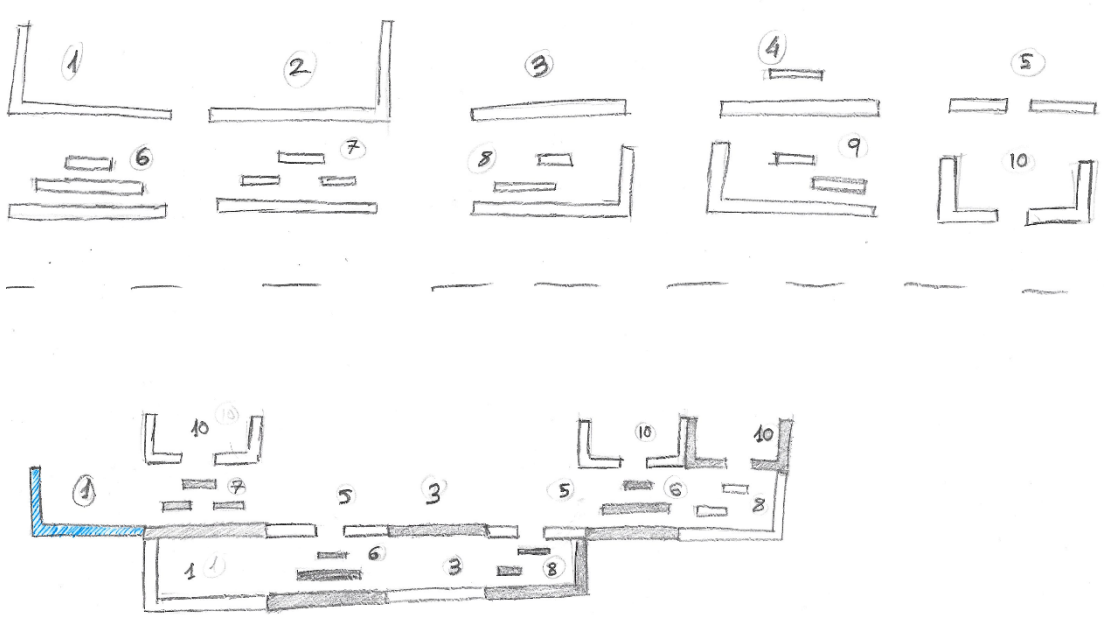


Figura 18: Boceto original del diseño de 10 salas.

añadir una o varias plantillas más.

En este primer boceto ya se puede observar el método por el cual se crearán los mapas. El juego eligirá una sala como la inicial, y a raíz de esa sala se generarán otras siempre y cuando se pueda, es decir, si la sala que se ha escogido está abierta por la derecha, se generará una sala a su derecha de forma aleatoria elegiendo entre todas las posibilidades. Repitiendo este proceso con todas las salas obtenemos un mapa aleatorio. Debido a como funciona este método, cuantas más plantillas de salas existan, más aleatorio se volverá el resultado final y es por esto que se buscará desarrollar como mínimo 30 salas distintas.

4.6. Estilo de arte

El arte del juego se engloba en un estilo *cartoon* donde los personajes poseen rasgos faciales exagerados para asemejarse a un dibujo animado. Los modelos se realizarían en Blender de forma autónoma.

4.7. Descripción técnica

El juego se desarrollará para una resolución de 1920x1080, pero será escalable a cualquier monitor con resolución inferior o superior mientras mantenga una ratio de 16:9. Se buscará optimizar el juego para conseguir una tasa de fotogramas de 60 o superior. Como límite inferior se establecen los 30 fps (*frames per second*) ya que, por debajo de este valor, el ojo humano es capaz de detectar las transiciones entre las imágenes lo que elimina la sensación de movimiento y continuidad al jugador.

Unity da la posibilidad de exportar el juego a la gran mayoría de plataformas, pero por el momento el juego será desarrollado para ordenador exclusivamente y será programado en el lenguaje C#.

5. Desarrollo de la aplicación

En la siguiente sección se detalla el proceso de desarrollo del proyecto. Primero analizaremos cada elemento relevante individualmente, seguidamente se detallará cómo todas esas piezas individuales se ensamblan para formar el producto final siguiendo el flujo de una ejecución normal del juego y por último se detallarán las pruebas que se han realizado para comprobar el correcto funcionamiento del programa.

La aplicación se ha desarrollado exclusivamente para Windows ya que no se poseía un terminal Linux o MacOS donde realizar las pruebas pertinentes.

5.1. Definiciones y palabras clave

Antes de comenzar a estudiar la implementación que se ha seguido, es importante definir ciertos términos relacionados con la programación en Unity para comprender mejor los siguientes apartados.

- **Script:** Programa o secuencia de comandos relativamente simple. En el marco de este proyecto hace referencia a los ficheros programados en C# que incorporan funcionalidades al juego.
- **Escena**: Es un conjunto de objetos en Unity. Este programa permite cambiar entre escenas por lo que puedes rápidamente eliminar todos los elementos que se ven en pantalla e introducir otros nuevos simplemente cambiando de escena.
- **Frame:** Traducción del inglés, cuadro. Hace referencia a una imagen renderizada por computador. En los juegos se renderizan decenas de cuadros por segundo para dar al espectador la sensación de movimiento.
- **Frame rate:** Cuadros por segundos. Representa la velocidad a la que se muestran estas imágenes.
- **GameObject**: Hace referencia a una entidad que exista en una escena del proyecto de Unity. En los scripts, puede usarse como un tipo de variables que almacena todo el objeto y aquellos componentes que tenga incorporados.
- **Canvas**: Elemento que permite renderizar imágenes o componentes 2D. En Unity se usa para crear representaciones 2D que se superponen al escenario 3D, por ejemplo, una barra de vida.
- **Vector2**: Tipo de variable que almacena un vector de dos posiciones. Los valores de cada posición se denominan x e y. Se puede acceder a ellos mediante el siguiente código: *nombreVector.x*.

- **Vector3**: Similar al caso anterior, pero con tres posiciones que almacenar. Igual que en el anterior, estos valores se denominan x, y, z y se acceden de la misma forma.
- **Start**: Método propio de Unity que es llamado en el primer frame. Cuando carga una escena y el script está activo, en el primer frame que se renderiza, esta función se ejecuta. No se vuelve a ejecutar nunca más.
- **Awake:** Método propio de Unity similar a Start pero se activa en el primer frame sin importar si este script esta activado o no.
- **OnEnable:** Método propio de Unity que se ejecuta cuando el script es activado.
- **Update:** Método propio de Unity que se ejecuta una vez cada frame. Este elemento depende del frame rate al que se esté ejecutando el juego que normalmente suele variar de un dispositivo a otro, por lo que puede no ser consistente en sus llamadas.
- **FixedUpdate:** Método similar a Update pero que se ejecuta un número predefinido de veces por segundo. Al contrario que el anterior método, este no depende del frame rate por lo que es una opción consistente y siempre se ejecuta las mismas veces por segundo.
- **OnSceneLoaded:** Método propio de Unity que se ejecuta al cargar una escena. Recibe como parámetros la escena y el modo en el que se ha cargado.
- **OnTriggerEnter:** Método propio de Unity que se ejecuta cada vez que el objeto al que está asociado colisiona con otro. Es necesario que el objeto al que vaya asociado también lleve asociado algún tipo de Collider.
- **Collider:** Traducido del inglés, colisionador. Es un componente que se puede asociar a GameObjects y que permite detectar o calcular colisiones. Tienen dos modos, uno en el que el colisionador funciona de barrera y nada puede atravesarlo y otro en el que puede ser atravesado y detecta esas colisiones. Algunos tipos son el BoxCollider, con forma de cubo o el SphereCollider, con forma de esfera.
- **RayCast:** Función incorporada con el motor de físicas de Unity que lanza un rayo y devuelve si ha chocado contra algún elemento. Se puede especificar por parámetro el origen de este rayo, su dirección y la distancia máxima que debe recorrer

5.2. Implementación de Beans Out

Para explicar la implementación que se ha seguido en este proyecto debemos entender un elemento importante de la programación en Unity. Alternativamente a como se podría esperar, al programar en Unity no existe un script principal que contenga todo el código fuente necesario. Al contrario, Unity permite crear pequeños scripts muy especializados en ciertas tareas y asociarlos con distintos elementos del juego, por ejemplo, el jugador puede tener un script que controle el movimiento, otro para disparar y otro que controle la vida. De esta forma

es sencillo estructurar y conocer rápidamente que hace cada script además de ser una buena manera de reutilizar un mismo código varias veces como veremos más adelante.

En los siguientes diagramas de clases, una clase representará un script y debido a esto, veremos que hay varias clases que se asocian a varios objetos del juego. En los diagramas de clases también se podrán observar los símbolos "-/" en las variables o atributos de la clase. Esto significa que el valor es privado, como indica el "-" pero se puede modificar desde la aplicación de Unity. para esto se hace uso de la etiqueta *SerializeField* que hace que dicho parámetro aparezca como un desplegable en el componente del script. La Figura 19: Captura de la interfaz

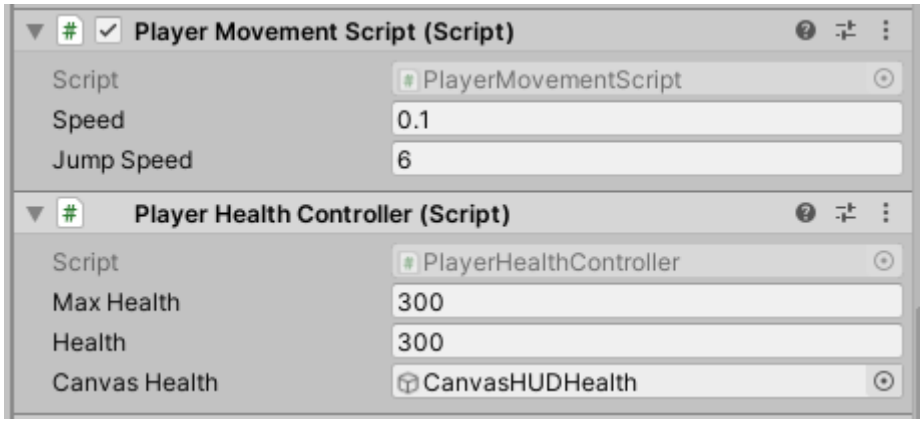


Figura 19: Captura de la interfaz de Unity donde se muestran los componentes PlayerMovementScript y PlayerHealthController.

de Unity donde se muestran los componentes PlayerMovementScript y PlayerHealthController. ilustra esto.

Dividiremos toda la implementación según el elemento al que más aporten dichos scripts y en el siguiente apartado se dará una visión general del proyecto y cómo todo se une para crear la aplicación final.

5.2.1. General

En esta sección se detallan aquellos scripts que afectan al conjunto del juego o que se pueden reutilizar para distintas funciones.

Comenzamos mencionando el GameManager, un script que como su nombre indica es el encargado de controlar el juego. No se indagará mucho en él ya que actualmente no posee muchas funciones, pero cabe destacar que este script es el más relevante del proyecto a la hora de añadir nuevos contenidos ya que, este proceso es el que se encarga de unir todos los contenidos del juego y hacer transiciones desde

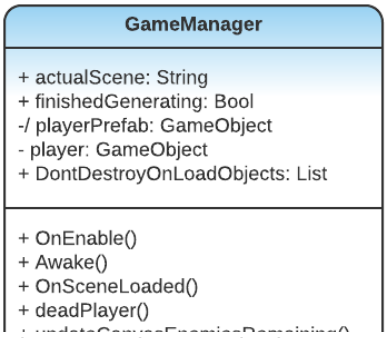


Figura 20: Diagrama de clases para GameManager

distintas escenas, como por ejemplo pasar del menú al propio juego, un cambio que a primera vista parece sencillo, pero donde hay que tener en cuenta ciertas variables para que todo funcione correctamente. Los puntos más importantes son las funciones OnSceneLoad y deadPlayer. La primera de ellas es un método donde se realizan ciertas acciones según la escena, como, por ejemplo, cuando cambiamos del menú al juego, este algoritmo se encarga de instanciar al jugador ya que en el menú no lo necesitamos, pero después sí. La segunda función se usa como dice su nombre para captar cuando muere el jugador. Esta función elimina el estado del juego y lo devuelve de nuevo al menú para que el usuario pueda comenzar otra partida.

Continuando, existen algunos scripts que han sido usados en varios objetos distintos, estos son healthController y HazardTriggerHandler. Comenzaremos hablando sobre el primero de ellos.

La clase healthController es la encargada de controlar la vida de aquel objeto al que esté asociada. Todos los enemigos tienen esta clase asociada para mantener un control sobre su vida. Es una clase sencilla con un atributo; la vida, que se puede definir en Unity a un valor cualquiera por lo que podemos usar este mismo script, pero variando ese parámetro para cada enemigo. Después tenemos tres funciones, getHealth, setHealth y checkLiving. Esta última es la más importante ya que es la que comprueba cuando el enemigo ha muerto y eliminarlo.

La clase HazardTriggerHandler también va asociada a los enemigos y funciona de forma contraria, esta clase es la encargada de infringir daño al jugador. Este script tiene un atributo damage y una función OnTriggerEnter propia de Unity que se activa siempre que dos objetos colisionan. Dentro de esta función se comprueba que haya chocado contra el jugador y se le resta a su vida el daño definido en dicho script.

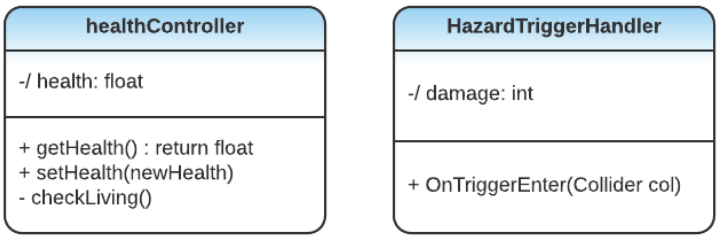


Figura 21: Diagrama de clases para healthController y HazardTriggerHandler

Ambas clases están relacionadas con los enemigos, pero se ha decidido mencionarlas aquí ya que en un futuro pueden ser usadas para otros elementos como trampas que hagan daño al jugador u otros personajes a los que haga falta mantener contralada su vida. Por esto son considerados scripts de uso general a pesar de que en esta primera versión solo se usen en los enemigos.

5.2.2. Personaje

El personaje posee varias clases ya que cada habilidad principal de este se ha separado en un script individual para mejor comprensión de sus tareas. Estos son:

- PlayerMovementScript
- PlayerHealthController
- PlayerDashController
- PlayerShootController
- PlayerAlternativeMovementController

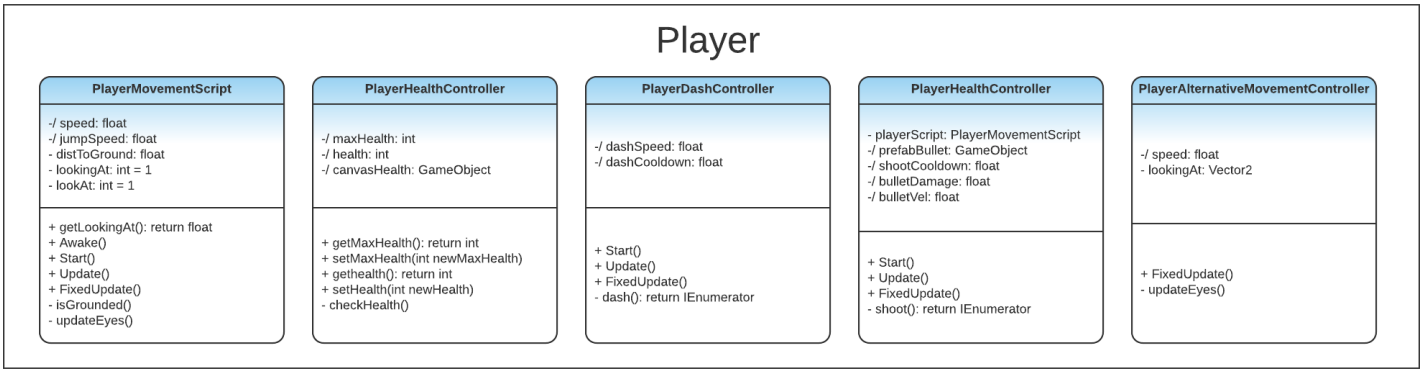


Figura 22: Diagrama de clases para el jugador

5.2.2.1. PlayerMovementScript

Comenzaremos explicando PlayerMovementScript y seguiremos en orden descendente. En este primer script se define el movimiento básico del personaje que engloba las habilidades, mencionadas con anterioridad, movimiento horizontal y movimiento vertical. Cada uno de movimientos se realiza de forma distinta por lo que comenzaremos explicando el desplazamiento horizontal. Para él se ha hecho uso de la función FixedUpdate. En esta se comprueba si el usuario ha pulsado la tecla A o D, que fueron las escogidas para desplazarse a la izquierda y derecha, y si es así modifica la posición del jugador restando o sumando en el eje X el valor de la velocidad. Debido a este método, es necesario realizar este cálculo dentro de la función FixedUpdate ya que no queremos que la velocidad dependa del frame rate.

En contraposición, para el salto usamos la función Update. En esta comprobamos cuando el usuario ha pulsado la W y si es así añadimos una fuerza en vertical hacia arriba. Al aplicar una fuerza, el jugador se mueve de forma distinta a como lo haría si modificamos su posición, ya que es Unity el que se encarga de calcular como esa fuerza afecta a la velocidad y como la gravedad modifica la trayectoria. Sin embargo, con este método no solucionamos uno de los requisitos que impusimos anteriormente y es que el jugador solo podría saltar si está tocando el suelo. Para esto hacemos uso de la función RayCast de Unity la cual lanza un rayo en

una dirección y devuelve si dicho rayo ha chocado con algo. De esta forma podemos lanzar un rayo desde la base del jugador y así saber cuándo está tocando el suelo.

El resto de variables y funciones sirven para ajustar la apariencia del personaje, es decir, hacer que el personaje esté mirando hacia donde se desplaza. Estas funciones comprueban si el usuario ha cambado de dirección y si es así, giran 180 grados el modelo del personaje.

5.2.2.2. PlayerHealthController

Este script funciona de forma similar al controlador de la salud general que se usa con los enemigos, pero era necesario implementar algunos cambios, por lo que la tarea de crear un script propio para el jugador era más sencilla.

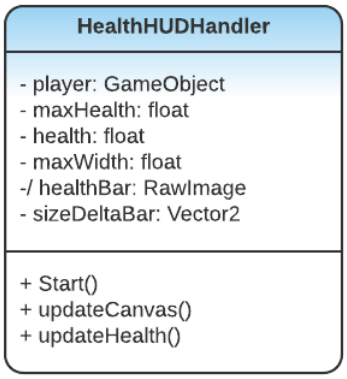


Figura 23: Diagrama de clases para HealthHUDHandler

Todo es igual que en el anterior script excepto el método que comprueba si el jugador está vivo o no. Si el jugador tiene la salud menor o igual a 0, se llamará a la función deadPlayer del GameManager para indicar que el jugador ha perdido. Además de este pequeño cambio también tenemos un problema que resolver y es actualizar el canvas donde se muestra la vida del jugador. Para ello se ha añadido a los métodos de setHealth y setMaxHealth una llamada aun script secundario que controla este canvas para que lo actualice con los nuevos valores de vida. Dicha clase se llama HealthHUDHandler. Para cambiar la interfaz se realiza el cálculo del porcentaje de vida que le queda al jugador haciendo el cociente entre la vida actual y la vida total y se escala la barra de vida con esa proporción. De esta forma la barra de vida parece reducirse según el daño que recibe el usuario.

5.2.2.3. PlayerDashController

Para realizar la acción de carrera se aplicó un funcionamiento muy similar al salto aplicando una fuerza al personaje, pero con algunas modificaciones. El primer cambio es que a la comprobación de que se pulsa la tecla E, se le añade la comprobación de que el *cooldown* debe estar a 0, mientras que en el salto comprobábamos que estuviera tocando el suelo. Este *cooldown* se calcula restando a una variable el tiempo que ha pasado desde la última llamada a la función Update, lo cual podemos hacer con la variable del sistema Time.deltaTime. Una vez la variable ha llegado a cero o menos dejamos de restar para no hacer cálculos innecesarios. De esta forma podemos mantener la variable actualizada cada frame y una vez se active la habilidad podemos volver a colocarla a su valor inicial comenzando así de nuevo la cuenta atrás.

Para realizar el movimiento del *dash* necesitamos que el jugador se desplace muy rápido por un corto periodo de tiempo y después vuelva a su velocidad anterior. Para ello no podemos usar solo el mismo método que el salto ya que lo que produciría sería un empujón en una dirección que se iría perdiendo por el rozamiento. Para eso se hace uso de las corrutinas. Estas funciones se ejecutan en segundo plano al resto del juego y permiten esperar una cantidad especifica de tiempo sin parar la ejecución del juego. Entonces, lo que hacemos es aplicar la fuerza correspondiente en la dirección que mira el jugador. Esperamos a que pasen 0'3 segundos y eliminamos la fuerza que quede aplicada contra el jugador. De esta forma en el juego obtenemos un efecto de propulsión que solo dura un instante y que acaba con el jugador quieto en el sitio para que pueda moverse y no decelerando por rozamiento hasta que pierda todo el momento.

5.2.2.4. PlayerShootController

Para activar la habilidad de disparo el usuario debe pulsar la tecla Q. El proceso de esta habilidad es muy similar a la anterior. Usando el método Update podemos ir restando tiempo al *cooldown* para mantenerlo actualizado. Cuando esté en cero o menos, el jugador podrá disparar. Aquí se llama a la función que realiza el disparo, que es el mayor cambio entre ambas.

Este método crea una instancia del proyectil creando un clon de una bala de ejemplo (PrefabBullet), en la posición del jugador. Esta bala lleva asociado un script que controla su movimiento llamado bulletHandler. A este script se le dan valores de dirección, velocidad y daño en función de los valores del jugador para estos atributos, es decir, la bala hereda los valores del jugador, de esa forma si en un futuro se implementa un método para dar mejoras al jugador y este consigue aumentar su daño, no habrá que aplicar este cambio a las balas, solo al personaje principal.

Este script es muy sencillo. Mientras se mantenga a una distancia determinada del jugador, se desplazará en la dirección predefinida sin parar a velocidad constante. Este script incluye la comprobación de colisión con otro objeto. Cuando eso pasa, se comprueba que sea un enemigo. Si es un enemigo, se accede a su controlador de vida y se le resta el daño correspondiente y se destruirá la bala. En cambio, si se choca con otro elemento, como una pared, por ejemplo, simplemente se destruirá la bala.

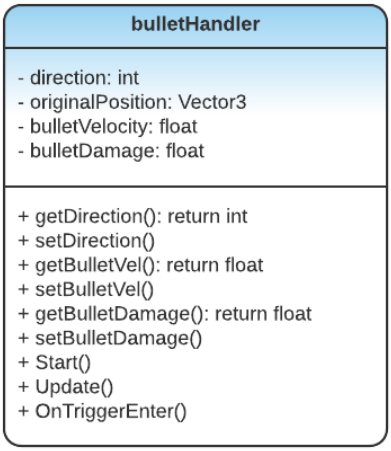


Figura 24: Diagrama de clases para bulletHandler

5.2.2.5. PlayerAlternativeMovementController

Este script es usado solo en un momento concreto del juego en el que se necesita desplazarse en el eje Z y romper con el movimiento 2D que existía hasta ese momento en el juego. Por esto, el jugador no puede saltar y la W pasa usarse para desplazarse en el eje Z. Además, se añade la S como botón para retroceder en este nuevo eje. El cambio más sustancial lo tenemos en el método updateEyes, donde antes tenía que ajustar los ojos a la derecha o a la izquierda, ahora puede rotar el personaje cada 45 grados según que movimiento que esté realizando el usuario. Se realiza cada 45 grados ya que el jugador puede pulsar a la vez dos teclas, como por ejemplo W y D, para desplazarse en diagonal, por lo que necesitamos que el personaje mire en esa dirección. Esto se logra con comprobaciones sucesivas del movimiento del personaje.

5.2.3. Enemigos

Cada enemigo está compuesto de 3 clases principales. Un controlador de su movimiento, único para cada uno y los dos scripts mencionados anteriormente que controlan su vida y su daño. De esta forma, en este apartado nos centraremos exclusivamente en la implementación del movimiento.

5.2.3.1. Hormiga

La hormiga posee la capacidad de detectar cuando llega al final de una plataforma, ya sea porque se encuentra con un muro o porque esta se acaba. Debido a estas dos situaciones, necesitamos comprobar ambas. Para ello se hace uso de los rayos del motor de físicas de Unity como ya habíamos mencionado anteriormente. En este caso debemos desplazar los rayos para que se proyecten desde la parte delantera de la hormiga. Estos rayos funcionarían como el propio sentido de vista de las hormigas. Uno de ellos se lanzaría en horizontal en la dirección que se mueve la hormiga y comprobaría muros en frente suya. Mientras que el segundo se proyectaría contra el suelo y detectaría cuando este se acaba. Con estas dos funciones implementadas en los métodos IsWalled y IsGrounded podemos programar el movimiento del enemigo.

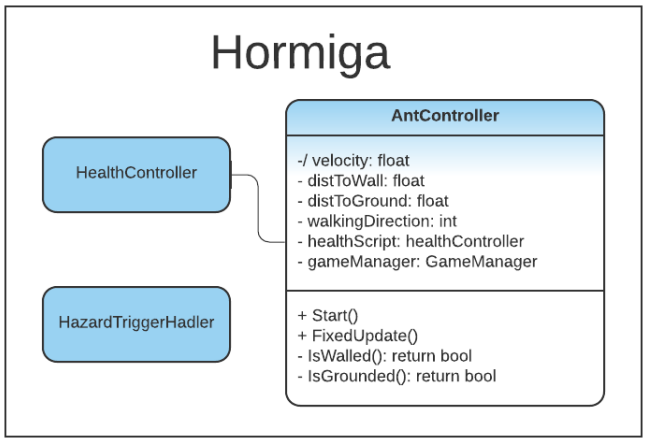


Figura 25: Diagrama de clases para la hormiga enemiga

Esto se realiza en la función FixedUpdate. La hormiga siempre se va a desplazar su velocidad en la dirección de su movimiento, solo necesitamos cambiar dicha dirección en el

momento adecuado. Para ello se comprueba si se cumplen algunas de las dos condiciones anteriores y si es así, se da la vuelta al objeto y se cambia el sentido de su movimiento.

Por último, necesitamos implementar que este enemigo se desplace el doble de rápido si el jugador está delante suya. Para esto se calcula el vector de dirección entre la hormiga y el jugador y se comprueba si este apunta en la misma dirección que en la que se mueve el enemigo. Si es así, se desplaza el doble en dicha dirección.

5.2.3.2. Mosca

La mosca es probablemente el enemigo con el comportamiento más simple, pero a la vez que requiere algunos elementos especiales y esto se debe a su capacidad de volar. Esto ayuda mucho ya que su movimiento es más sencillo, pero genera problemas al moverse por cualquier sitio, por lo que hay que tener en cuenta más situaciones de las normales.

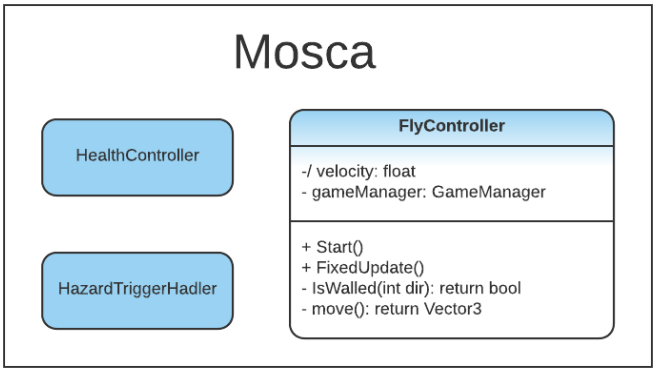


Figura 26: Diagrama de clases para la mosca enemiga

Este enemigo se desplaza por el camino más corto hasta el jugador. Para ello necesitamos obtener un vector normalizado que indique la dirección entre la mosca y el personaje principal. Necesitamos un vector normalizado para que al multiplicarlo por la velocidad y obtener el desplazamiento, el módulo del vector de desplazamiento sea igual a la velocidad. Si no normalizásemos este vector, la mosca se desplazaría de forma inconsistente según donde este el jugador.

También necesitamos comprobar si este enemigo está bloqueado por una pared en alguna dirección. Esto se realiza comprobando los valores del vector de dirección y comprobando si la mosca tiene paredes en aquellas direcciones a las que se va a mover. Por ejemplo, si tenemos un vector de movimiento (0’3, 0’95) sabríamos que debe desplazarse a la derecha y hacia arriba, por lo que comprobaríamos la existencia de paredes en dichas direcciones. Si en alguna de ellas hubiera un obstáculo, la mosca no se desplazaría en ese eje. Si en el ejemplo anterior, hacia arriba existe un muro, la mosca se desplazaría con el siguiente

vector (0’3, 0). Estas comprobaciones se realizan en el método IsWalled y la función move es la encargada de calcular todo lo relacionado con los vectores de desplazamiento.

5.2.3.3. Araña

La araña es el enemigo con el comportamiento más complejo, pero a su vez se utilizan mayoritariamente elementos que ya hemos usado antes.

Lo primero a destacar es que la araña se desplaza como la hormiga, por el suelo, pero esta lo hace siempre persiguiendo al jugador. Como no posee la capacidad de volar, había que dotarla de mecanismos para que pudiera navegar el escenario hasta el jugador. Estos son la capacidad de ascender o descender de plataformas y de saltar plataformas.

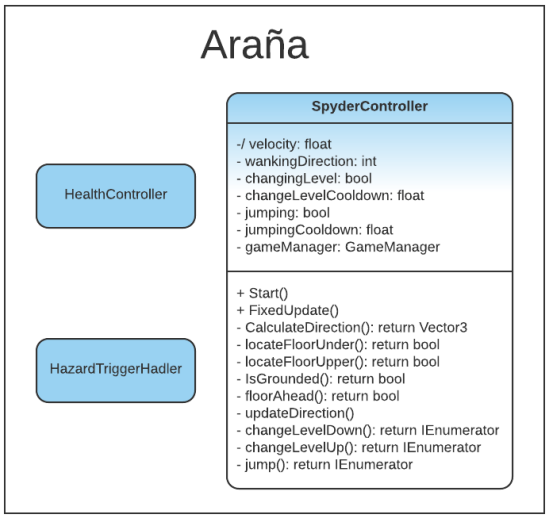


Figura 27: Diagrama de clases de la araña enemiga

Lo primero que se realiza es calcular la dirección hacia la que se encuentra el jugador ya que este dato será clave para elegir que acción tomar. Si el jugador está por debajo o encima de la araña entones el enemigo bajará o subirá una plataforma respectivamente si es que es posible. Para comprobar esta posibilidad se han implementado los métodos locateFloorUnder y locateFloorUpper. Ambos hacen uso de los rayos ya mencionados para comprobar si existe alguna plataforma encima o debajo suya. Si esto se cumple, se realizaría la acción de changeLevelUp o changeLevelDown dependiendo de la situación. Estas corrutinas desplazan a la araña hacia arriba o abajo hasta que se encuentre en la nueva plataforma. Esto se realiza de forma rápida para que se asemeje a un salto del enemigo. Mientras se realiza esta corrutina, la variable interna changingLevel se mantiene como verdadero. Esto se usa para que no pueda realizar ninguna otra acción mientras esta se lleva a cabo.

En el caso de que el jugador se encuentre a la misma altura que la araña esta se desplazará por la plataforma en dicha dirección. Esto también sucede si el jugador está en diagonal, pero no se puede tomar un camino hacia arriba o hacia abajo. Al igual que la hormiga, la araña comprueba siempre que haya suelo delante de ella y si no es así no puede seguir avanzando. Esto se realiza por medio de otro rayo. En el caso de encontrarse sin posibilidad de seguir caminado, puede que necesite saltar para llegar al otro lado. Para esto se utiliza la función floorAhead que detecta si existe una plataforma delante de la araña. Si es así se ejecuta la

corrutina jump que aplica una fuerza a la araña similar al funcionamiento del salto en el jugador. De esta forma, la araña aterriza en la plataforma adyacente y podría continuar su movimiento.

En la Figura 28: Captura de los rayos usados por la araña se muestra una captura de la escena de pruebas donde se pueden apreciar todos estos rayos. Los rayos de color rojo son los encargados de detectar plataformas arriba o abajo, el rayo verde, comprueba la existencia de suelo delante de la araña, en este ejemplo la araña no se movería hacia la derecha ya que ese rayo no detecta ningún suelo. Por último, el rayo azul se proyecta a la altura del suelo que hay debajo de la araña y en la dirección en la que está mirando para detectar si más adelante

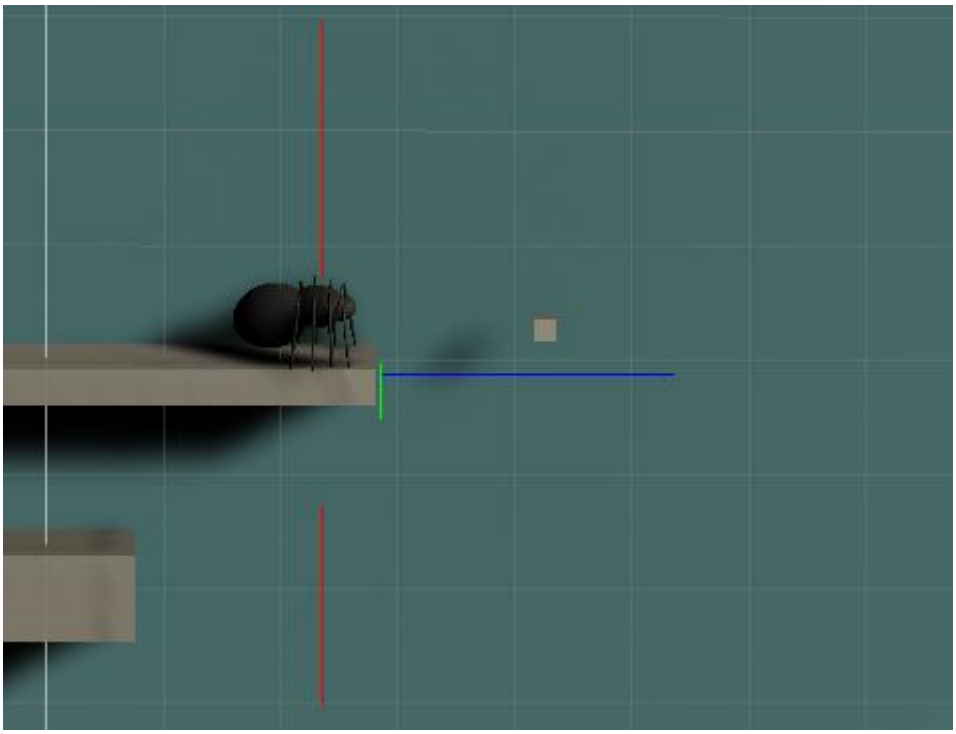


Figura 28: Captura de los rayos usados por la araña

comienza otra plataforma a la que saltar.

Por último, las funciones restantes se encargan de hacer algunos cálculos concretos, como por ejemplo el método calculateDirection que se encarga de calcular el vector de dirección entre la araña y el jugador.

5.2.4. Escenario

Para la generación procedural del escenario se han usado dos scripts; RoomTemplates y RoomSpawner. A estas dos clases se añaden otras dos; AddRoomToList y InstantiateEnemies para completar la generación añadiendo enemigos y un sistema para controlar el tamaño del escenario.

Comenzaremos explicando el funcionamiento general de la generación aleatoria de salas y después detallaremos los elementos que se han aplicado para crear los escenarios finales.

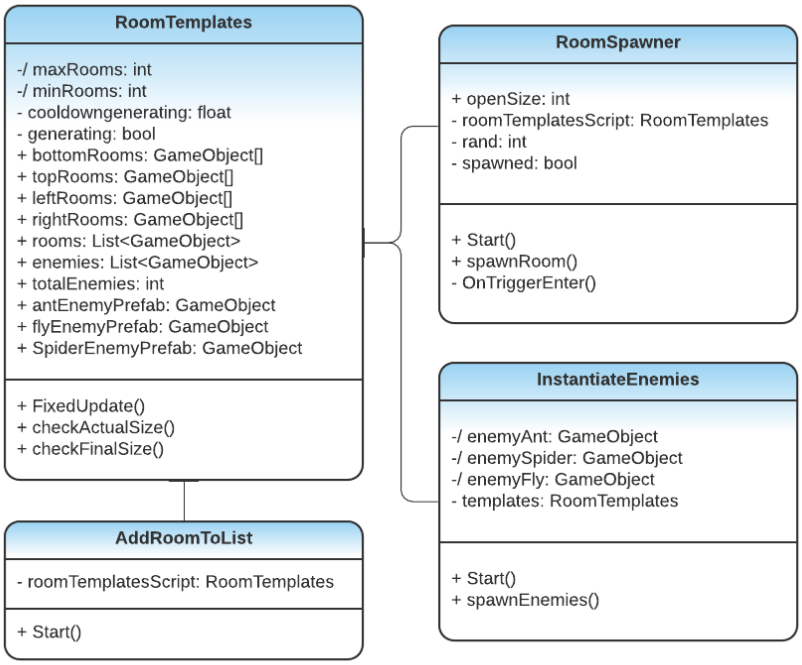


Figura 30: Diagrama de clases relacionadas con la generación de escenarios

Lo primero que analizaremos serán las salas que se han usado de plantilla. En total se han generado 33 salas distintas, que pueden dividirse en 4 grupos: salas con abertura a la izquierda, salas con abertura a la derecha, salas con abertura arriba y salas con abertura abajo. Una sala puede estar en varias de estas categorías ya que puede estar abierta por varios lados a la vez. Por ejemplo, la sala que se muestra en la Figura 29: Sala LRT , es una sala abierta por arriba, derecha e izquierda.

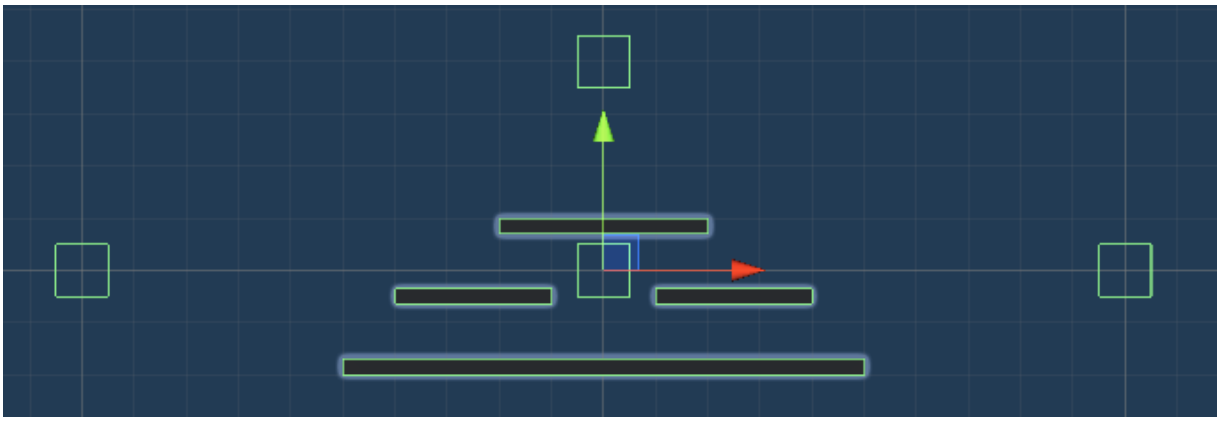


Figura 29: Sala LRT

Esta sala es denominada LRT por sus aberturas*, left, right, top*. Todas las plantillas que se han diseñado tienen el mismo tamaño, 10x5. Esto es necesario ya que, una vez se unan para generar el escenario, si algunas fueran de distinto tamaño, no encajarían y dejarían huecos o se entrelazarían con otras salas.

En la imagen pueden verse cuatro cuadrados verdes. El central se usa para detectar dicha sala si ya se ha generado, pero los otros tres son los encargados de generar las salas adyacentes. Estos puntos llevan asociados el *script* RoomSpawner que genera una sala en dicho punto, pero no podemos generar cualquier tipo de sala ya que una sala como la que observamos en la Figura 31: Sala R, vista isométrica , no debería aparecer en el punto de la derecha. Para esto se determina qué tipo de salas necesitamos en cada punto, es decir, para el cuadrado de la derecha, necesitamos generar una sala con, al menos una abertura en la parte izquierda.

Para esta tarea está diseñada la clase RoomTemplates. Una de las funciones de este script es mantener una lista con los cuatro grupos que hemos descrito antes. De esta forma el punto derecho, al crear una sala solo debe escoger aleatoriamente de la lista de habitaciones con aberturas a la izquierda e instanciarla. Esa sala a su vez puede tener más puntos de creación, por lo que este proceso puede seguir de forma indefinida.

Como se ha mencionado, el punto central se usa para detectar cuando se ha generado ya una sala. Cuando se genera una sala, por ejemplo, con el caso anterior, si colocamos una habitación con abertura por el lado izquierdo a la derecha de la sala, esa también tendrá un punto de creación de salas hacia la izquierda donde ya existe una sala. Para esto se comprueba si ese punto choca con el central de la sala que ya existe y si es así se elimina para que no generé ninguna habitación. Como todas las salas son del mismo tamaño, los puntos siempre coinciden para que estas comprobaciones se puedan realizar.

Por probabilidad, gracias a que existen salas que solo poseen un solo punto de *spawn*, como la de la Figura 31: Sala R, vista isométrica, el mapa no debería ser infinito. Esto es porque estas salas funcionan como tapón ya que se generan con la abertura conectando y no posen ningún otro punto donde crear salas. Sin embargo, los mapas si pueden llegar a ser lo suficientemente grandes como para causar un problema en el ordenador, por la cantidad de procesos que se ejecutan simultáneamente mientras se crean. Además, al igual que existe la posibilidad de generar un mapa enorme, existe la posibilidad de generar uno muy pequeño, con solo dos o tres salas. En el juego queremos evitar estos casos para que las partidas sean aleatorias dentro de unos márgenes.

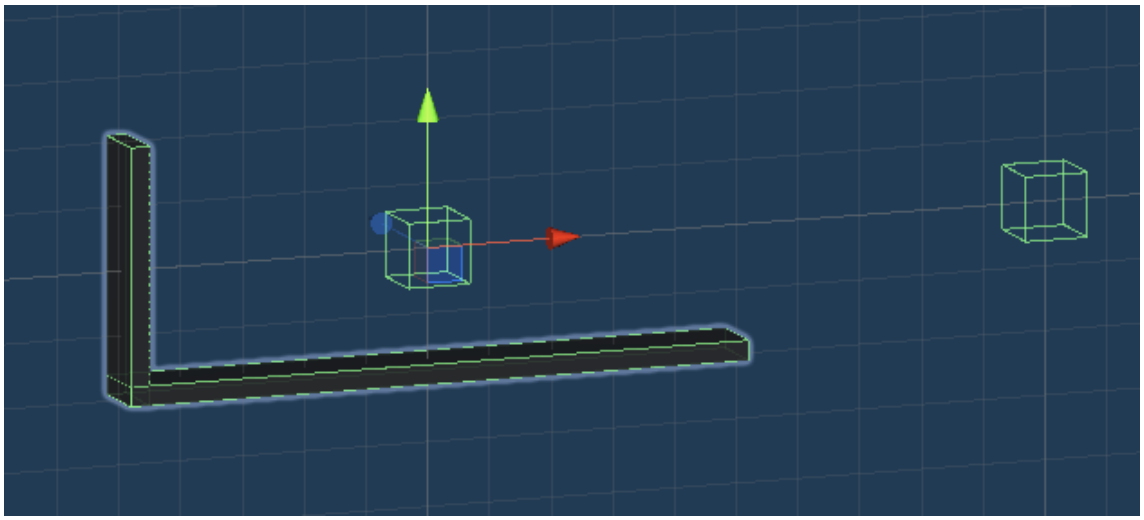


Figura 31: Sala R, vista isométrica

Para ello existen los métodos checkActualSize y checkFinalSize que se encargan de comprobar que el número de salas generadas se mantengan en unos límites. Estos límites se determinan en las variables maxRooms y minRooms. En la versión definitiva el juego genera mapas entre 10 y 20 salas. Antes de explicar cómo funciona este método debemos explicar la clase AddRoomToList.

Esta se encarga de añadir una habitación que acaba de ser generada a la lista de salas. Aquí es donde entran los límites superiores e inferiores. Lo que se realiza es comprobar el tamaño de la lista por si hemos superado el límite cada vez que instanciamos una nueva sala. Si es así eliminamos todas las estancias y volvemos a generar otro posible escenario. Esto se realiza en la funcion checkActualSize. checkFinalSize se ejecuta cuando ya se ha terminado de generar el mapa para comprobar si el número de salas generadas es mayor al límite inferior. Si no es así, se vuelve a generar un mapa.

Este método no es el más eficiente si se busca un número de salas exacto, pero para este proyecto buscamos un rango relativamente amplio por lo que es una solución aceptable. Durante la implementación de estos algoritmos se probaron distintos valores para los límites superiores e inferiores. Se obtuvo que para un rango de 4 o menos salas entre el límite superior e inferior, el juego reseteaba el proceso de creación una media de 70 veces, que se traducían en unos 2 minutos de espera, y aumentaba rápidamente según el rango disminuía. Con unas 10 habitaciones de diferencia, el juego necesitaba reiniciar la creación solo 6 veces de media, alrededor de 5 segundos. Es por eso que se eligió el rango de 10-20, ya que daba buenos resultados y todos los escenarios que generaba estaban dentro de unos márgenes aceptables.

Por último, cada sala posee también unos puntos que sirven como lugar para instanciar enemigos. Este método se lleva a cabo por la clase InstantiateEnemies. En esta se recorren todos

estos puntos y se elige un enemigo que generar para cada uno de ellos. Las probabilidades de generar cada uno de ellos son de:

- Hormiga: 14’28 % (1/7)
- Mosca: 14’28 % (1/7)
- Araña: 14’28 % (1/7)
- Nada: 57’14 % (4/7)

Existe la posibilidad de no instanciar ningún enemigo para no generar demasiados. La forma de pensar en este sistema es crear muchos puntos posibles para generar enemigos, pero no generar en todos ellos para que cada vez que se generen sean enemigos distintos y en posiciones distintas. Es por eso que más de la mitad de las veces no se genera ningún enemigo.

Cada vez que se genera un enemigo también es añadido a la lista de enemigos de la clase RoomTemplates. Esa lista es la que usa la clase GameManager para mantener actualizado el contador de enemigos restantes ya que cuando el jugador mata a un insecto, se borra de esa lista y se pide al GameManager que actualice la interfaz.

Para comprobar si la cantidad de diseños era suficiente se realizó un cálculo para averiguar las probabilidades de que se generasen dos mapas iguales en el juego solo teniendo en cuenta las salas y no las posibles posiciones de los enemigos. Para ello se ha elegido que el mapa tenga 15 salas, la media del rango que hemos establecido. Salas con aberturas a la derecha o a la izquierda hay 16 y con aberturas arriba o abajo hay 10 en cada grupo. Esto nos deja con una media de 13 salas entre las que elegir siempre. Para generar el mismo mapa, las 15 veces que se generen salas deben ser la misma, es decir, 1/13. Es decir que podemos declarar la probabilidad como P(A) siendo A la representación que le damos a nuestro problema:

$$P(A) = \frac{1}{13} \cdot \frac{1}{13} \cdot \frac{1}{13} \dots \frac{1}{13} = \left(\frac{1}{13}\right)^{15} \approx \frac{1}{5'11 \cdot 10^{16}} \approx 1'95 \cdot 10^{-17} \approx 0$$

Por hacer una pequeña comparación, el hecho de ver dos mapas iguales actualmente en este juego, es aproximadamente un billón de veces más improbable que ganar la lotería en España según los datos aportados por la RTVE [53].

El proceso de generar el escenario comienza cuando se carga la escena KitchenLevel y mientras se realizan los procesos de generación, el usuario puede ver una pantalla de carga. Una vez se acaba la generación del mapa de forma satisfactoria, el GameManager se encarga de avisar a los enemigos para que comiencen a moverse y coloca en el punto de inicio del nivel al jugador.

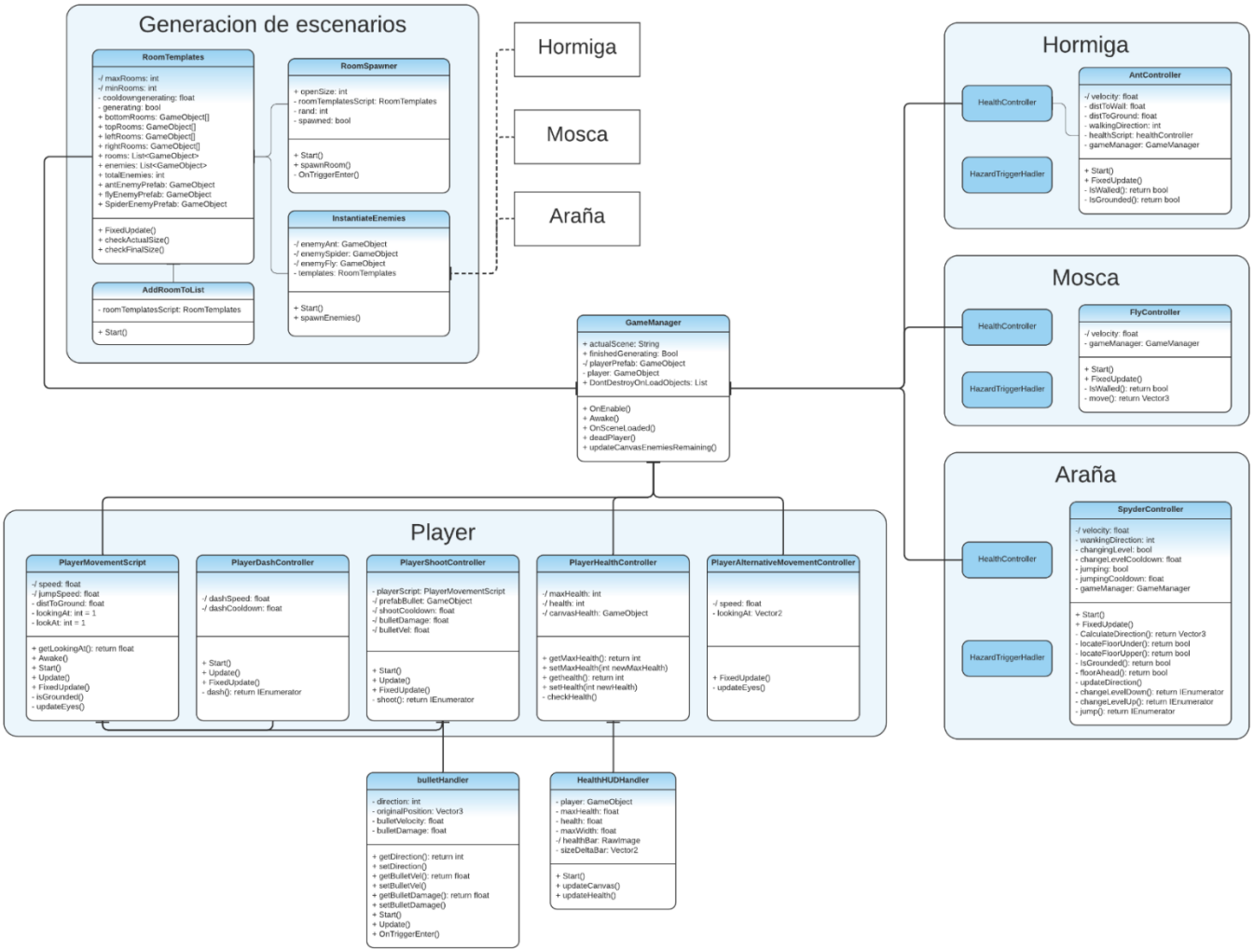


Figura 32: Diagrama de clases para el proyecto completo

5.3. Funcionamiento de Beans Out

En el siguiente apartado se realiza una explicación del funcionamiento del programa al completo siguiendo el flujo de una ejecución común.

El juego comienza mostrando un menú donde el usuario puede elegir entre comenzar a jugar o salir del juego. En este momento el programa ha generado un objeto con el componente GameManager acoplado que controlará todo el juego. Cuando el jugador haga clic en la opción de jugar, el GameManager instanciará el personaje del jugador y se cambiará la escena al Tarro de alubias. Esta escena es una zona donde el jugador no puede disparar ni usar el *dash,* ya que por la historia del juego, es un lugar seguro. En esta sala se encuentra el rey alubia que en esta versión no realiza ninguna acción, pero es el encargado de contar al jugador acerca de la historia del juego si se conversa con él.

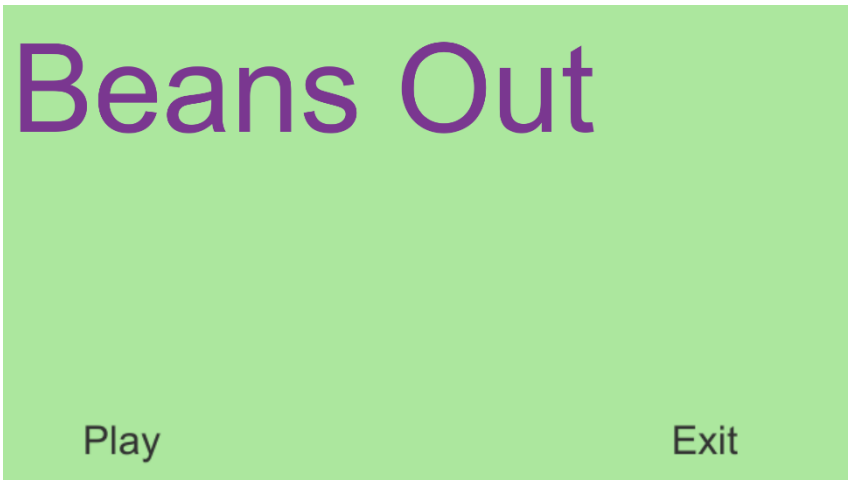


Figura 33: Menu principal de Beans Out

En esta sala el jugador aparece en el piso superior. Esta escena está diseñada para funcionar como un tutorial implícito en el juego ya que nada más comenzar hay un escalón que el jugador debe saltar para poder avanzar. Debido a esto el jugador aprenderá a moverse de forma sencilla y sin necesidad de tutoriales explícitos. Una vez avanza debe dejarse caer al piso inferior donde se encuentra el rey alubia esperando y la puerta para comenzar una partida. En el piso inferior se cambian los controles del jugador activando el script PlayerAlternativeMovementController y desactivando PlayerMovementScript ya que para alcanzar la puerta se necesita avanzar hacia el interior de la sala.

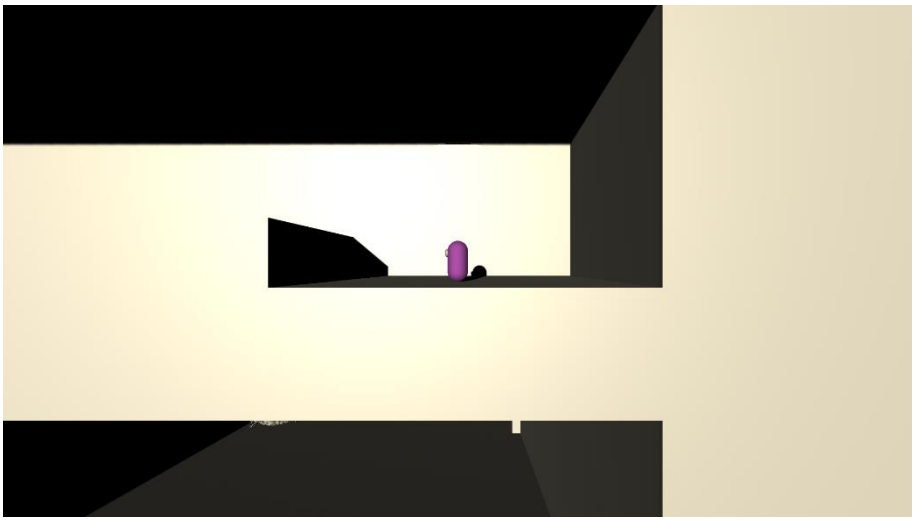


Figura 34: Escena del tarro de alubias justo al comenzar el juego

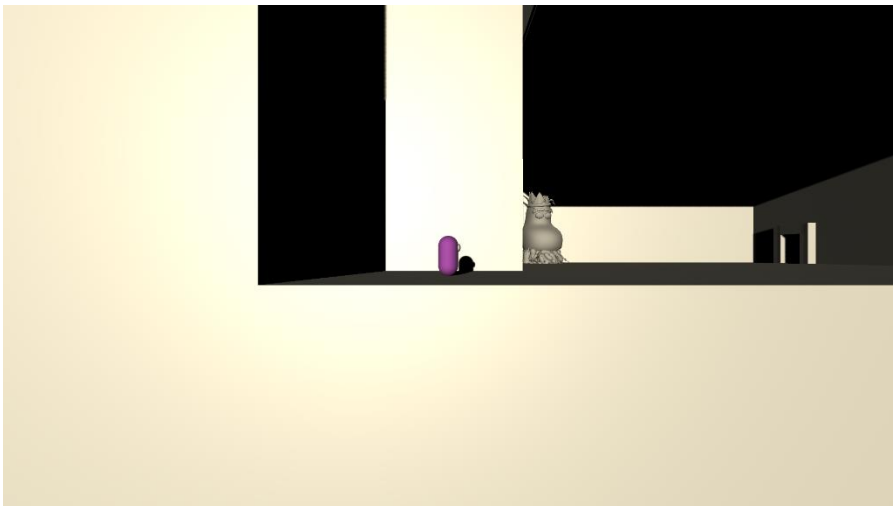


Figura 35: Escena del tarro de alubias después de descender al piso inferior

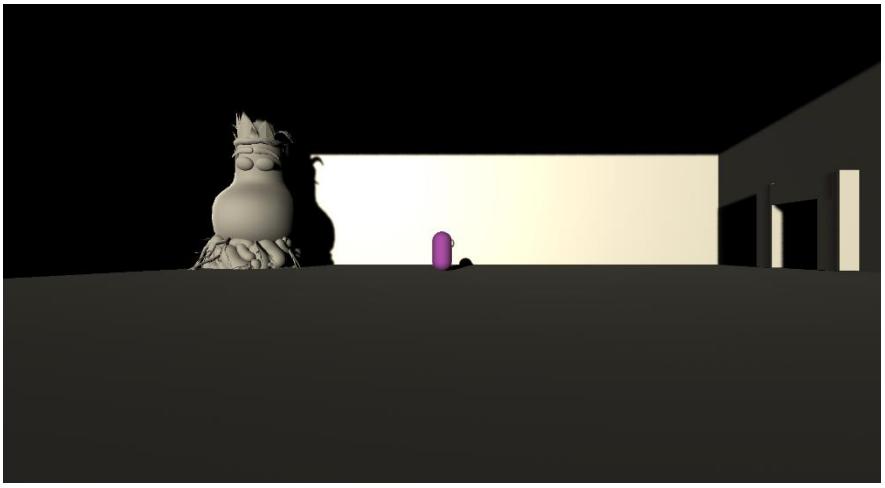


Figura 36: Escena del tarro de alubias. A la izquierda se puede ver el modelo del rey alubia y a la derecha la puerta para comenzar una partida

Una vez se atraviesa la puerta a la derecha de la sala, el juego comienza a cargar la escena donde se jugará. En este momento los algoritmos relacionados con la generación del mapa comienzan a crear el escenario mientras el jugador observa una pantalla de carga, formada por un vídeo del personaje principal y un mensaje avisando de que el nivel se está cargando. Cuando el nivel se ha generado, el GameManager avisa a los enemigos para que puedan comenzar a moverse y se quita la pantalla de carga. En ese momento el jugador es colocado en la sala de inicio y se activan sus habilidades de disparo y carrera además de devolverle al movimiento estándar y desactivar el movimiento alternativo.

En ese momento el jugador puede moverse por la pantalla de forma normal y los enemigos cercanos comenzarán a perseguirle. Al comenzar, la interfaz se actualiza y comienza a mostrar la vida del jugador en una barra en la esquina superior izquierda de color verde y el contador de enemigos en la esquina superior derecha. Cuando el jugador reciba daño la barra

de vida se encogerá representando la vida perdida y el contador mostrará los enemigos restantes en todo momento.

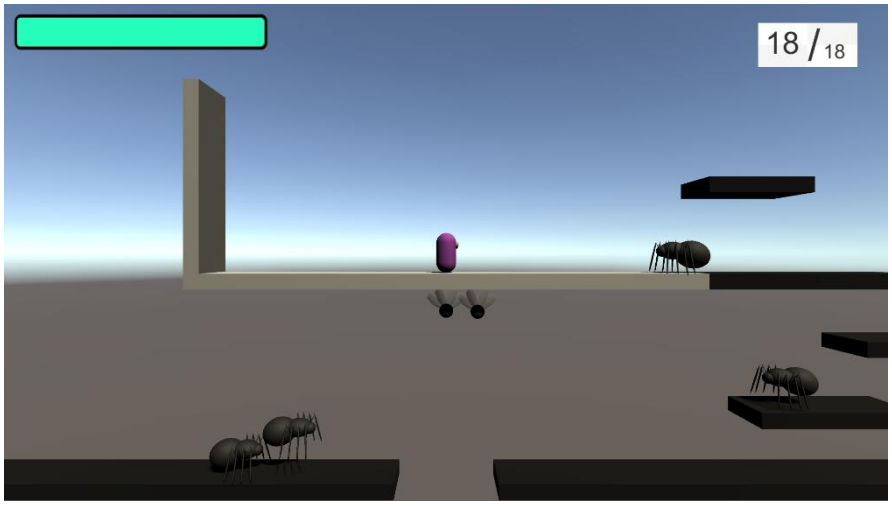


Figura 37: Captura de una ejecución de Beans Out



Figura 39: Pantalla de carga

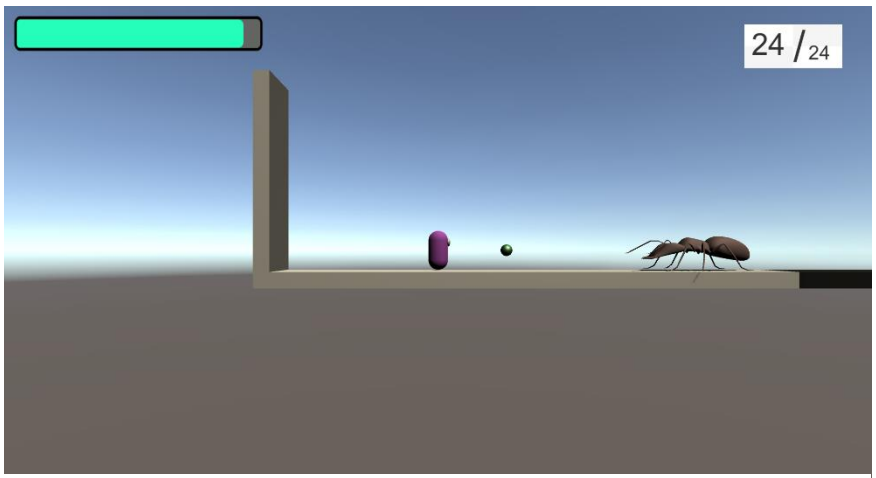


Figura 38: Captura de una ejecución de Beans Out, la barra de vida no está completa y se observa el disparo

En esta situación, el jugador puede o bien ganar o perder. Si consigue eliminar a todos los enemigos ganaría y volvería al menú del juego. En caso de morir antes de completarlo también seria devuelto al menú. No se han implementado mensajes de victoria ni derrota ya que en esta versión no se busca un sistema real de ganar el juego. Este sistema se implementaría en un futuro y requeriría que el jugador avanzara por varios mapas distintos hasta llegar al final del juego.

En este momento, el jugador puede también pausar el juego mediante el botón escape del teclado. En tal caso todo el juego se congelaría y el ratón del usuario volvería a estar disponible. En este menú se podría regresar al juego y entonces todo volvería a funcionar como antes o se puede salir al menú principal.



Figura 40: Menú de pausa

Para poder mostrar mejor una ejecución de esta aplicación se ha publicado un video en YouTube donde se muestra una ejecución en la que el jugador elimina a todos los enemigos del mapa. Para acceder a dicho video se debe usar el siguiente enlace: Video de demostración.

El video está subido en oculto lo que significa que nadie puede verlo a menos que posea el enlace del video. Esto se ha realizado así ya que este video no pretende ser ningún tipo de publicidad y solo es usado para ilustrar el proceso de jugar una partida en este documento.

5.4. Pruebas

Para comprobar el correcto funcionamiento del sistema se han realizado dos tipos de pruebas. Unas diseñadas, ejecutadas y probadas internamente y otras menos específicas realizadas por un grupo de personas voluntarias. Comenzaremos analizando las pruebas internas y después pasaremos a detallar el proceso para las pruebas externas.

5.4.1. Pruebas internas

Lo primero que hay que aclarar es que el proceso de pruebas no solo se llevó a cabo una vez terminada la implementación si no que ambos procesos fueron simultáneos. Una vez se diseñaba algún elemento, se probaba y analizaba su funcionamiento. Sin embargo, una vez completada la implementación del programa se realizaron un conjunto de pruebas finales para revisar el correcto funcionamiento de todos los elementos.

Para los principales elementos del juego se crearon unas tablas con aquellos aspectos a probar y se anotó el resultado de la prueba. Si la prueba no resultaba correcta se buscaba solucionar el problema y se volvía a ejecutar la prueba. De esta forma nos aseguramos de que todo aquello importante en el juego funciona.

Para representar las pruebas se diseñó la tabla mostrada en la Tabla 1: Tabla de ejemplo para los casos de prueba. El identificador es un código único, que identifica la prueba, formado por CP de Caso de Prueba y un número. La descripción es un texto breve que resume lo que se quiere probar. El elemento hace referencia a en qué parte del juego se ubica esta prueba, como por ejemplo personaje o enemigos. El resultado esperado y obtenido son una estimación de lo que debería pasar en la prueba y una confirmación de si se cumple.

Identificador	CP-000
Descripción	
Elemento	
Resultado esperado	
Resultado obtenido	

Tabla 1: Tabla de ejemplo para los casos de prueba

A continuación, se exponen las pruebas realizadas

Identificador	CP-001
Descripción	Comprobar el movimiento del jugador hacia la derecha
Elemento	Personaje principal
Resultado esperado	El personaje se desplaza sin problemas
Resultado obtenido	Correcto

Tabla 2: CP-001

Identificador	CP-002
Descripción	Comprobar el movimiento del jugador hacia la izquierda
Elemento	Personaje principal
Resultado esperado	El personaje se desplaza sin problemas
Resultado obtenido	Correcto

Tabla 3: CP-002

Identificador	CP-003
Descripción	Comprobar el salto del jugador
Elemento	Personaje principal
Resultado esperado	El personaje realiza el movimiento correctamente
Resultado obtenido	Correcto

Tabla 4: CP-003

Tabla 5: CP-004

Identificador	CP-004
Descripción	Comprobar el disparo del jugador
Elemento	Personaje principal
Resultado esperado	El personaje lanza el proyectil
Resultado obtenido	Correcto

Tabla 5: CP-004

Identificador	CP-005
Descripción	Comprobar el daño del disparo del jugador
Elemento	Personaje principal
Resultado esperado	La bala mata a un enemigo cuando impacta contra él
Resultado obtenido	Correcto

Tabla 6: CP-005

Identificador	CP-006
Descripción	Comprobar la carrera del jugador
Elemento	Personaje principal
Resultado esperado	El personaje realiza el movimiento correctamente
Resultado obtenido	Correcto

Tabla 7: CP-006

Identificador	CP-007
Descripción	Comprobar el movimiento alternativo del jugador
Elemento	Personaje principal
Resultado esperado	El personaje puede moverse correctamente en el eje Z
Resultado obtenido	Correcto

Tabla 8: CP-007

Identificador	CP-008
Descripción	Comprobar la vida del jugador
Elemento	Personaje principal
Resultado esperado	El personaje pierde vida cuando un enemigo le golpea
Resultado obtenido	Correcto

Tabla 9: CP-008

Identificador	CP-009
Descripción	Comprobar la barra de salud
Elemento	Interfaz
Resultado esperado	La barra de salud se actualiza cada vez que la vida del jugador varia
Resultado obtenido	Correcto

Tabla 10: CP-009

Identificador	CP-010
Descripción	Comprobar el movimiento de la hormiga
Elemento	Enemigo: Hormiga
Resultado esperado	La hormiga se desplaza correctamente
Resultado obtenido	Correcto

Tabla 11: CP-010

Identificador	CP-011
Descripción	Comprobar el cambio de sentido de la hormiga
Elemento	Enemigo: Hormiga
Resultado esperado	La hormiga realiza el cambio de sentido cuando encuentra un hueco o un muro
Resultado obtenido	Correcto

Tabla 12: CP-011

Identificador	CP-012
Descripción	Comprobar la persecución de la hormiga
Elemento	Enemigo: Hormiga
Resultado esperado	La hormiga se desplaza el doble de rápido cuando el jugador está en frente suya
Resultado obtenido	Correcto

Tabla 13: CP-012

Identificador	CP-013
Descripción	Comprobar el movimiento de la mosca
Elemento	Enemigo: Mosca
Resultado esperado	La mosca se desplaza correctamente
Resultado obtenido	Correcto

Tabla 14: CP-013

Identificador	CP-014
Descripción	Comprobar las colisiones de la mosca
Elemento	Enemigo: Mosca
Resultado esperado	La mosca se detiene si encuentra un muro en su camino
Resultado obtenido	Correcto

Tabla 15: CP-014

Identificador	CP-015
Descripción	Comprobar el movimiento de la araña
Elemento	Enemigo: Araña
Resultado esperado	La araña se desplaza correctamente
Resultado obtenido	Correcto

Tabla 16: CP-015

Identificador	CP-016
Descripción	Comprobar salto de la araña
Elemento	Enemigo: Araña
Resultado esperado	La araña salta correctamente de una plataforma a otra
Resultado obtenido	Correcto

Tabla 17: CP-016

Identificador	CP-017
Descripción	Comprobar el cambio de plataforma hacia arriba de la araña
Elemento	Enemigo: Araña
Resultado esperado	La araña sube correctamente de plataforma
Resultado obtenido	Correcto

Tabla 18: CP-017

Identificador	CP-018
Descripción	Comprobar el cambio de plataforma hacia abajo de la araña
Elemento	Enemigo: Araña
Resultado esperado	La araña baja correctamente de plataforma
Resultado obtenido	Correcto

Tabla 19: CP-018

Identificador	CP-019
Descripción	Comprobar el daño de los enemigos
Elemento	Enemigos
Resultado esperado	El jugador recibe daño cuando choca con los enemigos
Resultado obtenido	Correcto

Tabla 20: CP-019

Identificador	CP-020
Descripción	Comprobar la generación de escenarios
Elemento	Escenario
Resultado esperado	Los mapas se generan correctamente
Resultado obtenido	Correcto

Tabla 21: CP-020

Identificador	CP-021
Descripción	Comprobar la instanciación de enemigos
Elemento	Escenario
Resultado esperado	Los mapas instancian enemigos correctamente
Resultado obtenido	Correcto

Tabla 22: CP-021

Identificador	CP-022
Descripción	Comprobar el funcionamiento del menú principal
Elemento	Interfaz
Resultado esperado	El menú ejecuta las acciones correctamente
Resultado obtenido	Correcto

Tabla 23: CP-022

Identificador	CP-023
Descripción	Comprobar el funcionamiento del menú de pausa
Elemento	Interfaz
Resultado esperado	El menú ejecuta las acciones correctamente
Resultado obtenido	Correcto

Tabla 24: CP-023

Identificador	CP-024
Descripción	Comprobar el funcionamiento de la pantalla de carga
Elemento	Interfaz
Resultado esperado	La pantalla de carga se reproduce correctamente
Resultado obtenido	Correcto

Tabla 25: CP-024

Identificador	CP-025
Descripción	Comprobar el contador de enemigos
Elemento	Interfaz
Resultado esperado	El contador de enemigos muestra la información correcta y la actualiza
Resultado obtenido	Correcto

Tabla 26: CP-025

Identificador	CP-026
Descripción	Comprobar la victoria del jugador
Elemento	General
Resultado esperado	Si se eliminan a todos los enemigos se acaba el juego
Resultado obtenido	Correcto

Tabla 27: CP-026

Identificador	CP-027
Descripción	Comprobar la derrota del jugador
Elemento	General
Resultado esperado	Si la vida del jugador llega a 0 el juego termina
Resultado obtenido	Correcto

Tabla 28: CP-027

5.4.2. Pruebas externas

Para mejorar el proceso de probar el juego y encontrar todos aquellos fallos que existan se usó el método de *beta-testers*. Este es un método que usan muchas empresas como Ubisoft [54], cada una con sus condiciones, pero donde la idea principal es la misma: conseguir un grupo de voluntarios que probarán tu juego y te darán retroalimentación sobre los fallos que este pueda tener. Muchas veces este trabajo es remunerado, pero para nuestro caso se ha contactado con un grupo de voluntarios que están dispuestos a probar la aplicación gratuitamente. Estos fueron contactados por foros relacionados con videojuegos y por redes sociales. Si querían participar después de conocer lo que sería su tarea podrían rellenar un formulario donde se recogía su información personal para poder contactar más adelante con ellos y continuar el proceso. Este formulario está disponible en la sección de anexos del documento.

Después de unas semanas se consiguió un grupo de 21 personas interesadas en probar el juego. Cuando se consideró que el grupo era los suficientemente grande se cerraron las inscripciones y se envió un correo a todas aquellas personas que se apuntaron con un documento de confidencialidad que debían firmar para poder continuar con el proceso. Esto se realizó para mantener la seguridad de los ficheros que se compartirían con ellos. Una vez, los documentos fueron firmados se envió otro correo con toda la información necesaria para descargar el juego y ejecutarlo además de las instrucciones sobre el proceso de prueba. Este mail y el documento con las instrucciones también se encuentran en los anexos.

Para almacenar los resultados de estos usuarios se pusieron a su disposición dos formularios. Uno para documentar errores y otro para conocer la satisfacción del público. El primero es un formulario que se puede rellenar todas las veces que haga falta ya que cada respuesta corresponde con un error que se ha encontrado. Se realizan algunas preguntas sobre donde se ha dado el fallo y su relevancia además de pedir al usuario que documente lo más detallado posible el error que ha visto. El segundo formulario realiza preguntas más subjetivas sobre cómo ha percibido el usuario el producto. Primero se analizarán los resultados del primer

formulario analizando los errores encontrados y a continuación se detallarán las respuestas obtenidas para la encuesta de satisfacción.

5.4.2.1. Errores reportados

En el formulario se pedía describir el error en una pequeña frase y después detallarlo. De esta forma, a la hora de analizar los errores podemos fijarnos en la descripción corta para filtrar entre todos. En total se recibieron 30 reportes de errores.

Una de las preguntas iba enfocada a definir y englobar el error según el elemento del juego con el que se relacionase más. Existían cinco opciones; enemigos, escenario, interfaz, controles y personaje, pero el usuario podía añadir otra si lo creía oportuno. Los resultados muestran que la mayor parte de los errores provienen de los enemigos y el escenario. Estos resultados tienen sentido y se corresponden con lo esperado ya que los enemigos y el escenario son las partes más complejas del proyecto y al unir los movimientos de los enemigos con mapas totalmente aleatorios se producen algunas situaciones que son complicadas de prever o probar por su probabilidad de ocurrir. Vemos que la interfaz también muestra problemas, pero revisando estas respuestas, muchas de ellas documentan el mismo error, es decir que es un error común y a su vez, que las interfaces no tienen tantos errores como podría parecer.

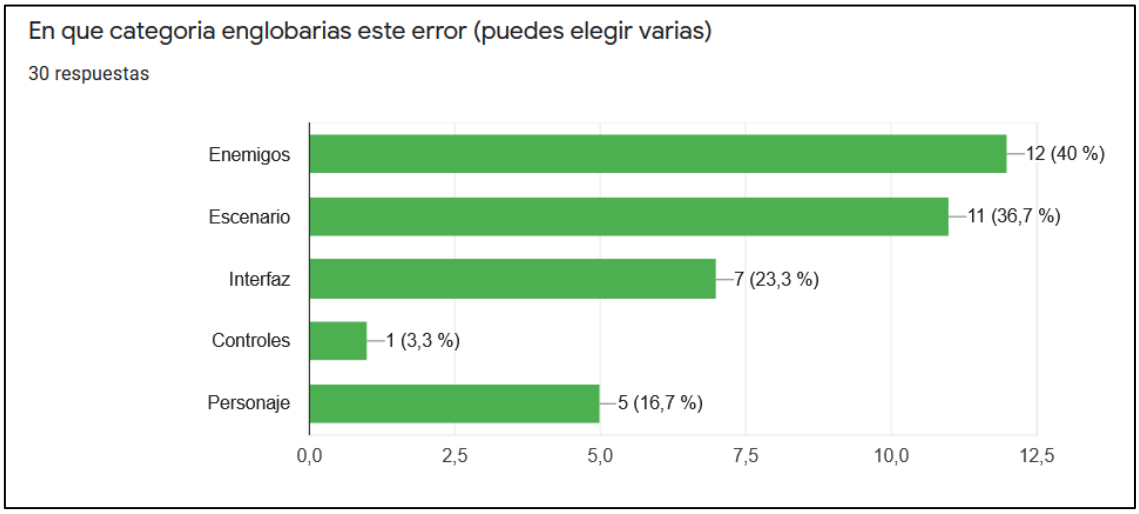


Figura 41: Resultados para la categoría de los errores

Continuando, pedíamos a los usuarios que respondieran a una simple pregunta. Se quería comprobar si el error que les había aparecido les había eliminado la posibilidad de continuar jugando, ya sea porque la ventana del juego se cierre o se cuelgue o porque algún comportamiento interno no funcionase correctamente.

En esta grafica se puede observar que el 36’7% de las respuestas eran errores que han impedido al jugador continuar jugando. Es un número más grande de lo esperado y por eso se analizaron estos casos. Se descubrió que esos 11 reportes eran debidos a 2 fallos principalmente.

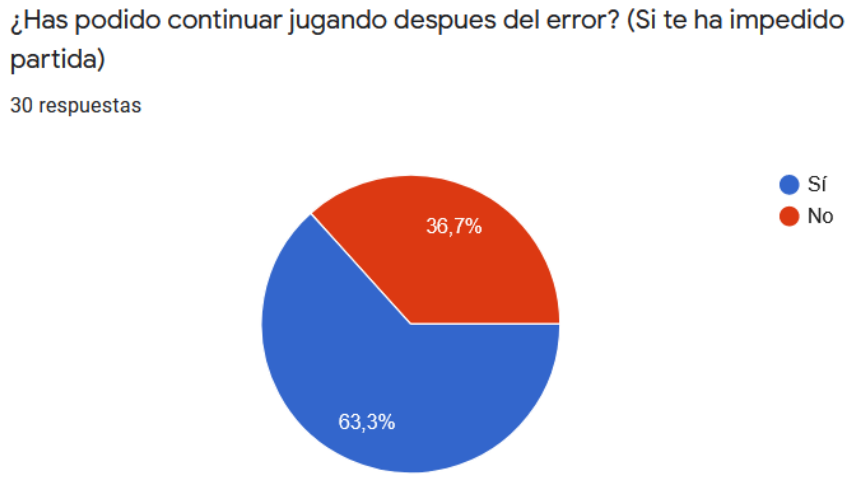


Figura 42: Resultados obtenidos para el comportamiento del juego después del error

Fallos también englobados con los enemigos y escenarios que se vio en la Figura 41: Resultados para la categoría de los errores. Esto nos da la información de que existen un par de errores muy importantes y que se dan con bastante frecuencia por lo que pasarían a ser la principal prioridad dentro los errores que se han encontrado.

La última cuestión preguntaba la gravedad del error sin ser tan concreto como la anterior. En esta se pedía al usuario que catalogase el error dentro de un rango de 5 valores, desde poco grave hasta muy grave. Podemos observar que los resultados se agrupan en los laterales y en el punto medio. Las 10 respuestas del lateral derecho corresponden con diez respuestas que impidieron continuar con el juego en la pregunta anterior. Esto se asemeja a lo esperado ya que, cuando un error no te permite continuar jugando es probablemente lo más grave que podrás encontrar. Estas respuestas nos sirven a modo de guía a la hora de determinar la importancia de los errores y el orden en el que deberían ser resueltos.

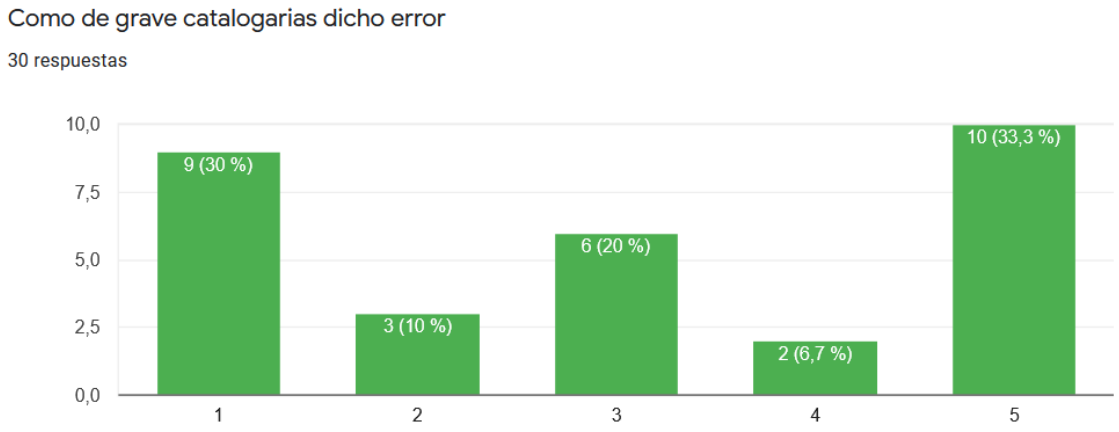


Figura 43: Respuestas obtenidas para la gravedad del error

Después de recibir todos los reportes de errores se realizó un proceso para detectar que errores habian sido reportados más veces y se creó una tabla ordenando estos. Ha habido ciertos casos en los que no se ha podido analizar el error ya que no estaba bien documentado y no se entendia la situación ni lo que pasó.

Error	# reportes	gravedad
En determinadas situaciones, al chocar contra enemigos te pueden desplazar en el eje Z y caerte del mapa al vacío	8	Muy alta
Existe la posibilidad de que las arañas se caigan de una plataforma y caigan al vacío. Esto hace que el jugador no pueda matarlas y nunca pueda ganar la partida.	5	Muy alta
La generación de mapas puede fallar y no generar algunas salas lo que deja un hueco en el mapa que permite que los jugadores caigan al vacío.	1	Alta
Hay zonas en el mapa donde las moscas se quedan quietas porque colisionan con algún elemento invisible del mapa	1	Alta
Si el jugador se coloca en el borde de una plataforma, el juego detecta que no está tocando el suelo y no le deja saltar	1	Alta
La pantalla de carga no se muestra si juegas más de una partida seguida por lo que el usuario puede ver el proceso de creación del escenario	4	Media
En determinadas ocasiones el personaje mira hacia el lado contrario al que debería.	3	Media
Si de alguna forma, el personaje cae al vacío, no hay forma de salir más que usando el menú de pausa para volver al menú principal	1	Media
Las hormigas pueden crearse en plataformas muy pequeñas que hacen que esta esté rotando todo el rato 180 grados	1	Baja
Para resoluciones distintas de 16:9 el juego puede no ser compatible ya que la pantalla se escala y puede dejar información fuera.	1	Baja
Se ha encontrado una zona del mapa donde el jugador puede cochar entre dos paredes y genera unas colisiones que le suben poco a poco hasta arriba de la pared	1	Muy baja

Tabla 29: Recopilación de errores encontrados en las pruebas externas

5.4.2.2. Resultados obtenidos

Para poder concluir correctamente los resultados del proyecto se analizarán las respuestas obtenidas para las encuestas de satisfacción. Se han obtenido diecinueve respuestas, lo que no nos da un espectro muy grande, esto provoca que los resultados que obtenemos aquí no pueden ser generalizables y pueden estar sesgados. Para futuras pruebas externas como estas, se deberían buscar grupos más amplios.

La encuesta estaba dividida en 5 secciones, una primera que omitiremos donde se pedía el nombre y apellidos del encuestado para mantener un registro. La segunda estaba formada por tres preguntas sobre el control y manejo del jugador y un apartado de comentarios sobre estos. Como podemos observar en los resultados, los controles han resultado ser intuitivos a la gran mayoría de los encuestados. En comodidad y sencillez, los resultados se expanden más a lo largo de las posibilidades, pero siguen siendo positivos con la moda en 4 de 5, siendo 5 muy de acuerdo y 1 muy en desacuerdo. En la tercera pregunta vemos unos resultados muy positivos que nos demuestran que los controles funcionan correctamente en la mayoría de situaciones.

Con respecto a los comentarios personales, la mayoría hacían referencia al hecho de que tener todos los botones juntos hacía difícil realizar algunas acciones, esto se asemeja en lo que obtenemos en los datos, ya que con respecto a la comodidad es donde más disparidad de opiniones ha habido.

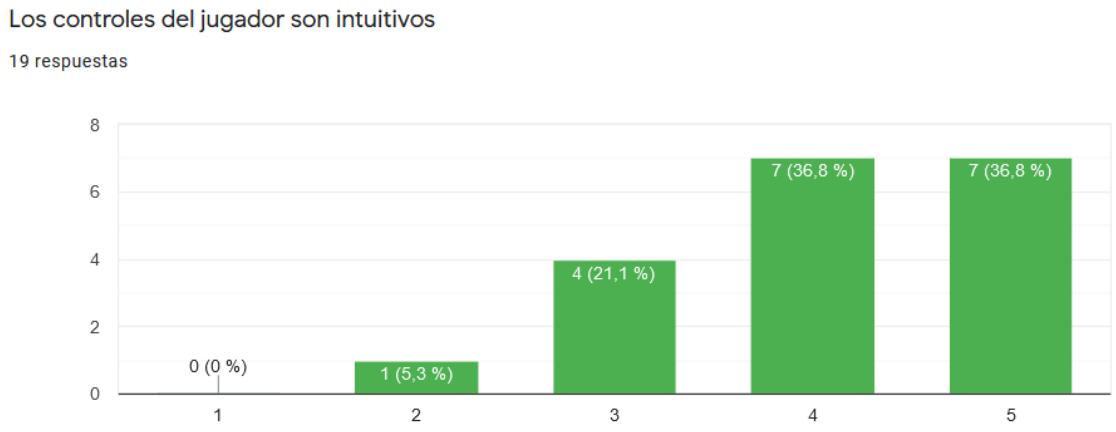


Figura 44: Resultados obtenidos para la intuición de los controles

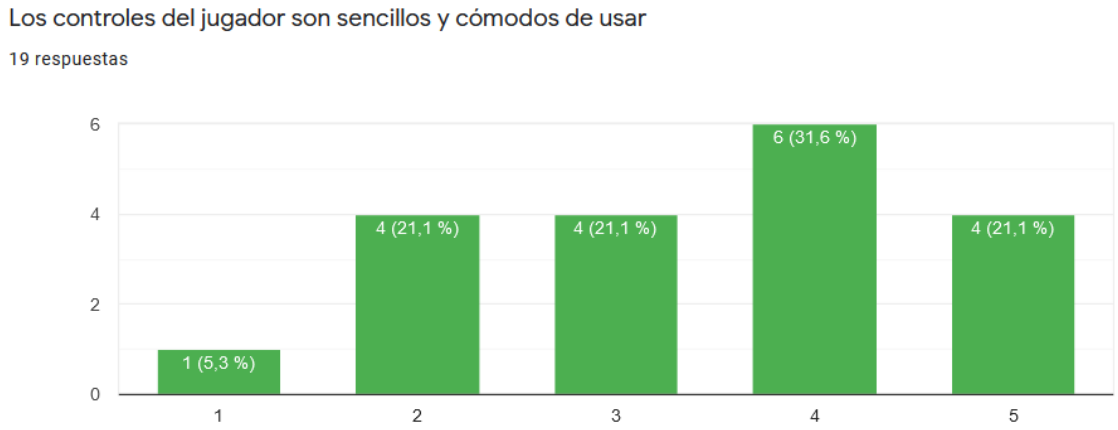


Figura 45: Resultados obtenidos para la comodidad de los controles

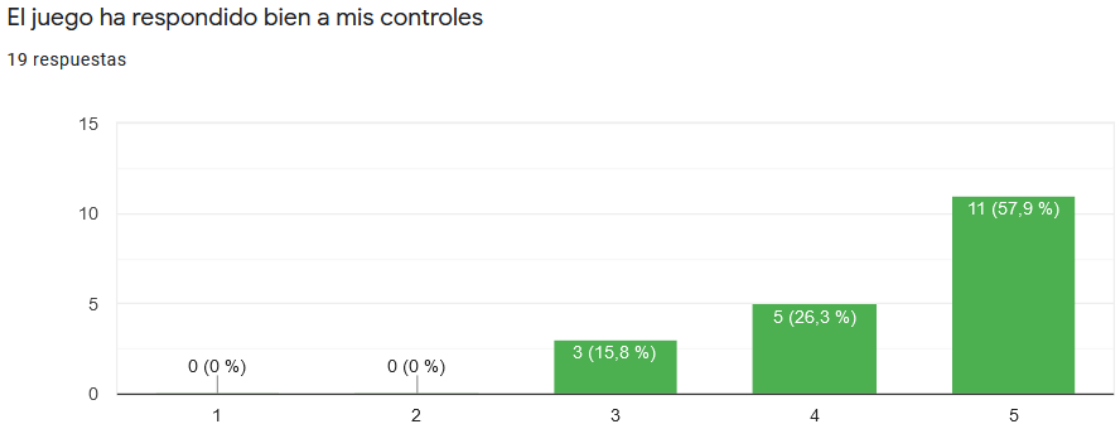


Figura 46: Resultados obtenidos para la respuesta de los controles

La siguiente sección del formulario se centraba en los enemigos y la dificultad del juego. Comenzaba con una primera pregunta sobre el comportamiento de los enemigos de forma general. Los resultados son muy similares a los esperados ya que el comportamiento no es perfecto en muchos casos y gracias al proceso de pruebas se descubrieron incluso más fallos sobre estos, pero sí que habían sido probados en muchas situaciones y se había logrado un funcionamiento pulido.



Figura 47: Resultados obtenidos para el comportamiento general de los enemigos

Las siguientes tres preguntas van relacionadas más concretamente con cada uno de los enemigos. El enemigo que más problemas parece tener es la hormiga, pero esto es debido al problema encontrado en la sección de pruebas externas por el cual la hormiga es capaz de tirarte del escenario. La mosca no parece tener muchos errores ya que es el enemigo con el comportamiento más simple y la araña tiene unas respuestas variadas, ya que había algunos encuestados que pensaban que la habilidad de descender o ascender plataformas era un error. Esto muestra que se debería pensar en otro método para realizar esta acción o que la animación sea distinta para no dar esa visión al espectador.



Figura 48: Resultados obtenidos para el comportamiento de las hormigas



Figura 49: Resultados obtenidos para el comportamiento de las moscas

Por último, se añade una pregunta sobre la dificultad del juego que aporta resultados buenos. Muchos comentarios después detallan que la dificultad parece arbitraria, ya que en algunas situaciones aparecen muchos enemigos y en otras pocas. Este elemento es uno a tener en cuenta en las líneas futuras de trabajo.



Figura 50: Resultados obtenidos para el comportamiento de las arañas

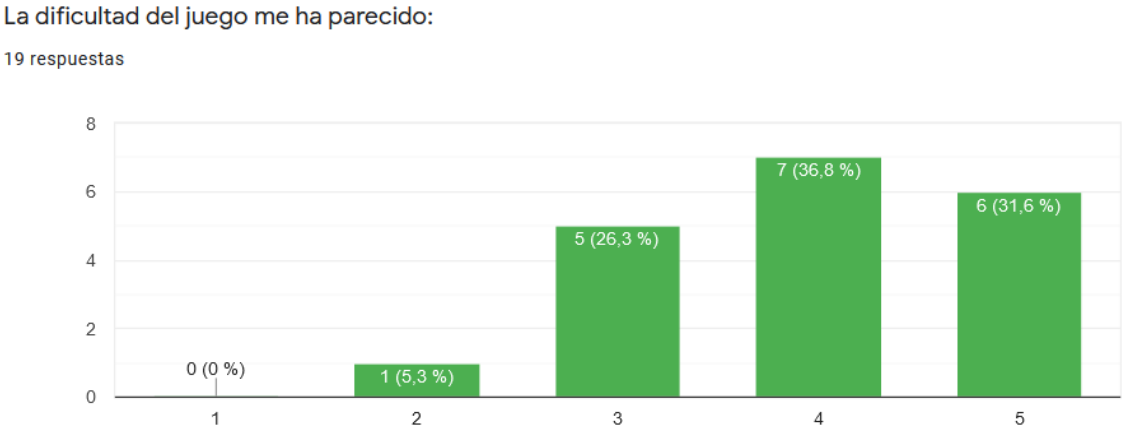


Figura 51: Resultados obtenidos para la dificultad del juego

La cuarta sección estaba relacionada directamente con el escenario. En esta se hacían dos preguntas; una sobre si el juego se siente distinto en cada partida y otra sobre la movilidad por el mapa. La primera arroja resultados variados. La moda se encuentra en un punto positivo pero los resultados están muy repartidos como para poder obtener muchos datos. Observando los comentarios de los encuestados vemos que muchos decían que el juego no se sentía muy distinto debido a que no hay mucha variedad de enemigos y eso hace que se repitan situaciones a pesar de que el mapa sea distinto cada vez.

Los resultados de la segunda pregunta son claramente positivos aportando como resultado la certeza de que el desplazamiento por el mapa es agradable. Esto nos indica que las salas que se han diseñado son correctas.

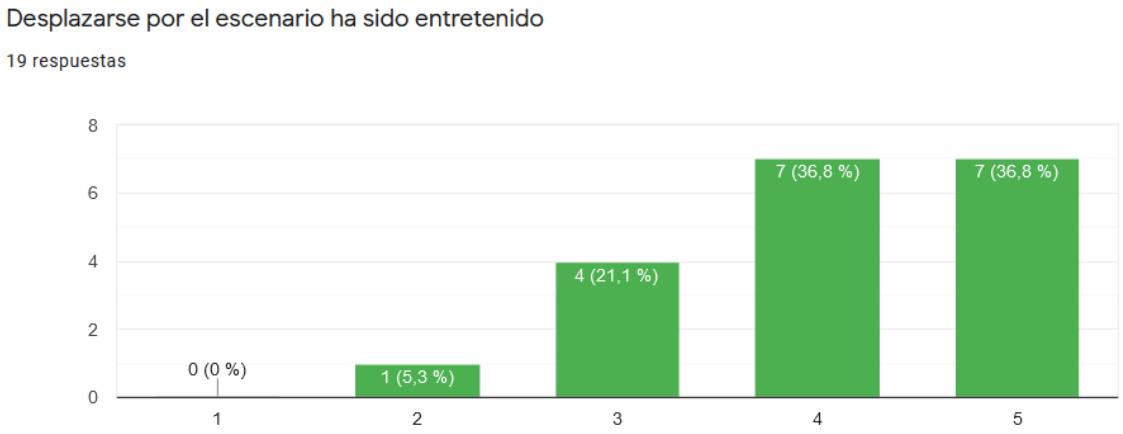


Figura 52: Resultados obtenidos con respecto a la movilidad en el escenario

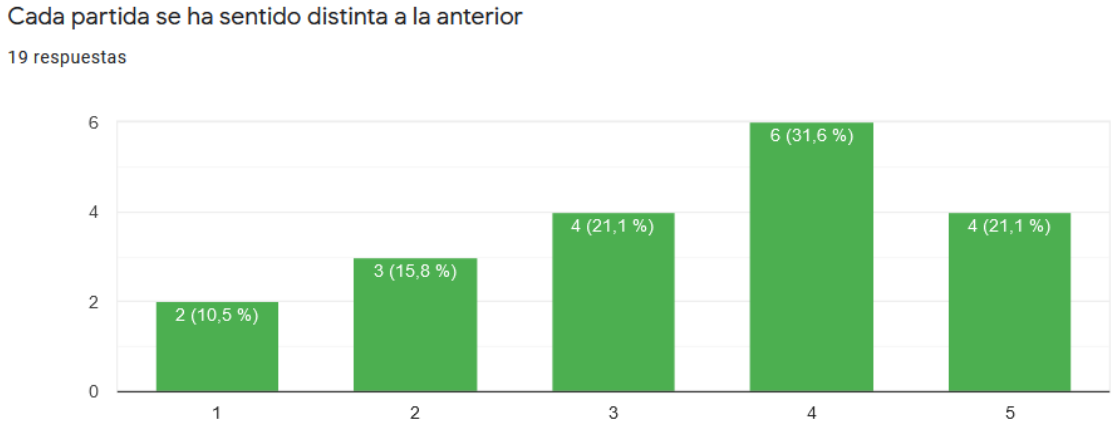


Figura 53: Resultados obtenidos para la diferencia entre partidas

La última sección del formulario contenía una pregunta general para los comentarios del usuario sobre el juego y una segunda para que el usuario pudiera expresar cualquier cosa que no se hubiera preguntado antes

6. Planificación y presupuesto

6.1. Planificación

Para planificar el proceso de creación de este proyecto se llevó a cabo un diagrama Gantt dividido en cinco fases, cada una centrada en una sección importante del trabajo. En la Figura 54: Diagrama Gantt estimado del proyecto se muestra el Gantt esperado para todo el proyecto

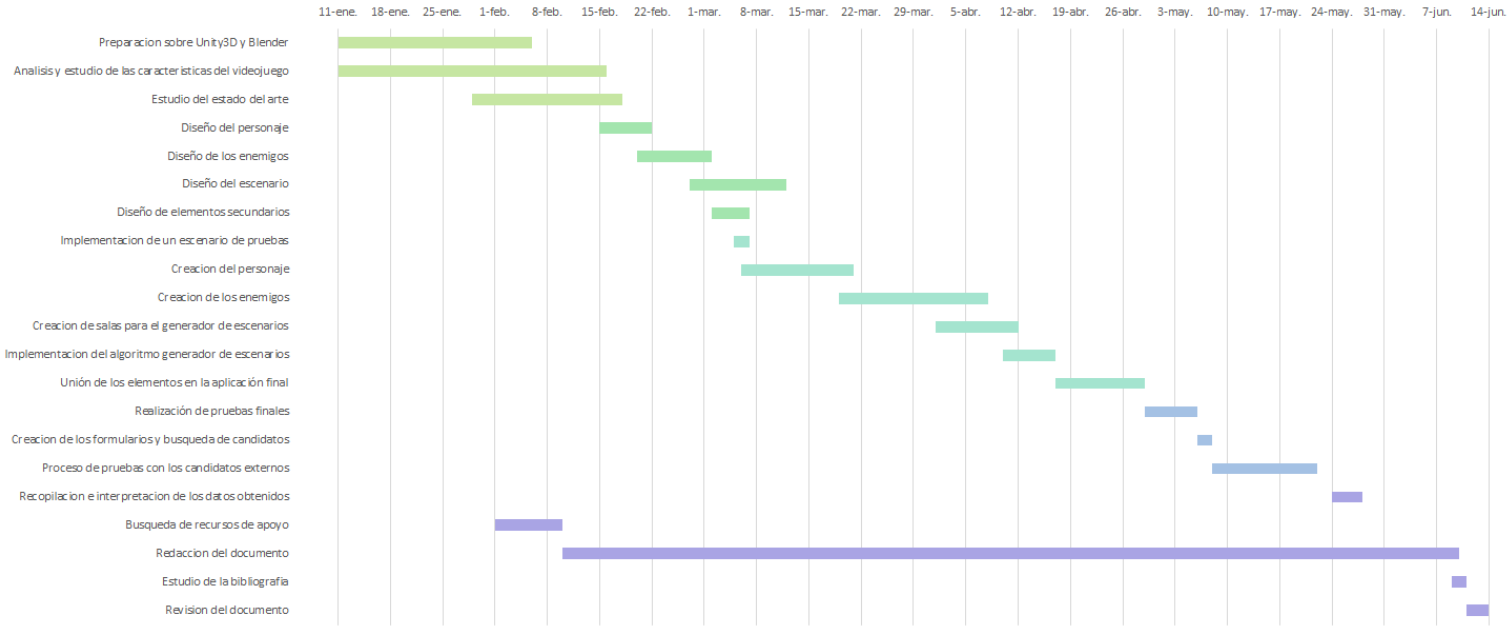


Figura 54: Diagrama Gantt estimado del proyecto

y a continuación se detallará por fases.

La primera fase corresponde con el planteamiento inicial del problema. En esta parte del proyecto se decidieron las ideas básicas de este. Se buscaba obtener una idea general para el juego, aprender lo básico de Unity y Blender mediante los cursos adquiridos en Udemy y comenzar estudiando el estado de la cuestión. Esta sección correspondería con un primer mes y medio de trabajo, donde se asentarían las bases del proyecto.

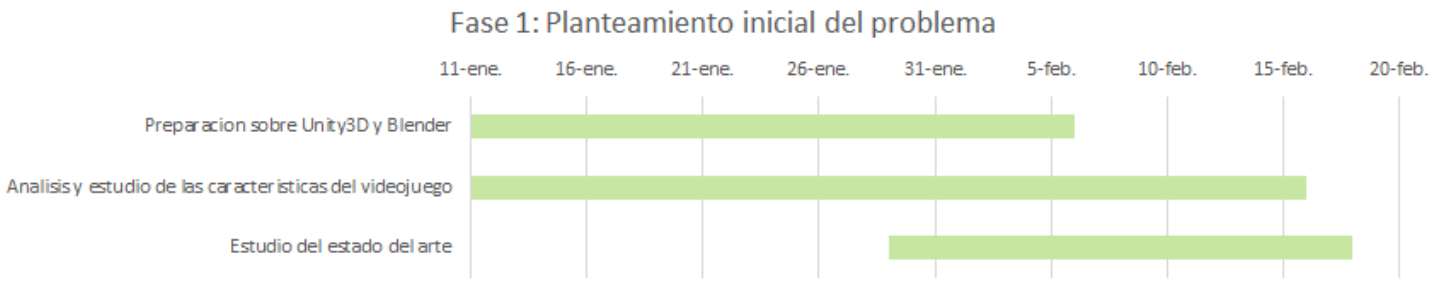


Figura 55: Diagrama Gantt estimado para la fase 1

La fase dos corresponde con el diseño de la aplicación. Se busca obtener los diseños de aquellos elementos más importantes del juego. Esta tarea se realizará en un periodo de tiempo de un mes aproximadamente y estará centrada en diseñar el personaje principal, los enemigos y el escenario.

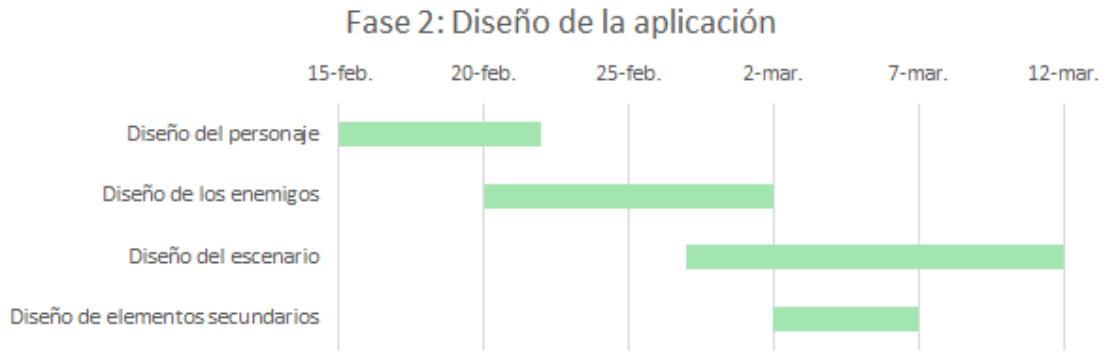


Figura 56: Diagrama Gantt estimado para la fase 2

Para la fase tres se espera realizar la implementación del programa. Esta parte se realizará de forma secuencial pasando al siguiente elemento cuando se haya terminado el anterior. En la siguiente parte veremos las pruebas realizadas, pero cabe destacar que mientras se implementen los elementos del juego se irán probando. Este proceso debería durar menos de 2 meses.

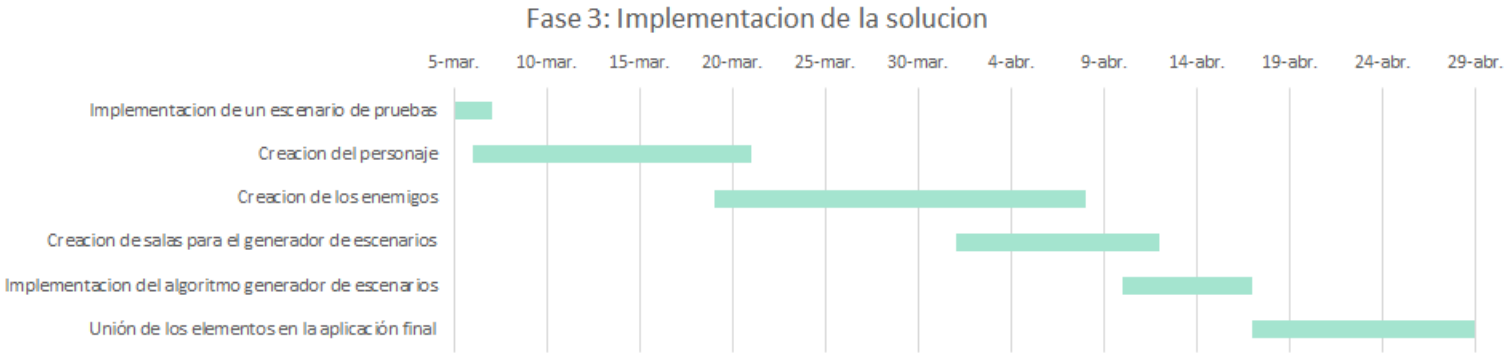


Figura 57: Diagrama Gantt estimado para la fase 3

Como se ha mencionado, en la cuarta fase se realizarán las pruebas finales del sistema. Se harán dos tipos de pruebas, unas propias y otras por un grupo de personas con las que se contactó para realizar unas encuestas y documentar fallos. Estas pruebas se realizarán en un periodo de alrededor de un mes.

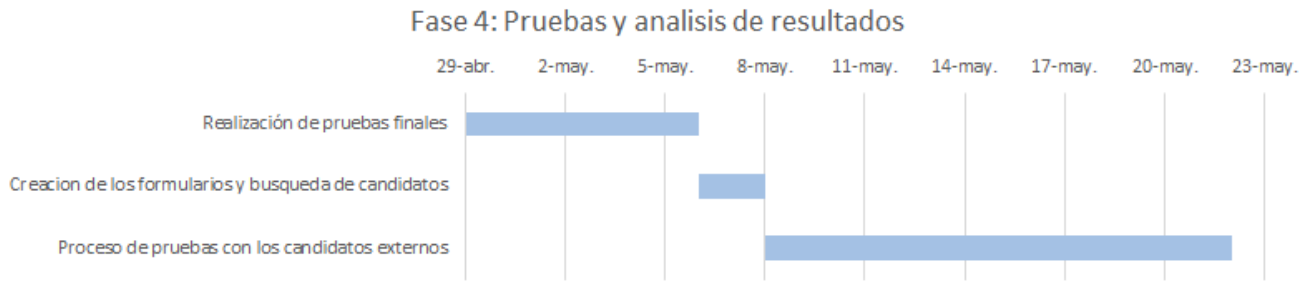


Figura 58: Diagrama Gantt estimado para la fase 4

En la última fase se engloba todo el proceso de redacción del documento. Este se realizará de forma simultánea al resto de las fases para repartir trabajo y documentar aquello que se realiza en el momento. Se realizará una revisión final al documento terminado. Se planea terminar el proyecto el 14 de junio.

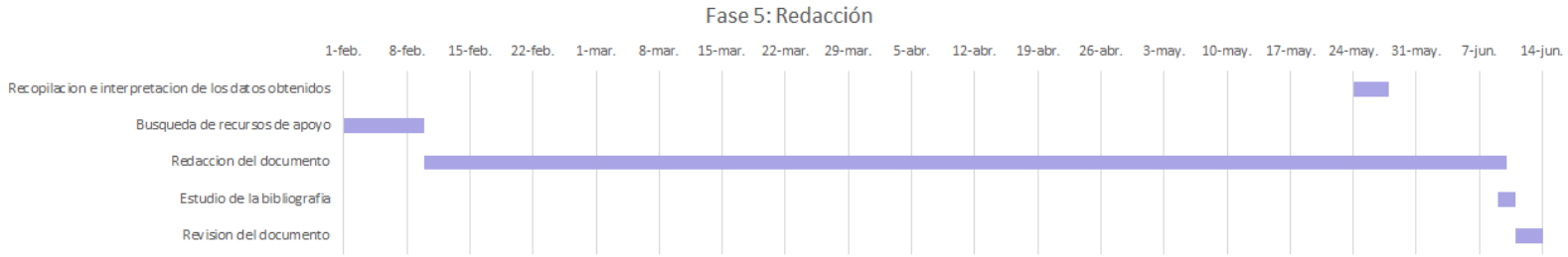


Figura 59: Diagrama Gantt estimado para la fase 5

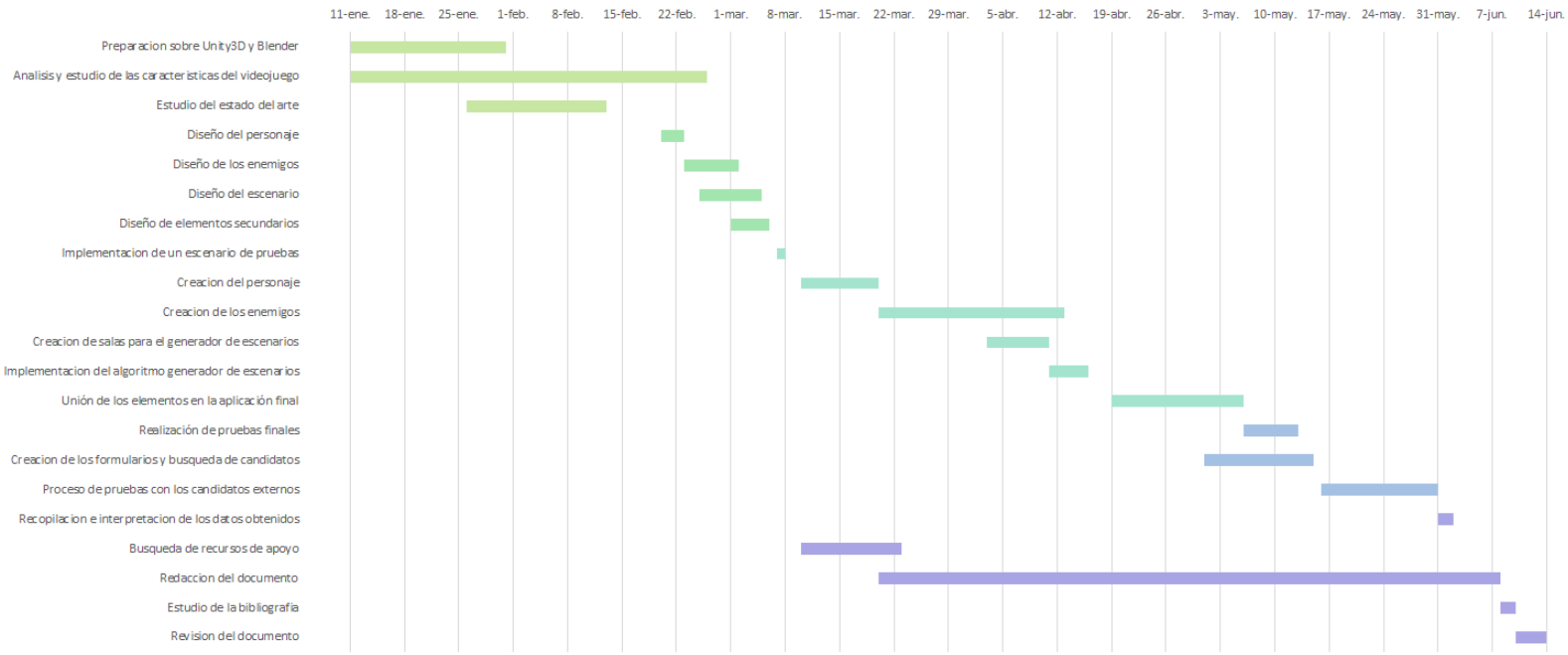


Figura 60: Diagrama Gantt real del proyecto

En la Figura 60: Diagrama Gantt real del proyecto se muestra el Gantt real para el proyecto. A continuación, se expondrán las fases y se comentarán los cambios más relevantes con respecto al esperado y a que se han debido.

En la primera fase hubo un cálculo erróneo de tiempo asignado a decidir los elementos relevantes del juego. Esto se debió al cambio de genero del que se ha hablado en varias ocasiones durante el documento. Este cambio obligó a repensar varias de las decisiones del juego lo que extendió este periodo por más tiempo del esperado.

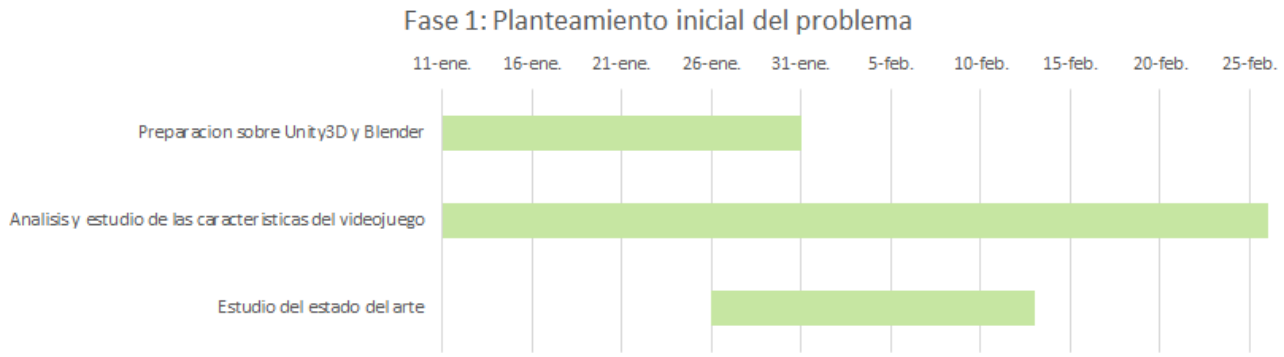


Figura 61: Diagrama Gantt real para la fase 1

Debido a este cambio, la fase dos se retrasó unos 5 días en comenzar con respecto de lo esperado. Sin embargo, debido a tener que repensar elementos del juego, se acabó diseñando varios de estos elementos antes de tiempo por lo que el periodo de diseño fue algo más corto de lo esperado.

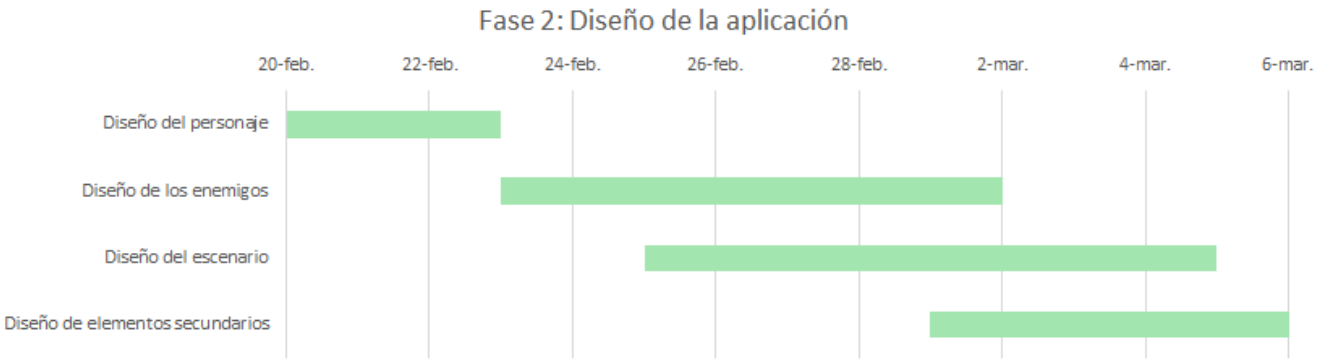


Figura 62: Diagrama Gantt real para la fase 2

La implementación de la aplicación comenzó como se había previsto, pero rápidamente los tiempos fueron descuadrándose. Esto pasó debido a que los enemigos necesitaban ser probados en los mapas finales por lo que ambas tareas se acabaron realizando de forma simultánea. El tiempo final de esta fase acabó ligeramente retrasado con respecto de lo estimado.

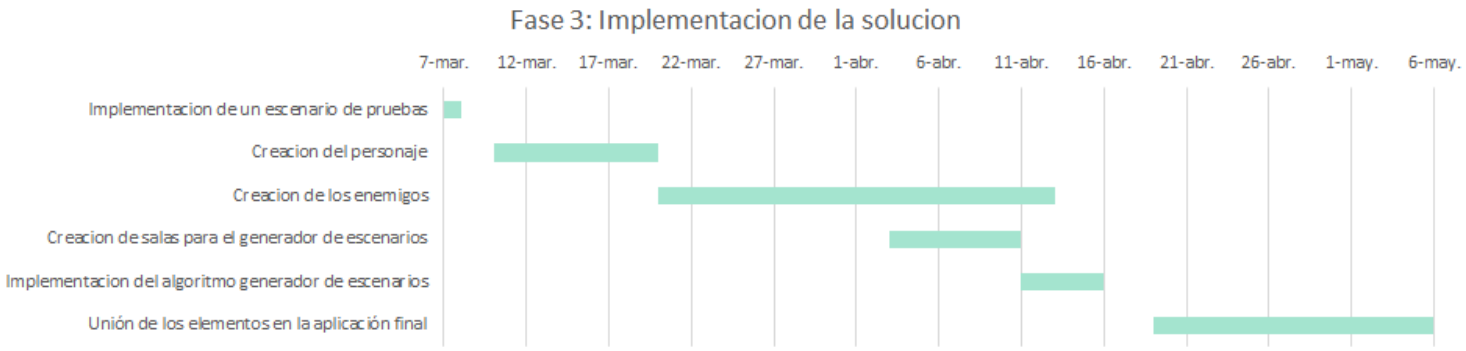


Figura 63: Diagrama Gantt real para la fase 3

Para la fase cuatro el mayor cambio fue el tiempo aplicado a buscar aquellos candidatos para que probasen el juego. Debido al retraso que se estaba sufriendo en la fase anterior, se decidió comenzar esta parte antes de tiempo y resultó ser la decisión correcta ya que encontrar el grupo de personas fue una tarea más compleja de lo esperado y requirió mucho más tiempo. Gracias a esto se consiguió no retrasar mucho la fase al completo.

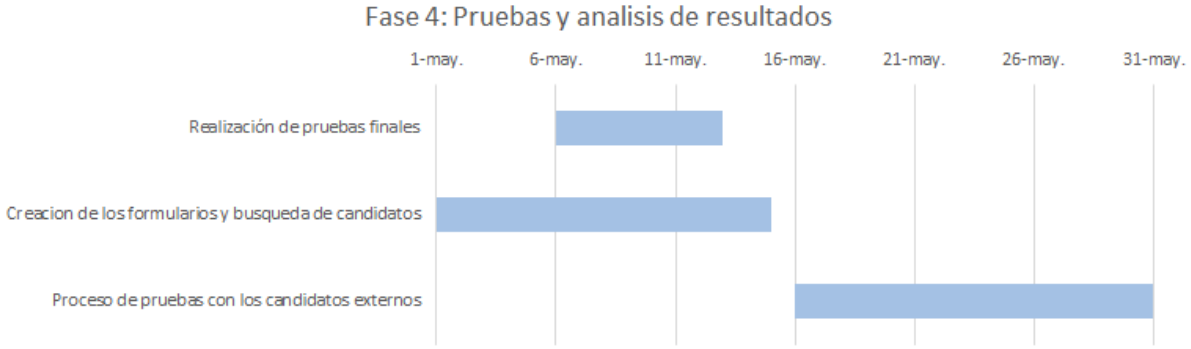


Figura 64: Diagrama Gantt real para la fase 4

Por último, la fase 5 se realizó de forma similar a la esperada, pero duró en conjunto menos tiempo ya que se comenzó la redacción del documento un mes después de lo esperado.

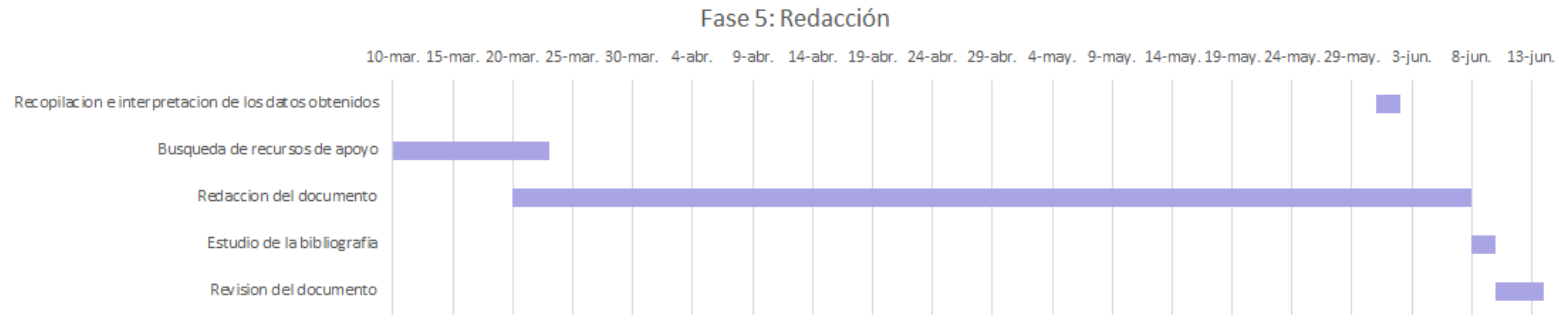


Figura 65: Diagrama Gantt real para la fase 5

Es difícil estimar las horas exactas de trabajo para el total del proyecto ya que se mantuvo un seguimiento sobre los días de inicio y fin para actividad y no se trabajó el mismo número de horas diarias a lo largo de todo el proyecto. Por eso se mantuvo también un conteo de horas para cada semana y fase. De esa forma podemos obtener el tiempo dedicado a cada fase y el tiempo total dedicado a este trabajo.

Para la fase 1 se dedicaron un total de 64 horas, en la segunda 42, para la tercera 89, la cuarta tomó 50 y la última fase constó de 148 horas. En total se estiman 393 horas dedicadas a este proyecto. Estos datos y los relacionados con los diagramas Gantt antes mencionados pueden consultarse en la sección de anexos al final de este documento.

6.2. Presupuesto

Para llevar a cabo este proyecto se han calculado los siguientes costes, desglosados por categorías.

6.2.1. Costes de personal

En esta sección se engloban los costes derivados del personal relacionado con el proyecto. En este proyecto solo han participado dos personas, el alumno y la tutora. Se han estimado las horas de trabajo de la tutora en función de las reuniones y de su tarea como revisora. Para el sueldo por hora para el jefe de desarrollo se han usado los datos aportados por la página web Indeed para el empleo de diseñador ux [55], ya que es el que más se ajusta a la categoría de "game designer". En caso de la directora de proyecto se ha aumentado el sueldo con respecto al anterior debido a la responsabilidad y peso que conlleva ese puesto.

Persona	Tiempo Empleado	Remuneración	Coste
Directora de proyecto	25h	16€/h	400€
Jefe de desarrollo	393h	13€/h	5.109€
		Total	**5.509€**

Tabla 30: Costes de personal del proyecto

6.2.2. Costes materiales

En esta sección se toman en cuenta los costes derivados de elementos físicos que se han usado durante la realización de este trabajo. Para algunos elementos se ha hecho una estimación de la vida útil y se ha calculado el coste gastado para este proyecto.

Concepto	Cantidad	Precio unitario	Uso	Coste
Ordenador de sobremesa Dell	1	1300€	13 %	169€
Periféricos: teclado	1	20€	-	20€
Periféricos: ratón	1	20€	-	20€
Periféricos: monitor	2	120€	-	240€
Silla de oficina	1	50€	-	50€
Material de oficina	1	30€	-	30€
			Total	**529€**

Tabla 31: Costes materiales del proyecto

6.2.3. Costes indirectos y otros costes

En la siguiente sección se enmarcan los costes indirectos y aquellos que no se han podido clasificar en las anteriores categorías. Para estos costes se ha hecho una estimación del uso dado durante el proyecto a estos componentes.

Concepto	Uso	Precio	Tiempo	Coste
Conexión a internet	40%	39’99€/mes	6 meses	95’97€
Curso online sobre Unity3D (Udemy)	100%	94’99€	-	94’99€
Curso online sobre Blender (Udemy)	100%	94’99€	-	94’99€
Licencia de Unity3D	100%	0€	6 meses	-
Licencia de Blender	100%	0€	6 meses	-
Licencia adobe completa estudiantes	100%	19’66€/mes	6 meses	117’96€
Licencia Microsoft 365 Personal	100%	7€/mes	6 meses	42€
			Total	**445’91€**

Tabla 32: Costes indirectos y otros costes del proyecto

6.2.4. Coste final

Para finalizar, obtenemos los siguientes costes finales para el proyecto aplicando el coste de la seguridad social sobre el coste humano y los impuestos sobre el total.

Concepto	Coste
Costes humanos	5.509€
Costes materiales	529€
Otros costes	445’91€
Total antes de seguridad social	**6.483’91€**
Seguridad social (30%)	1.652’7€
Total antes de impuestos	**8.136’61€**
Impuestos (21%)	1.708’69€
Total	**9.845’30€**

Tabla 33: Coste total del proyecto

El coste final del proyecto es de 9.845’30€ lo que dividido por los seis meses de duración nos deja con un coste total de 1.640’89€ al mes.

6.3. Impacto socio-económico

La versión actual del juego no se planea publicar de ninguna forma y se continuará trabando en sus mejoras hasta obtener un producto final que publicar de forma gratuita. Por esto, el proyecto no generará ningún impacto económico. Una vez el juego se publique, cualquier persona podrá acceder a él por lo que el impacto social de este proyecto se engloba en la industria del entretenimiento, habiendo creado un producto totalmente nuevo en este sector.

Esta memoria será publica para aquellas personas relacionadas con la Universidad Carlos III y se podrá usar como referencia o inspiración para futuros trabajos bajo la licencia *Creative Commons*.

6.4. Marco regulador

El juego plantea llevarse a cabo para la plataforma PC, tanto Windows, MacOS y Linux. El juego será *free to play* por lo que no entrará en cuenta ninguna legislación sobre los ingresos finales. Por ejemplo, Unity 3D da la posibilidad de usar su motor de videojuegos de forma gratuita, pero según los ingresos que tenga tu empresa, deberás abonar un pago mensual o anual [56].

Con respecto a la Ley orgánica de protección de datos personales [57], no afecta al producto final generado ya que este no recogerá nada de información del usuario.

En el proceso de creación de este proyecto se usan varias encuestas a aquellas personas que prueben el juego para conocer como recibirá el público este producto. Para estas encuestas, me he ajustado a la ley anteriormente mencionada y los datos obtenidos solo serán usados con fines académicos además de hacer saber a la persona que rellena dicha encuesta de esto. Estos datos serán almacenados como máximo un año y el usuario que rellenó dicha encuesta puede ponerse en contacto directo conmigo en cualquier momento para pedir que sus datos sean eliminados.

Por mantener la confidencialidad a la hora de compartir el juego con terceros, se ha realizado un contrato de confidencialidad con cada una de las personas que después recibieron el juego. Este documento puede verse en la sección de Anexos de este documento. El contrato se mantiene en vigor mientras exista la información privilegiada que obtiene dicha persona y obliga a esta a no poder compartir ni publicar a terceros nada de la información que ha recibido. Este contrato se ha realizado siguiendo la plantilla publicada por ModeloContrato en su web [58] realizando las variaciones necesarias en las cláusulas para ajustarlo a este trabajo.

7. Conclusiones

Durante la extensión de este proyecto se ha analizado, diseñado, implementado y probado una demo de un juego con todos aquellos elementos importantes que la componen: un personaje controlado por el usuario con todas sus habilidades, un grupo de enemigos con comportamientos distintos y únicos y un sistema capaz de generar escenarios totalmente aleatorios. Además de estudiar el estado del arte sobre el ámbito en el que se engloba este proyecto.

Durante el desarrollo del proyecto surgieron ciertos problemas. Algunos muy básicos y simples como errores al programar debido a que el entorno de Unity era algo nuevo, pero también problemas más grandes. Hubo dos grandes problemas durante la realización de esta aplicación, el primero vino muy pronto en el proyecto y fue derivado de no haber analizado lo mejor posible la idea. Debido a que el concepto general del juego ya había sido pensado desde hace mucho tiempo, incluso antes de empezar este proyecto, se subestimó ligeramente la tarea de analizar el problema. Al comenzar a pensar en todas las posibilidades se entendió que no se había escogido la mejor y se tuvo que replantear ciertas ideas. Sin embargo, no fue un error muy grave ya que no retrasó mucho el resto del proyecto.

El segundo error fue más general y fue el tiempo. Nunca se estuvo muy fuera de los márgenes esperados, pero era muy complicado seguir las fechas señaladas. Donde más se pudo ver este error fue en la fase de pruebas ya que se estimó que duraría poco tiempo mientras que en la realidad se extendió mucho más de lo esperado. Al final el proyecto se ha podido llevar a cabo sin retrasos, pero sí con cambios en la planificación global.

7.1. Resultados obtenidos

Para poder valorar los objetivos que se han cumplido, es necesario comprobar los resultados que se obtuvieron en la encuesta de satisfacción en la fase de pruebas. En estas pruebas se recibieron diecinueve respuestas, lo que hace que los resultados obtenidos no sean generalizables y podrían estar sesgados.

Sobre los controles del jugador se han obtenido resultados que muestran como estos son considerados por los jugadores como intuitivos y sencillos. El movimiento del jugador se siente fluido, lo cual estaba estipulado en los objetivos del documento. Sin embargo, se podrían pulir determinados aspectos de estos para mejorarlos aún más.

Sobre los enemigos se ha visto que cuando estos interaccionan con el escenario y con otros enemigos es donde suceden la mayoría de los fallos. En general estos enemigos funcionan

correctamente y son diferenciables por el usuario, pero se deberían cambiar o modificar algunas funciones para que se sintieran más justos.

El escenario ha dado resultados satisfactorios, pero debería aumentarse la cantidad de salas plantillas y modificar los lugares de aparición de enemigos para que el espectador perciba cada escenario como uno distinto al anterior.

7.2. Objetivos cumplidos

Con respecto a los objetivos que se definieron al comienzo del documento, podemos confirmar que se han cumplido todos aquellos que se declararon como principales. Se ha diseñado el juego completo entrando en elementos como la trama y el estilo que no eran principales en la implementación. Se han implementado los comportamientos del jugador y la interacción con el usuario. Como hemos visto en los resultados anteriores, también se ha conseguido esa sensación de fluidez en los controles. Se han implementado tres tipos de enemigos distintos con comportamientos totalmente autónomos capaces de desplazarse por un escenario de plataformas. Se ha creado un método capaz de generar escenarios de forma aleatoria de forma consistente y ajustando parámetros como el máximo número de salas que podrían aparecer. También se ha diseñado una interfaz sencilla que muestra la información relevante. Por último, se han comprobado estos elementos tanto con pruebas internas como externas con un grupo de voluntarios.

Sobre los objetivos secundarios del trabajo cabe mencionar que se han creado modelos 3D para el personaje principal y para el Rey alubia. Ambos han sido desarrollados en Blender, el primero no ha podido ser implementado por falta de tiempo y el segundo está introducido dentro del juego, pero no tiene ninguna habilidad ni interacción. El resto de metas no se han implementado, pero serán puntos importantes del trabajo a futuro.

7.3. Líneas futuras de trabajo

Para finalizar, se detallan aquellos puntos que se trabajarán en un futuro. La tarea final es completar el juego y publicarlo de forma gratuita y para eso se necesitan realizar muchas pruebas y ampliaciones al contenido aquí expuesto. Sin embargo, esas tares serán sencillas debido a que ya se poseen los cimientos sobre los que se sustenta el juego.

Lo primero que se realizaría, seria arreglar aquellos errores encontrados en la fase de pruebas y obtener una versión lo más pulida posible. Podría ser útil realizar más pruebas externas sobre esta aplicación con la finalidad de encontrar más fallos. Esto es lo principal actualmente debido a que, si se dejan fallos sin resolver en esta fase del proyecto, más adelante pueden escalarse hasta puntos irremediables. También sería bueno mejorar los controles para

mejorar la comodidad del usuario. Se ha pensado como posibilidad la compatibilidad con mando para poder jugar usando estos dispositivos.

Una vez conseguida la versión definitiva de esta demo se buscaría añadir sonidos y música al juego ya que es un elemento que ayuda mucho a la inmersión y disfrute del juego. Junto con esto también se trabajaría la estética del juego que en esta primera versión se ha dejado de lado por no ser principal.

También sería necesario ampliar los niveles y añadir progresión para el jugador. Todos estos elementos deberían ser diseñados antes de comenzar con otras mejoras para tener claro cuál es el producto al que se quiere llegar. Una idea interesante seria añadir peleas con jefes finales cada nivel para dar la sensación de haber completado un hito importante del juego. Además de esta forma el juego se sentiría diferente y no siempre se trataría de eliminar insectos.

Se deberían añadir más enemigos y mejorar el método de aparición de estos para que los combates se sintieran más justos. Estos enemigos se diseñarían siguiendo la misma filosofía que se siguió para los primeros tres, cada uno debe ser y comportarse de forma distinta al resto para que el jugador pueda diferenciarlos fácilmente.

Por último, se debería completar todos los elementos relacionados con la trama, desde conversaciones con los personajes secundarios hasta cinemáticas en determinados puntos del juego para explicar algunos acontecimientos importantes.

Con estas mejoras, el juego podría considerarse completo y podría publicarse. Nunca se cierra la puerta a posibles cambios o a más contenido ya que el proyecto puede ampliarse de muchas formas distintas y eso deja mucha libertad para crear.

Bibliografía

[1] Mojang Studios, «Minecraft,» 2011.

[2] Riot Games, «League of Legends,» 2009.

[3] Epic Games, «Fortnite,» 2017.

[4] Niantic, «Pokemon Go,» 2016.

[5] InnerSloth, «Among Us,» 2018.

[6] P. McClintock, «2019 Global Box Office Revenue Hit Record $42.5B Despite 4 Percent Dip in U.S.,» The Holliwood Reporter, 10 Enero 2020. [En línea]. Available: https://www.hollywoodreporter.com/news/general-news/2019-global-box-office-hit-record-425b-4-percent-plunge-us-1268600/. [Último acceso: 14 Febrero 2021].

[7] T. Wijman, «The Global Games Market Will Generate $152.1 Billion in 2019 as the U.S. Overtakes China as the Biggest Market,» Newzoo, 18 Junio 2019. [En línea]. Available: https://newzoo.com/insights/articles/the-global-games-market-will-generate-152-1-billion-in-2019-as-the-u-s-overtakes-china-as-the-biggest-market/. [Último acceso: 14 Febrero 2021].

[8] W. Witkowski, «Videogames are a bigger industry than movies and North American sports combined, thanks to the pandemic,» MarketWatch, 22 Diciembre 2020. [En línea]. Available: https://www.marketwatch.com/story/videogames-are-a-bigger-industry-than-sports-and-movies-combined-thanks-to-the-pandemic-11608654990. [Último acceso: 14 Febrero 2021].

[9] PuroMarketing, «La edad de oro de Twitch: logra un crecimiento del 83% en tiempo de visionado en un año,» PuroMarketing, 13 Enero 2021. [En línea]. Available: https://www.puromarketing.com/12/34701/edad-oro-twitch-lograun-crecimiento-delen-tiempo-visionado-ano.html. [Último acceso: 14 Febrero 2021].

[10] «Twitch,» Amazon, Inc, 6 Junio 2011. [En línea]. Available: https://www.twitch.tv/.

[11] S. Barry Cooper y J. van Leeuwen, Alan Turing: His Work and Impact, Elsevier Science, 2013.

[12] W. Higinbotham, «Tennis for two,» 1958.

[13] S. L. Kent, La gran historia de los videojuegos, Nova, 2016.

[14] R. Stanton, A Brief History Of Video Games: From Atari to Virtual Reality, Robinson, 2015.

[15] R. H. Baer, «Television gaming and training apparatus». Estados Unidos Patente 3728480, 17 Abril 1973.

[16] L. A. Lippincott, «Non-volatile reprogrammable ram cartridge». Estados Unidos Patente 4620707, 4 Noviembre 1986 .

[17] Jobs, S. P. Manock, J. C. Hovey, D. A. Kelley y D. M., «Personal computer». Estados Unidos Patente D268584, 12 Abril 1983.

[18] T. Nishikado, «Space Invaders,» Taito Corporation, 1978.

[19] W. Robinett, «Sorcerer's Apprentice,» Atari, Inc, 1983.

[20] UnoCero, «¿Qué son los juegos first, second y third party, y por qué son importantes?,» UnoCero, 5 Julio 2020. [En línea]. Available: https://www.unocero.com/videojuegos/gaming/que-son-los-juegos-first-party-second-party-y-third-party/. [Último acceso: 23 Febrero 2021].

[21] S. Miyamoto, «Super Mario 64,» Nintendo, 1996.

[22] Goto y Teiyu, «Game machine». Japon Patente D367895, 12 Marzo 1996.

[23] Stephens y Adrian, «Video game system and methods for enhanced processing and display of graphical character elements». Estados Unidos Patente 5707288, 13 Enero 1998 .

[24] Miyamoto, N. Shigeru, K. Yasunari, N. Takumi y Satoshi, «Video game system and method with enhanced three-dimensional character and background control due to environmental conditions». Japon Patente 6454652, 24 Septiembre 2002.

[25] F. G. Matas, «Las 20 consolas más vendidas de la historia,» Vandal, 5 Junio 2021. [En línea]. Available: https://vandal.elespanol.com/reportaje/las-20-consolas-mas-vendidas-de-la-historia. [Último acceso: 10 Junio 2021].

[26] Ikeda, I. Akio, K. Kuniaki, T. Ryoji, U. Genyo y Masahiro, «Video game system with wireless modular handheld controller». Estados Unidos Patente 8313379, 20 Noviembre 2012.

[27] G. Lucas, Dirección, *La guerra de las galaxias.* [Película]. Estados Unidos: Lucasfilm, 1977.

[28] Halfbrick, «Fruit Ninja,» Halfbrick, 2010.

[29] King, «Candy Crush Saga,» King, 2012.

[30] RTVE, «Microsoft compra Mojang, el estudio responsable de 'Minecraft', por 1.900 millones de euros,» RTVE, 15 Septiembre 2014. [En línea]. Available: https://www.rtve.es/noticias/20140915/microsoft-compra-mojang-estudio-responsable-minecraft-1900-millones-euros/1011860.shtml. [Último acceso: 2 Marzo 2021].

[31] E. Barone, «Stardew Valley,» Eric Barone, 2016.

[32] Matt Makes Games, «Celeste,» Matt Makes Games, 2018.

[33] Nomada Studio, «Gris,» Nomada Studio, 2018.

[34] A. García, «El videojuego catalán GRIS recibe tres nominaciones para los The Game Awards,» La Vanguardia, 21 Noviembre 2019. [En línea]. Available: https://www.lavanguardia.com/videojuegos/20191121/471778812901/gris-the-game-awards-nominaciones-nomada-studios.html. [Último acceso: 5 Marzo 2021].

[35] Nintendo Research & Development 1, «Mario Bros,» Nintendo, 1983.

[36] Team Cherry, «Hollow Knight,» Team Cherry, 2017.

[37] Michael, G. Toy, W. y. Ken y Arnold, «Rogue,» Epyx, 1980.

[38] E. McMillen, «The binding of Isaac,» Edmund McMillen, 2011.

[39] Derek, A. Yu y Hull, «Spelunky,» Mossmouth, 2008.

[40] Supergiant Games, «Hades,» Supergiant Games, 2018.

[41] Motion Twin, «Dead Cells,» Motion Twin, 2017.

[42] P. R. Sean Tracy, Cryengine 3 Game Development: Beginner's Guide, Packt Publishing, 2012.

[43] Crytek, «Crisis 2,» Electronic Arts, 2011.

[44] F. Siddi, «Blender by the Numbers – 2019,» Blender, 10 Febrero 2020. [En línea]. Available: https://www.blender.org/press/blender-by-the-numbers-2019/. [Último acceso: 25 Marzo 2020].

[45] S. de Blas de Miguel, «Diseño y desarrollo de una aplicación enUnity3D para la enseñanza deprogramación, Trabajo de Fin de Grado,» 2019. [En línea]. Available: http://hdl.handle.net/10016/30261.

[46] E. G. González Chacón, «Desarrollo de un videojuego de rolestratégico con LibGDX, Trabajo de Fin de Grado,» 2018. [En línea]. Available: http://hdl.handle.net/10016/29589.

[47] D. Rivero Martínez, «Desarrollo de un videojuego 2.5D con Unity, Trabajo de Fin de Grado,» 2017. [En línea]. Available: http://hdl.handle.net/2445/119104.

[48] J. G. Gomilla Salas, «+ 100 horas - Curso Completo Videojuegos con Unity 2018 y C#,» Udemy, Noviembre 2020. [En línea]. Available: https://www.udemy.com/course/unity-2018/. [Último acceso: 20 Febrero 2021].

[49] O. Fernandez, «Blender 2.9X: Modelado y texturizado enfocado a videojuegos,» Udemy, Junio 2020. [En línea]. Available: https://www.udemy.com/course/blender-27x-modelado-y-texturizado-enfocado-a-videojuegos/. [Último acceso: 26 Febrero 2021].

[50] L. Gonzalez, «How to Write a Game Design Document,» Gamasutra, 26 Julio 2016. [En línea]. Available: https://www.gamasutra.com/blogs/LeandroGonzalez/20160726/277928/How_to_Write_a_Game_Design_Document.php. [Último acceso: 14 Marzo 2021].

[51] R. Alonso, «¿Por qué la mayoría de juegos usan las teclas WASD del teclado?,» Hardzone, 14 Septiembre 2020. [En línea]. Available: https://hardzone.es/tutoriales/componentes/teclas-wasd-teclado-juegos/. [Último acceso: 17 Junio 2021].

[52] Ben Cowley, Darryl Charles, Michaela Black, and Ray Hickey. 2008. Toward an understanding of flow in video games. Comput. Entertain. 6, 2, Article 20 (April/June 2008), 27 pages. [En linea] Available: https://doi.org/10.1145/1371216.1371223

[53] RTVE, «¿Qué probabilidad hay de que me toque la lotería?: Estas son las estadísticas,» RTVE, 09 Abril 2021. [En línea]. Available: https://www.rtve.es/rtve/20210409/loteria-probabilidad-toque-estadistica/2084973.shtml. [Último acceso: 10 Abril 2021].

[54] Ubisoft, «Playtest,» Ubisoft, [En línea]. Available: https://playtest.ubisoft.com/montreal/en-US/. [Último acceso: 13 Mayo 2021].

[55] Ideed, «¿Cuánto se gana en España de Diseñador/a ux?,» [En línea]. Available: https://es.indeed.com/career/diseñador-ux/salaries?salaryType=YEARLY&from=careers_serp. [Último acceso: 10 Junio 2021].

[56] Unity, «Planes de pago para Unity,» Unity, [En línea]. Available: https://store.unity.com/es/compare-plans. [Último acceso: 2 Junio 2021].

[57] Gobierno de España, «Agencia Estatal Boletin Oficial del esatado, Ley Organica 3/2018,» 5 Diciembre 2018. [En línea]. Available: https://www.boe.es/buscar/doc.php?id=BOE-A-2018-16673. [Último acceso: 26 Marzo 2021].

[58] ModeloContrato, «Contrato de confidencialidad: Modelo,» [En línea]. Available: https://www.modelocontrato.net/contrato-de-confidencialidad-modelo.html. [Último acceso: 17 Mayo 2021].

Anexo A: Formulario de inscripción para beta-tester

Inscripción para Beta-Tester

Si quieres ser Beta-Tester para el proyecto Beans Out rellena este formulario con tus datos. Así podremos ponernos en contacto contigo.

*Obligatorio

1. Nombre *

2. Apellidos *

3. Email *

Ley de protección de datos

Con respecto a la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. Sus datos solo se almacenarán de forma temporal y con fines académicos.

Google Formularios

Anexo B: Contrato de confidencialidad

CONTRATO DE CONFIDENCIALIDAD

CONTRATO DE CONFIDENCIALIDAD QUE CELEBRAN POR UNA PARTE _____. REPRESENTADA POR _____ Y POR LA OTRA PARTE D. _____ AL TENOR DE CLAUSULAS SIGUIENTES:

CLAUSULAS

PRIMERA. Las partes se obligan a no divulgar a terceras partes, la "Información Confidencial", que reciban de la otra, y a darle a dicha información el mismo tratamiento que le darían a la información confidencial de su propiedad.

Para efectos del presente convenio "Información Confidencial" comprende toda la información divulgada por cualesquiera de las partes ya sea en forma oral, visual, escrita, grabada en medios magnéticos o en cualquier otra forma tangible y que se encuentre claramente marcada como tal al ser entregada a la parte receptora.

SEGUNDA. La parte receptora se obliga a mantener de manera confidencial la "Información Confidencial" que reciba de la parte divulgante y a no darla a una tercera parte.

TERCERA. La parte receptora se obliga a no divulgar la "Información Confidencial" a terceros, sin el previo consentimiento por escrito de la parte divulgante.

CUARTA. Las partes convienen que en caso que la parte receptora incumpla parcial o totalmente con las obligaciones a su cargo derivadas del presente contrato, la parte receptora será responsable de los daños y perjuicios que dicho incumplimiento llegase a ocasionar a la parte divulgante.

QUINTA. No obstante, lo dispuesto en contrario en este convenio ninguna parte tendrá obligación de mantener como confidencial cualquier información:

1. Que previa a su divulgación fuese conocida por la parte receptora, libre de cualquier obligación de mantenerla confidencial, según se evidencie por documentación en su posesión;

2. Que sea desarrollada o elaborada de manera independiente por o de parte del receptor o legalmente recibida, libre de restricciones, de otra fuente con derecho a divulgarla;

3. Que sea o llegue a ser del dominio público, sin mediar incumplimiento de este convenio por la parte receptora; y

4. Que se reciba de un tercero sin que esa divulgación quebrante o viole una obligación de confidencialidad.

SEXTA. La vigencia del presente convenio será indefinida y permanecerá vigente mientras exista relación comercial entre ambas partes.

SEPTIMA. Este convenio constituye el acuerdo total entre las partes respecto a dicha información confidencial y substituye a cualquier otro entendimiento previo, oral o escrito, que haya existido entre las partes.

OCTAVA. Este convenio solamente podrá ser modificado mediante consentimiento de las partes, otorgado por escrito.

NOVENA. Para la interpretación y cumplimiento del presente contrato, las partes se someten a la jurisdicción de las leyes y tribunales de España, Madrid, renunciando desde ahora a cualquier otro fuero que pudiera corresponderles por razón de sus domicilios presentes o futuros o por cualquier otra causa.

Enteradas las partes del contenido y alcance del presente contrato, lo firman a fecha _____.

Firma. Firma.

D. _____ D. _____

Anexo C: Mail de contacto para fase de pruebas externas

Buenas tardes

Es de mi agrado comunicarle que se ha abierto el plazo de prueba para el juego Beans Out para el que usted solicitó participar. En el siguiente correo encontrará toda la información relevante para este proceso. Lea atentamente y pregunte cualquier duda que tenga.

Adjuntado a este correo tiene un fichero con las instrucciones correspondientes. Por favor, léalo antes de realizar ninguna acción.

Para descargar el juego deberá acceder a este enlace Click aquí y descargar la versión que usted necesite, ya sea Windows o Linux. Para descomprimir estos ficheros y por motivos de seguridad será necesaria una contraseña, la cual está en el documento de instrucciones.

Por último, como habrá leído en el documento de instrucciones, existen dos formularios que usted podrá rellenar durante la realización de las pruebas. Ambos se pueden encontrar en los siguientes enlaces. Recomendamos guardar los enlaces en algún lugar fácilmente accesible para poder acceder a ellos de forma sencilla mientras se prueba la aplicación.

- Formulario de satisfacción
- Formulario de error

Recordamos que estos formularios solo estarán disponibles los próximos 10 días hábiles.

Muchas gracias por su atención.

Un saludo, El equipo de desarrollo de Beans Out.

Anexo D: Hoja de instrucciones para beta-testers

Instrucciones para Beta-Testers

En el siguiente documento se detallan las instrucciones para el proceso de testeo de la aplicación Beans Out. Por favor, léase este documento con atención y consulte cualquier duda que pueda tener con la persona que le envió este documento.

Primero que todo, agradecer su colaboración con este proyecto, su aportación es de gran ayuda. Desde el equipo creativo de Beans Out, gracias.

Descripción del juego

Comenzaremos con una pequeña descripción de la aplicación. Beans Out es actualmente un juego donde la tarea final es eliminar una cantidad de enemigos en un mapa aleatorio. Una vez usted ejecute la aplicación encontrará un menú para comenzar a jugar. Una vez empezada la partida debe conocer esta información:

Controles:

- **A, D** - Mover al personaje a la izquierda o derecha respectivamente
- **W** - Saltar
- **Q** - Disparar
- **E** **-** Dash (Carrera)
- **Esc** - Menú de pausa

En la pantalla se muestra la información de la vida y los enemigos restantes del nivel. La misión principal del juego es acabar con todos los enemigos del nivel sin morir.

Formularios

Para almacenar y facilitar la comunicación y el feedback entre ustedes y el equipo de desarrollo, podrán preguntar cualquier duda por medio de correo electrónico a la misma persona que les envió este documento, o al siguiente correo: miguelangeltorrijos299@gmail.com. Pero, además de este sistema, se pondrán a disposición de los testers, dos formularios que estos deberán rellenar dependiendo de la situación.

El primero de ellos es el formulario de satisfacción, este formulario solo acepta una respuesta por usuario por lo que este debe responderlo una vez ya ha realizado su trabajo de pruebas del juego al completo. En este formulario se preguntarán datos relacionados con la experiencia de juego.

El segundo, es un formulario general para errores encontrados en la aplicación. Este formulario puede rellenarse todas las veces que haga falta. En cuanto usted encuentre un error o algún comportamiento que considere anormal de nuestra aplicación, deberá documentarlo en este formulario de la forma más detallada posible. Este formulario añade categorías para englobar el fallo, pero si no se considera que ninguna de ellas engloba el error encontrado, pueden añadirse una propia manualmente.

Se pide que estos formularios sean rellenados lo más extenso y detallado posible explicando cualquier detalle que se considere importante. Ambos formularios estarán disponibles durante los siguientes 10 días hábiles desde que este documento fue enviado.

Pruebas a realizar

Como encargados de pruebas, deberéis probar el juego en su totalidad para intentar encontrar y documentar el máximo número de fallos posible. Por eso se han establecido las siguientes reglas en cuanto a las pruebas del juego.

- Usted, como usuario deberá completar, como mínimo, 50 partidas (i.e. eliminar a todos los enemigos del nivel al menos 50 veces).

- Dedicar mínimo 180 minutos en total a la aplicación.

- Analizar consistentemente lo que se muestra en pantalla mientras se juega para encontrar posibles fallos.

- Rellenar el formulario de satisfacción al final de las pruebas, ya que no se aceptará más de una respuesta por usuario.

- Rellenar el formulario de errores si es que se encuentra algún error en la aplicación.

Acceso a la aplicación

Toda la información sobre como acceder y obtener la aplicación esta detallada en el correo donde se adjunta este documento. La contraseña para descomprimir el fichero es: “*BeansOut_1a”* sin comillas y con las mayúsculas y minúsculas como se determinan.

Por motivos de seguridad, nos reservamos el derecho legal de llevar a cabo acciones legales contra cualquiera que distribuya información o la propia aplicación de forma ilegal. Con bases en el documento legal de confidencialidad firmado con nosotros.

FAQ (Preguntas frecuentes)

- ¿No puedo acceder a los ficheros de la aplicación o no puedo descomprimir el fichero?

Para cualquier error con el acceso a los materiales, por favor contactar con miguelangeltorrijos299@gmail.com explicando el error. Se intentará solucionar lo más rápido posible.

- ¿Qué ocurre si no puedo completar las pruebas en el plazo de 10 días hábiles?

El formulario dejará de estar disponible por lo que no podrás rellenarlo. Simplemente no recibiremos tu información y será como si no hubieras participado. Si quieres participar, te aconsejamos que realices las pruebas lo antes posible y no esperes a los últimos días.

- Una vez finalizadas las pruebas, ¿qué pasa con los ficheros del juego?

Esos archivos puedes quedártelos siempre y cuando respetes la propiedad intelectual y no publiques nada de ellos. Tampoco se permite el intento de piratería o ingeniería inversa para obtener información de la base del juego.

- ¿Para qué van a ser usados los datos que proporcione?

En ningún caso se le pedirá información personal más allá de la que ya aportó en la propia inscripción para este proceso. Cualquier dato que aporte será usado enteramente para mejorar el producto.

Anexo E: Formulario de error

Formulario de error

En este formulario se detallan las especificaciones del error encontrado

*Obligatorio

1. Describe en una frase el error encontrado *

2. En que categoría englobarías este error (puedes elegir varias) *

   Selecciona todos los que correspondan.

   - [ ] Enemigos
   - [ ] Escenario
   - [ ] Interfaz
   - [ ] Controles
   - [ ] Personaje
   - Otro: [ ] _____

3. ¿Has podido continuar jugando después del error? (Si te ha impedido acabar o ganar la partida) *

   Marca solo un óvalo.

- [ ] Sí
- [ ] No

4. Como de grave catalogarías dicho error *

Marca solo un óvalo.

	1	2	3	4	5	
Poco grave						Muy grave

5. Describe de la forma más detalla posible este error *

6. Comentarios extra sobre el error que no se hayan detallado anteriormente

Google Formularios

Anexo F: Formulario de satisfacción

Formulario de satisfacción

En este formulario, se detallan preguntas sobre la sensación de juego y opiniones sobre la aplicación. Debe rellenarse una vez se han hecho todas las pruebas ya que solo se acepta una respuesta por persona.

*Obligatorio

1. Correo *

_____

2. Nombre *

_____

3. Apellidos *

_____

Controles y manejo

Formulario de satisfacción

4. Los controles del jugador son intuitivos *

Marca solo un óvalo.

	1	2	3	4	5	
Muy en desacuerdo	◯	◯	◯	◯	◯	Muy de acuerdo

5. Los controles del jugador son sencillos y cómodos de usar *

Marca solo un óvalo.

	1	2	3	4	5	
Muy en desacuerdo						Muy de acuerdo

6. El juego ha respondido bien a mis controles *

Marca solo un óvalo.

	1	2	3	4	5	
Muy en desacuerdo						Muy de acuerdo

7. Comentarios personales sobre los controles *

Formulario de satisfacción

Enemigos y dificultad

8. El comportamiento de los enemigos me ha parecido: *

Marca solo un óvalo.

Malo (Arbitrario)						Bueno
	1	2	3	4	5	

9. El comportamiento de las moscas me ha parecido: *

Marca solo un óvalo.

	1	2	3	4	5	
Malo						Bueno

10. El comportamiento de las hormigas me ha parecido: *

Marca solo un óvalo.

	1	2	3	4	5	
Malo						Bueno

11. El comportamiento de las arañas me ha parecido: *

Marca solo un óvalo.

	1	2	3	4	5	
Malo						Bueno

12. La dificultad del juego me ha parecido: *

Marca solo un óvalo.

	1	2	3	4	5	
Mala						Buena

13. Comentarios personales sobre los enemigos y la dificultad *

Escenarios

Formulario de satisfacción

14. Cada partida se ha sentido distinta a la anterior *

Marca solo un óvalo.

	1	2	3	4	5	
Muy en desacuerdo						Muy de acuerdo

15. Desplazarse por el escenario ha sido entretenido *

Marca solo un óvalo.

	1	2	3	4	5	
Muy en desacuerdo						Muy de acuerdo

16. Comentarios personales sobre los escenarios *

Comentarios finales

Formulario de satisfacción

17. Comentarios personales generales sobre el juego *

18. Comentarios personales concretos de algún tema que no se haya detallado anteriormente

Google Formularios

Anexo G: Hoja de cálculo para el Gantt estimado

	Gantt estimado			
	Actividad	Fecha inicio	duracion	fecha fin
	Fase 1: Planteamiento inicial del problema			
	Preparacion sobre Unity3D y Blender	11-ene.	26	6-feb.
	Analisis y estudio de las caracteristicas del videojuego	11-ene.	36	16-feb.
	Estudio del estado del arte	29-ene.	20	18-feb.
	Fase 2: Diseño de la aplicación			
	Diseño del personaje	15-feb.	7	22-feb.
	Diseño de los enemigos	20-feb.	10	2-mar.
	Diseño del escenario	27-feb.	13	12-mar.
	Diseño de elementos secundarios	2-mar.	5	7-mar.
	Fase 3: Implementacion de la solucion			
	Implementacion de un escenario de pruebas	5-mar.	2	7-mar.
	Creacion del personaje	6-mar.	15	21-mar.
	Creacion de los enemigos	19-mar.	20	8-abr.
	Creacion de salas para el generador de escenarios	1-abr.	11	12-abr.
	Implementacion del algoritmo generador de escenarios	10-abr.	7	17-abr.
	Unión de los elementos en la aplicación final	17-abr.	12	29-abr.
	Fase 4: Pruebas y analisis de resultados			
	Realización de pruebas finales	29-abr.	7	6-may.
	Creacion de los formularios y busqueda de candidatos	6-may.	2	8-may.
	Proceso de pruebas con los candidatos externos	8-may.	14	22-may.
	Fase 5: Redacción			
	Recopilacion e interpretacion de los datos obtenidos	24-may.	4	28-may.
	Busqueda de recursos de apoyo	1-feb.	9	10-feb.
	Redaccion del documento	10-feb.	120	10-jun.
	Estudio de la bibliografia	9-jun.	2	11-jun.
	Revision del documento	11-jun.	3	14-jun.

Anexo H: Hoja de cálculo para el Gantt real

	Gantt real			
	Actividad	Fecha inicio	duracion	fecha fin
	Fase 1: Planteamiento inicial del problema			
	Preparacion sobre Unity3D y Blender	11-ene.	20	31-ene.
	Analisis y estudio de las caracteristicas del videojuego	11-ene.	46	26-feb.
	Estudio del estado del arte	26-ene.	18	13-feb.
	Fase 2: Diseño de la aplicación			
	Diseño del personaje	20-feb.	3	23-feb.
	Diseño de los enemigos	23-feb.	7	2-mar.
	Diseño del escenario	25-feb.	8	5-mar.
	Diseño de elementos secundarios	1-mar.	5	6-mar.
	Fase 3: Implementacion de la solucion			
	Implementacion de un escenario de pruebas	7-mar.	1	8-mar.
	Creacion del personaje	10-mar.	10	20-mar.
	Creacion de los enemigos	20-mar.	24	13-abr.
	Creacion de salas para el generador de escenarios	3-abr.	8	11-abr.
	Implementacion del algoritmo generador de escenarios	11-abr.	5	16-abr.
	Unión de los elementos en la aplicación final	19-abr.	17	6-may.
	Fase 4: Pruebas y analisis de resultados			
	Realización de pruebas finales	6-may.	7	13-may.
	Creacion de los formularios y busqueda de candidatos	1-may.	14	15-may.
	Proceso de pruebas con los candidatos externos	16-may.	15	31-may.
	Fase 5: Redacción			
	Recopilacion e interpretacion de los datos obtenidos	31-may.	2	2-jun.
	Busqueda de recursos de apoyo	10-mar.	13	23-mar.
	Redaccion del documento	20-mar.	80	8-jun.
	Estudio de la bibliografia	8-jun.	2	10-jun.
	Revision del documento	10-jun.	4	14-jun.

Anexo I: Hoja de cálculo para seguimiento de horas dedicadas

	Fase 1	Fase 2	Fase 3	Fase 4	Fase 5
Semana 1	11				
Semana 2	10				
Semana 3	12				
Semana 4	10				
Semana 5	10				
Semana 6	6	6			
Semana 7	5	12			
Semana 8		24	3		
Semana 9			9		10
Semana 10			10		11
Semana 11			13		6
Semana 12			10		0
Semana 13			14		5
Semana 14			5		15
Semana 15			10		10
Semana 16			10	5	3
Semana 17			5	20	0
Semana 18				15	6
Semana 19				5	20
Semana 20				5	27
Semana 21					21
Semana 22					14

	Total
Fase 1	64
Fase 2	42
Fase 3	89
Fase 4	50
Fase 5	148
Total	393